



CPG Template for S7-1500

SIMATIC CPG Template

<https://support.industry.siemens.com/cs/ww/en/view/109475572>

Siemens
Industry
Online
Support



Legal information

Use of application examples

Application examples illustrate the solution of automation tasks through an interaction of several components in the form of text, graphics and/or software modules. The application examples are a free service by Siemens AG and/or a subsidiary of Siemens AG ("Siemens"). They are non-binding and make no claim to completeness or functionality regarding configuration and equipment. The application examples merely offer help with typical tasks; they do not constitute customer-specific solutions. You yourself are responsible for the proper and safe operation of the products in accordance with applicable regulations and must also check the function of the respective application example and customize it for your system.

Siemens grants you the non-exclusive, non-sublicensable and non-transferable right to have the application examples used by technically trained personnel. Any change to the application examples is your responsibility. Sharing the application examples with third parties or copying the application examples or excerpts thereof is permitted only in combination with your own products. The application examples are not required to undergo the customary tests and quality inspections of a chargeable product; they may have functional and performance defects as well as errors. It is your responsibility to use them in such a manner that any malfunctions that may occur do not result in property damage or injury to persons.

Disclaimer of liability

Siemens shall not assume any liability, for any legal reason whatsoever, including, without limitation, liability for the usability, availability, completeness and freedom from defects of the application examples as well as for related information, configuration and performance data and any damage caused thereby. This shall not apply in cases of mandatory liability, for example under the German Product Liability Act, or in cases of intent, gross negligence, or culpable loss of life, bodily injury or damage to health, non-compliance with a guarantee, fraudulent non-disclosure of a defect, or culpable breach of material contractual obligations. Claims for damages arising from a breach of material contractual obligations shall however be limited to the foreseeable damage typical of the type of agreement, unless liability arises from intent or gross negligence or is based on loss of life, bodily injury or damage to health. The foregoing provisions do not imply any change in the burden of proof to your detriment. You shall indemnify Siemens against existing or future claims of third parties in this connection except where Siemens is mandatorily liable.

By using the application examples you acknowledge that Siemens cannot be held liable for any damage beyond the liability provisions described.

Other information

Siemens reserves the right to make changes to the application examples at any time without notice. In case of discrepancies between the suggestions in the application examples and other Siemens publications such as catalogs, the content of the other documentation shall have precedence.

The Siemens terms of use (<https://support.industry.siemens.com>) shall also apply.

Security information

Siemens provides products and solutions with industrial security functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial security concept. Siemens' products and solutions constitute one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the Internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place. For additional information on industrial security measures that may be implemented, please visit <https://www.siemens.com/industrialsecurity>.

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Security RSS Feed at: <https://www.siemens.com/industrialsecurity>.

Table of contents

Legal information	2
1 Introduction	5
1.1 Overview	5
1.2 Mode of operation	7
1.2.1 CPG Template Foundations	7
1.2.2 PackML State Machine Model	7
1.2.3 PackML Mode Manager	9
1.2.4 PackML Standard Interface (PackTags)	9
1.2.5 S88 Make2Pack (modular design)	10
1.2.6 CPG Template Event Manager	11
1.2.7 LPMLV30 Function Blocks	12
1.3 Understanding the CPG Template	12
1.3.1 Make2Pack Division of Functions	13
1.3.2 What is meant by "Summation"?	15
1.3.3 Overview of Programming Tasks	16
1.4 Components used	19
2 Engineering	20
2.1 CPG Template Global Data Blocks	20
2.1.1 Alarm	20
2.1.2 Warning	21
2.1.3 Status	21
2.1.4 HMI_Data	22
2.1.5 HMI_Data_Custom	23
2.1.6 PackML	24
2.1.7 PackTags	24
2.2 CPG Template Configuration Data Blocks	28
2.2.1 AlarmCfg	28
2.2.2 WarningCfg	29
2.2.3 StatusCfg	29
2.2.4 MessagePrefix	30
2.2.5 PackMLCfg	30
2.3 Unit Machine Blocks	32
2.4 Equipment Module Blocks	34
2.5 Control Module Blocks	36
2.5.1 Control Module Programming Examples	36
2.6 The EM Data Block	40
2.7 LMPLV30 Supporting Blocks	43
2.8 CPG Supporting Blocks	44
2.8.1 CPG_Event	44
2.8.2 CPG_EventManager	46
2.8.3 CPG_EventSummation	49
2.8.4 CPG_EventSummationBegin	52
2.8.5 CPG_EventSummationEnd	53
2.8.6 CPG_EventSortFilter	54
2.8.7 CPG_AlarmHistory	56
2.8.8 CPG_AlarmMove	57
2.8.9 CPG_OEE	58
2.8.10 CPG_ModuleInitialize	59
2.8.11 CPG_MsgScroll	60
2.8.12 CPG_ArrayLength	61
2.9 CPG Template Data types	62
2.9.1 CPG_typeEvent	63
2.9.2 CPG_typeEventCfg	63
2.9.3 CPG_typeEventStatus	63

2.9.4	CPG_typeEventSummation	65
2.9.5	CPG_typeMessagePrefix.....	66
2.9.6	CPG_typePackML_ModuleSts.....	66
2.9.7	CPG_typeParameterDINT	67
2.9.8	CPG_typeParameterReal	67
2.9.9	CPG_typeParameterString	67
2.9.10	CPG_typeProdCountDINT	68
2.9.11	CPG_typeSortFilter.....	68
2.10	CPG Template Constants	69
3	HMI	72
3.1	Screen layout	72
3.2	Navigation	73
3.2.1	First level (primary navigation)	73
3.2.2	Second level (secondary navigation).....	74
3.2.3	Third level (optional)	74
3.3	Numbering and naming screens	74
3.4	Adjusting the Navigation	77
3.4.1	Adjusting the number of second level topic buttons	77
3.4.2	Screen Templates	77
3.4.3	Adjusting the second level topic button screen activation	78
3.4.4	Adjusting the second level topic button text.....	79
3.5	Functions/screens and their integration.....	81
3.5.1	HMI Tags.....	81
3.5.2	User Administration	81
3.5.3	Scripts.....	83
3.5.4	Text List.....	83
3.5.5	Graphic lists.....	84
3.6	Preconfigured HMI Screens	85
3.6.1	Machine Control	85
3.6.2	Settings.....	89
3.6.3	Alarms and Events	91
3.6.4	Diagnostics.....	93
3.6.5	Formats.....	95
3.7	Multi-language.....	95
4	Extending the template	96
4.1	Adding a new Equipment Module.....	96
4.1.1	Adding the EM into the module summation	99
4.1.2	Adding the EM into the event summation	100
4.2	Adding a new Control Module to an existing EM	100
4.2.1	Creating supporting tags for the new CM	103
5	Additional information	108
5.1	Tips and Recommendations	108
5.1.1	Template Modifications	108
5.1.2	Global Memory	108
5.1.3	Number of Event Languages	108
6	Appendix.....	109
6.1	Service and support.....	109
6.2	Application Support.....	110
6.3	Links and literature	110
6.4	Change documentation.....	110

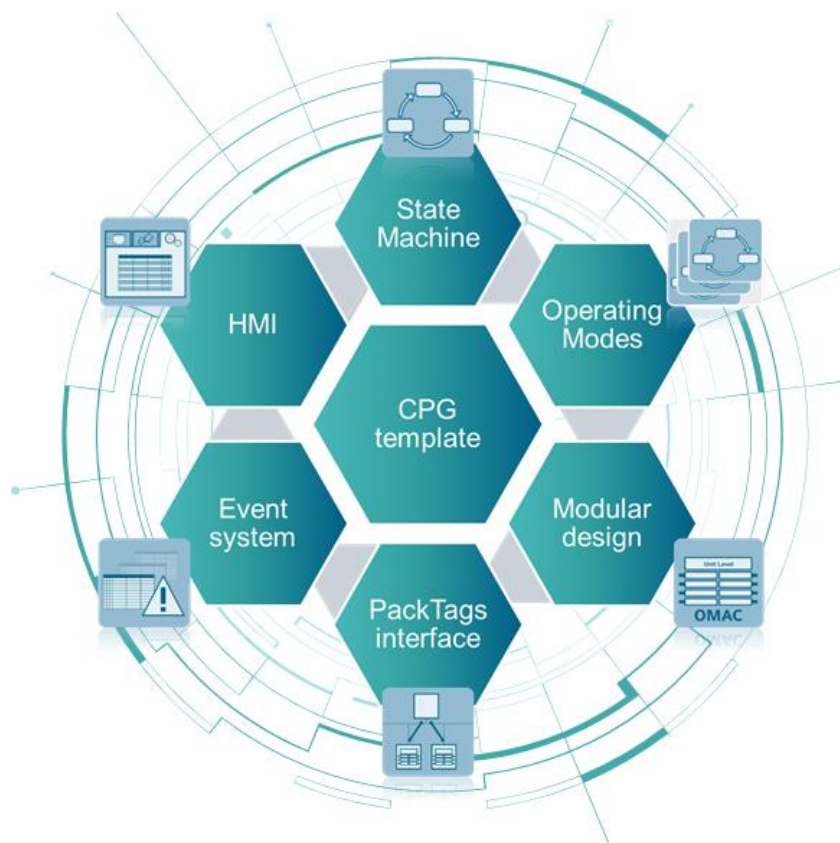
1 Introduction

1.1 Overview

State-of-the-art packaging machines and lines are extremely versatile and must provide high throughput rates with the highest possible degree of flexibility. Reducing the total cost of ownership and investment security are also crucial issues. Big end customers expect machines from different vendors to have the same usability and diagnostics. In addition, a standardized access to the machine data from upper level systems must be guaranteed to be able to e.g. analyze product flow, acquire machine operating hours or calculate performance data. The key to achieving these objectives is adopting international standards.

This application template, known as the SIMATIC CPG Template, can be used as a basis to implement an OMAC PackML compliant project and provides you with the tested code with clearly defined interfaces, including an HMI template.

The SIMATIC CPG template combines the PackML guidelines for mode and states management and standard interface (PackTags) together with the ISA-S88 Make2Pack modularity concept and an integrated alarming concept to provide you with a ready to use solution for your OMAC compliant projects.



Key facts

The CPG template is an OMAC PackML compliant template project which includes the following features:

- Based upon open source template from global CPG companies. User logic based on Ladder Logic (LAD) while the supporting logic is based on Structured Control Language (SCL).
- Built upon Siemens OMAC Mode and State management implementation (based on OMAC PackML Machine and Unit States library based upon TR88.00.02-2015)
- Integrated HMI Template (CPG philosophy)
- Integrated Alarms, Warnings and Status functionality (Code and HMI)
- Integrated PackML command and status philosophy (Code)
- Modular structure (based on ISA-S88 Make2Pack) including Unit, equipment modules (EMs) and control modules (CMs) levels
- OEE calculation (based upon TR88.00.02-2008) and visualization.
- Multi-lingual support
- Available for SIMATIC the S7-1500(T), adaptation for S7-1200 possible

Benefits

- Simple, intuitive
- It is based on a well-known template concept for CPG Industry
- It increases understanding and readability of the code
- Code designed for your machine and your purpose: start simple, add features/complexity as you need it
- Robust and tested code
- Code provided free of charge to the OMAC community
- Proven on simple and complex machines
- Reduced mean time to repair (MTTR).

NOTE

Knowledge about the contents of the ISA and OMAC documentations and our *OMAC PackML V3 Machine and Unit States* library is an advantage when it comes to understanding the solution described in this documentation. Siemens recommends that all customers purchase and download the 'ANSI/ISA-TR88.00.02-2015, Machine and Unit States: An implementation example of ANSI/ISA-88.00.01' standard.

The goal of the document is to describe:

- how the template structure is implemented
- all public blocks of the block library
- the functionality implemented through these blocks
- how to extend the template
- the HMI template structure.

See also:

ISA website: (<https://www.isa.org/>)

OMAC website: (<http://omac.org/>)

OMAC PackML V3 Machine and Unit States library - SIOS entry: (49970441)

1.2 Mode of operation

1.2.1 CPG Template Foundations

The ANSI/ISA-88.00.01-2015 standard is widely recognized and adopted for all the possible applications across a plant, including continuous, batch, and discrete processes. A step forward into the discrete applications standardization has been done by ISA (International Society of Automation) with the ANSI/ISA-TR88.00.02.2015 technical report, which demonstrates how to apply the general ISA-88 standard concepts to automated machines.

In this context, the OMAC PackML Unit/Machine Implementation Guide is a Best Practice Recommendation based on the ISA-TR88 technical report.

OMAC (Organization for Machine Automation and Control) is the global organization for automation and manufacturing professionals that is dedicated to supporting the machine automation and operational needs of manufacturing.

The objective of this organization is to define the necessary harmonized regulations and standard guidelines to reduce the development and delivery times, use the existing resources more efficiently and at the same time increase the profitability.

OMAC's Packaging Workgroup develops guidelines that simplify packaging line customization and integration. Members collaborate to create standards that improve machine to machine integration and packaging line to business system performance.

PackML (Packaging Machine Language) is therefore an industry technical standard for the control of packaging machines.

The primary objective of PackML is to enable easy integration of machines, and to bring a common "look and feel" to all machines that make up a Packaging Line, but it can be used also for other types of discrete processes.

OMAC PackML describes the OMAC mode and states management (unit mode manager and state machine). In addition, PackML defines standardized variable structures for the cross-machine coupling between machine controllers and to higher-level HMI, MES or Enterprise systems.

The SIMATIC CPG Template combines the PackML guidelines for mode and states management and standard interface (PackTags) together with the ISA-S88 Make2Pack modularity concept and an integrated alarming concept to provide you with a ready to use solution for your OMAC compliant projects.

1.2.2 PackML State Machine Model

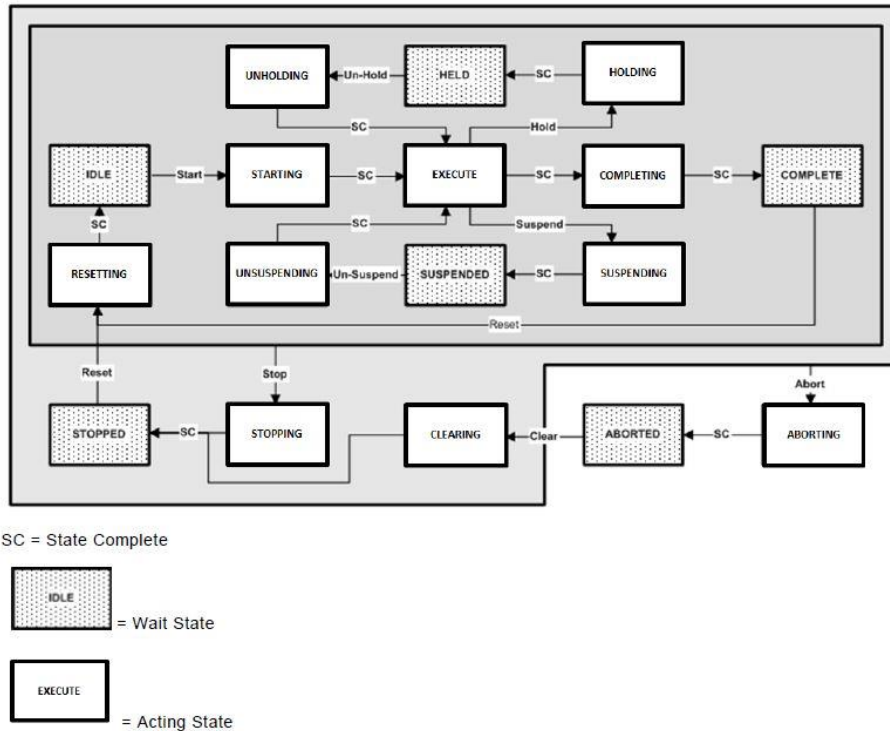
A State Machine is a control model that is defined by fixed operating states with defined requirements needed to transition from one state to another. The machine's operation can be summarized by providing the current state. State machines help operators understand what the machine is doing and what is needed in order to move into a more productive state. When writing the machine logic, a paradigm to keep in mind is "states drive outputs". If the machine can perform a function, consider the state(s) in which that function is active. The machine logic should be executed only if the machine is in those states. Also consider what conditions prompt a state transition. For example:

- The machine must execute the homing procedure of its axes. For this machine, this action should only take place in the Resetting state. Therefore, in the

machine code, the homing logic is only executed when the state is equal to Resetting.

- The machine must ramp down the axes to zero speed in the states Stopping, Suspending, Holding, and Aborting. The ramp down command is only sent to the drives if the state is one of the above.
- The machine should transition into a new state if the operator pushes the start, stop, or emergency stop buttons. Each button is connected to the appropriate command for the state machine (start, stop, and abort respectively).

Figure 1-1: ANSI/ISA TR88.00.02-2015 State machine diagram



The PackML State Machine contains a set of 17 predefined states, broken into two categories. Waiting states are stable situations for the unit/machine. The machine remains in a wait state until receiving a command to enter the next state. Waiting states include Idle, Stopped, Aborted, Suspended, Held, Completed, and Execute. Acting states are where the machine must perform some transient actions. The machine performs the functions, and then automatically moves onto the next (waiting) state when it is finished. Acting states end in “ing”, and include Resetting, Starting, Clearing, Stopping, Aborting, Suspending, Unsuspending, Holding, Unholding, and Completing. Not all states have to be used, but none can be added. Additionally, no transitions not included in the state diagram are permitted. If the machine receives a command which is invalid (such as attempting to start a machine in Execute), the command is ignored. The minimum set of states that must be implemented includes the following: Idle, Execute, Stopped, Aborted. In each state, logic can be executed until a state transition occurs. There are four ways in which a state transition can occur:

- The machine receives a control command from an operator action
- The machine receives a control command from a higher level or external source

- The logic in an Acting state completes normally and the machine states moves to the next Waiting state. In the state machine this is accomplished by setting a State Complete (SC) bit
- A PackML Alarm triggers, moving the machine into a predefined state (usually Stopping, Aborting, Suspending, or Holding).

1.2.3 PackML Mode Manager

A mode is an ordered subset of the PackML States. It is used to define a machine operational mode. The following are examples of control modes specified by PackML

- Production (mandatory): this is the production routine mode. When this mode is selected, the machine executes relevant logic in response to commands which are mainly coming from External Systems or entered directly by the operator.
- Maintenance: this mode allows authorized personnel to run the unit/machine independent of other systems. This mode would be typically used for troubleshooting, machine trials, or testing operational improvements.
- Manual: this mode allows the direct control of single machine modules. It may be used for commissioning of individual components, fault diagnostics of unplanned technical intervention, etc.
- Users can add additional user modes in the SIMATIC CPG Template. Examples may include „Cleanout“, „Product Change“, etc.

In each Mode, a different subset of the 17 PackML States can be used and the program can behave differently in the same state for different modes. For example:

- In the Production mode the Execute state in a labeling machine will mean that it is labeling bottles in a continuous fashion. In the Manual mode, the Execute state implies that the machine is waiting for or performing an operator command, such as jogging.
- In the Production mode, the full set of 17 states are to be used. In the Manual mode, the states Suspending, Suspended, and Unsuspending are not to be used. The Suspending branch is typically provided so the machine can respond to external stoppages on the line. In Manual mode this behavior is unnecessary, so the states are deactivated.

A mode change is not allowed while a state change is taking place. Mode-to-Mode transitions occur within the same state, common to both modes based on:

- Local or remote operator request
- Remote request from another machine

The CPG template allows mode changes in the Idle, Stopped, and Aborted states.

1.2.4 PackML Standard Interface (PackTags)

End Users run production lines and process plants which consist of components as single machines or process equipment. All those components are built from different OEMs. This can easily result in lack of software consistency between

machines, difficult and time consuming horizontal/vertical integration as well as challenging troubleshooting and service.

PackTags provide a uniform set of data structures used in the base state model. The aim is therefore to make sure machines deliver homogeneous data to higher level information systems and enable easier line integration and serviceability.

PackTags are divided into three groups: command, status and administration. Command and Status tags contain data required for interfacing between machines and line control for coordination, or for recipe/parameter download. Command tags are "written" to and consumed by the machine program, as the "information receiver", while status tags are produced by and read from the machine program. Administration tags contain data collected by higher level systems for machine performance analysis, or operator information. Generally informational data is passed using OPC on an Ethernet-based communication network.

- Command tags are prefixed by "Command".
- Status tags are prefixed by "Status".
- Administration tags are prefixed by "Admin".

PackML defines the minimum set of the data elements and their data types. It also defines many optional tags. Of course, it is allowed for Machine Suppliers to implement additional PackTags. The table below indicates the minimum set of PackTags in order to be OMAC PackML compliant. ANSI/ISA 88.00.02-2015 is the reference for all PackTags.

Table 1-1: Mandatory PackTags

PackTag Type	PackTag Name	Example of End User term	Datatype
Status	UnitModeCurrent	Mode	DInt (32bit)
Status	StateCurrent	State	DInt (32bit)
Status	MachSpeed	Nominal Speed	Real
Status	CurMachSpeed	Current Speed	Real
Status	EquipmentInterlock.Blocked	Blockage	Bool
Status	EquipmentInterlock.Starved	Starvation	Bool
Admin	StopReason.ID	Event and stop reason	DInt (32bit)
Admin	ProdProcessedCount[#.Count	OEE total count	DInt (32bit)
Admin	ProdDefectiveCount[#.Count	OEE bad count	DInt (32bit)
Command	UnitMode	Mode	DInt (32bit)
Command	UnitModeChangeRequest	Change mode	Bool
Command	MachSpeed	Machine Speed	Real
Command	CntrlCmd	Command	DInt (32bit)
Command	CmdChangeRequest	Change command	Bool

1.2.5 S88 Make2Pack (modular design)

Make2Pack (ISA88 Part 5) is a standard way of decomposing a machine into its physical and functional elements. The machine program is grouped into 3 logical categories:

- Unit Level (UN)
- Equipment Modules (EMs)
- Control Modules (CMs)

Each category has a specific function in the overall operation of the machine.

Control modules or whole equipment modules can be disabled without affecting the operation of the rest of the machine.

The Unit Level (UN)

The Unit Level contains the PackML State Machine and it is responsible for collecting all data and information from the lower level modules.

It acts as the main interface for the operation and control of the entire machine.

Equipment Modules (EMs)

An EM acts as a logical grouping of physical components that all function together to perform a single task. It is responsible for monitoring status and command requests from each individual control module.

Control Modules (CMs)

A CM controls and interacts with the physical elements in a machine and the outside world. It is responsible for monitoring and controlling its physical device based on the current machine state. It is also responsible for requesting a state change, if required. Here is where the user programs the machine logic of the machine.

For example:

UN:	A labeler
EM:	A label head
CM:	A label applicator, a label supplier

1.2.6 CPG Template Event Manager

The CPG template provides three types of events to aid in the diagnosing of operational problems: Alarms, Warnings, and Statuses. All three types of events are triggered inside of a particular control or equipment module and can be filtered as such. The template also tracks the time of trigger, the first fault, and provides for multilingual alarm texts. Alarms can be communicated to outside systems via the PackTags interface.

Alarms

Alarms imply a serious issue with the machine, typically one that inhibits production or presents a safety risk. Alarms are assigned to a "category", which allows the state machine to immediately respond to an alarm by moving into the appropriate state.

For example: a label applicator runs out of glue, forcing the machine to transition to the "Held" state until the issue is resolved. The alarm text is "Label head 1 out of glue"

Warnings

Warnings tell the operator that something is wrong. Warnings typically do not impact production immediately, but they will require operator action now or in the near future.

For example: a label applicator is low on glue. A warning is displayed with the text “Label head 1 glue running low”.

Statuses

Statuses do not indicate that something is wrong with the machine, but they give the operator extra information. Typically, they tell the operator what needs to be done to move the machine into Execute state.

For example: A machine in the Idle state triggers the status “Waiting for start PB”.

1.2.7 LPMLV30 Function Blocks

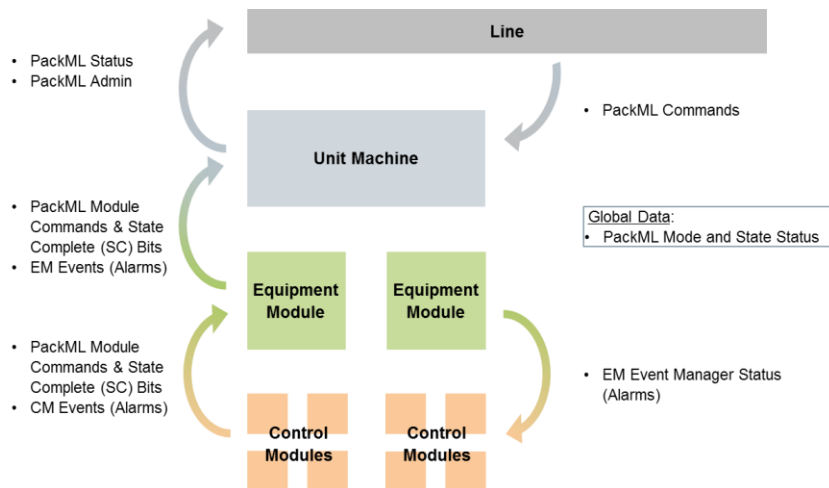
The CPG Template is constructed using several blocks from OMAC PackML V3 Machine and Unit States library, referred to as the LPMLV30. The LPMLV30 implements a fully compliant PackML state and mode manager. As discussed, the CPG Template extends this functionality considerably by adding PackTags support, the Make2Pack decomposition, integrated alarming, built in summation of commands, and many other features.

Blocks, constants, and data types taken from this template are prefixed with “LPMLV30_”. For complete information about them please refer to their manual directly.

1.3 Understanding the CPG Template

The CPG Template library provides you with a ready to use template project with equipment modules, Function Blocks for the data flow between control modules, equipment modules and Unit Machine layers, together with an HMI project template. It is possible to extend the template with other equipment modules as needed. The CPG Template provides standard blocks to handle state and mode transitions, event information and triggering, and the summation of commands and events from different control modules.

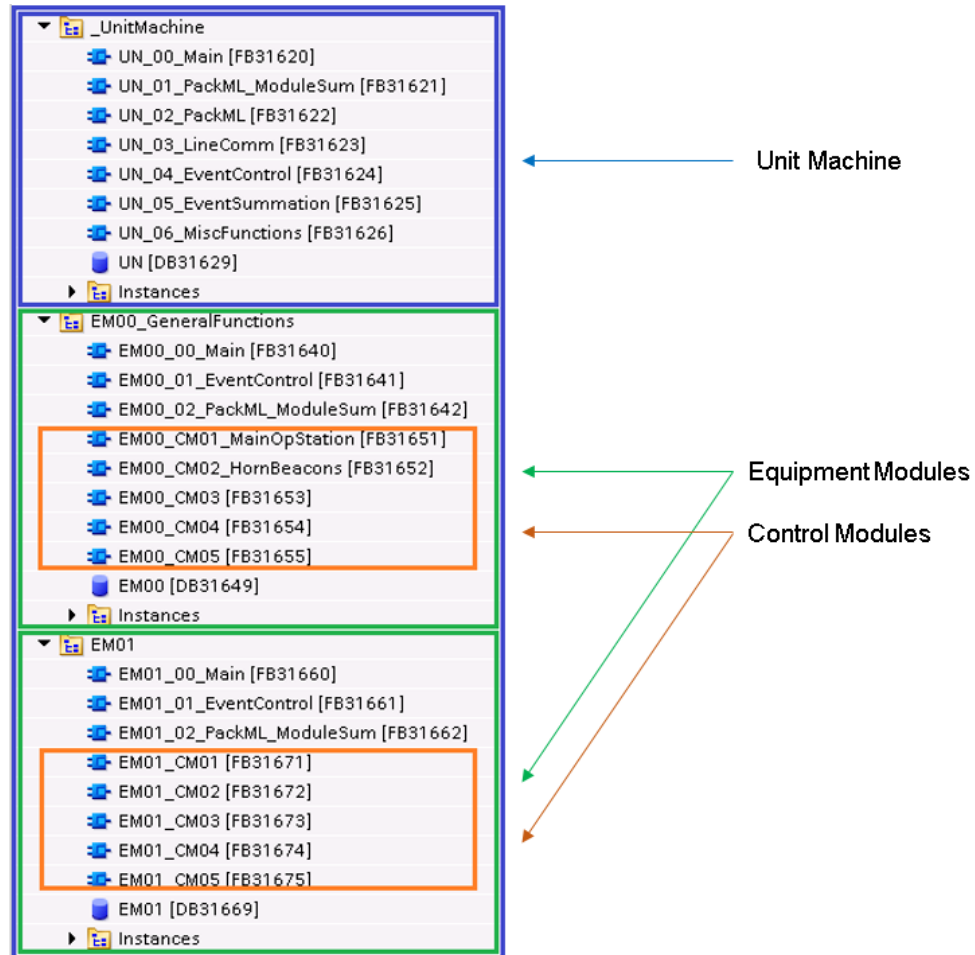
Figure 1-2: Understanding the CPG Template



1.3.1 Make2Pack Division of Functions

The Make2Pack ISA 88 Physical Model of the machine can be easily distinguished in the CPG template structure:

Figure 1-3: Make2Pack modular design in the CPG Template project tree



Unit Machine

The Unit Machine (UN) is the highest logical level for the CPG functional decomposition. Overall properties and tasks of the machine are executed here. Examples include determining the overall machine state and communicating with remote interfaces. On the Unit Machine level the following are accomplished:

- The PackML state machine and mode manager are executed.
- All state machine commands from EMs and the external PackTags interface are “summarized”. This information is used to determine the current PackML mode and state.
- Events from all EMs are “summarized”. The most recent alarm is logged, and a sortable list of alarm structures is provided for display. Any commands executed via events are determined.
- The summarized commands are combined with the event information to determine the current state and mode.

- Space is provided to execute events directly inside the UN level, rather than inside a module. These events are also summarized with the EM events.
- Space is provided for any other miscellaneous tasks to be executed at the UN level.

Equipment Modules

Equipment modules (EMs) refer to subsections of the machine. This is the intermediate level, summarizing the user logic of the control modules and relaying that information to the overall unit. Each equipment module in the machine has its own program folder and unique number. For this document, a generic EM is referred to as EMxx. By default, the CPG Template provides 3 equipment modules, though this can be changed. At the EM level the following tasks are accomplished:

- Blocks are provided for each control module. Most machine logic is typically executed at the CM level.
- All state machine commands from CMs are “summarized”. This information is passed to the UN level, where it ultimately helps determine the machine’s state.

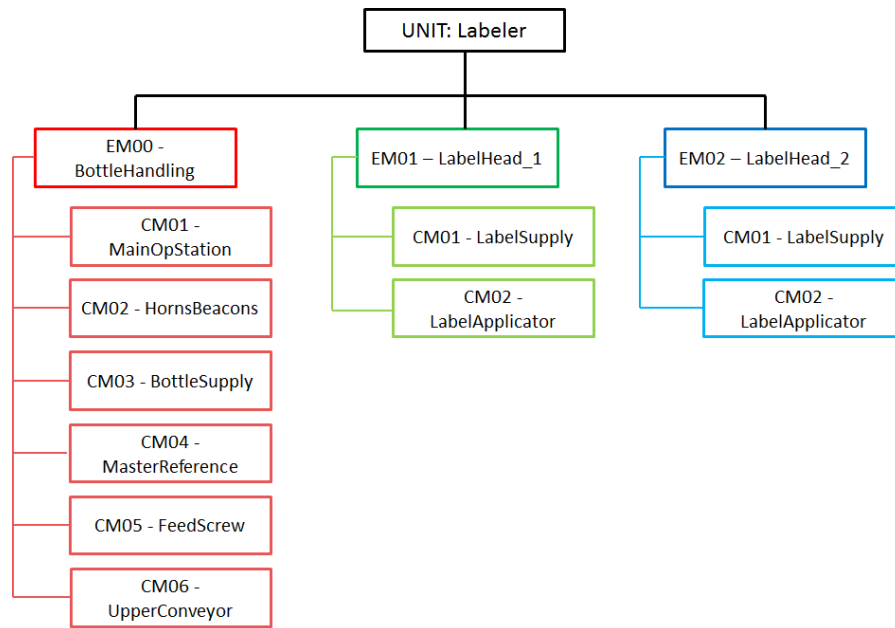
Events from all CMs are “summarized”. The most recent alarm is logged, and a sortable list of alarm structures is provided for display. Any commands executed via events are determined.

Control Modules

The control module (CM) is the lowest logical level of the functional decomposition. At this level, the user logic and most alarms are implemented. The critical questions are “in what PackML states will the control module execute which tasks?” and “under what circumstances will the control module issue a state complete or a state machine command?” By default, the CPG Template provides 5 control modules inside each equipment module, though this can be changed. On the control module level, the following is programmed:

- User specific machine logic.
- Most user programmed PackML events.
- State machine commands and state complete bits.

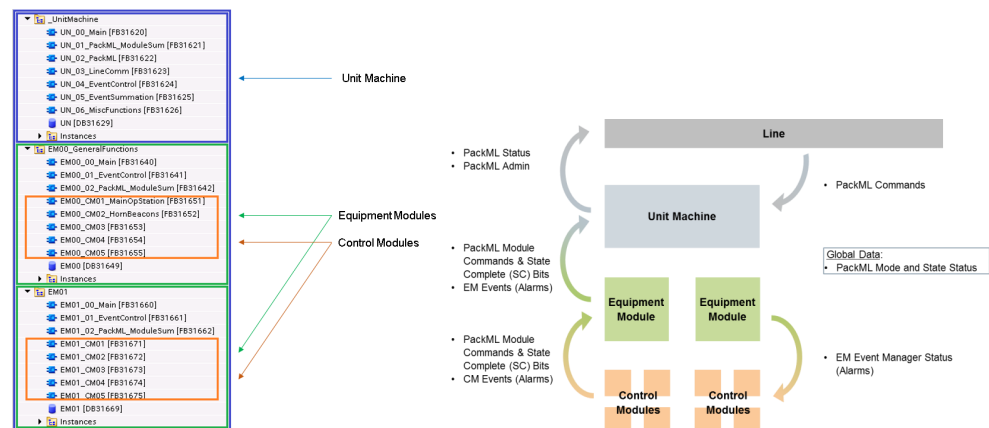
Figure 1-4: Physical Hierarchy example – a labeler machine



1.3.2 What is meant by “Summation”?

“Summation” refers to the combination of information from different modules into a coherent whole. This task is performed automatically by the CPG Template for both state machine commands and PackML events.

Figure 1-5: Dataflow inside the application



On the unit level the OMAC mode and state manager is executed. Therefore, at any point in time the machine is in one state. In other words, the PackML current mode/state is a property of the machine. For example, it is not possible to have one equipment module in Stopping and one in Idle at the same time. The unit machine determines the appropriate state and provides this information to all modules via the PackTags. For example, if the unit is in the Idle state, then the necessary actions for the machine will be executed directly in each CM. Regarding summation, there are two cases to consider:

Case 1: The machine is in a waiting state (for example Idle):

The machine remains in Idle until it receives a Start command. Each CM has its own Start command bit, which could be set. If it is set in any CM, the EM containing that CM will set its own Start command. The EMs send their Start commands to the UN. If any EM sends a Start command, the machine will transition to Starting. In this way, any CM can transition the entire machine out of a waiting state. The operator needs only to press a single start button, connected to a single start command in a single CM, to move the machine state.

Case 2: The machine is in an acting state (for example Starting):

The machine remains in Starting until the Starting SC bit is set. Each CM has its own SC bit, which could be set. If any CM has its Starting SC bit false, the EM will not set its own Starting SC. The EMs send their Starting SC bits to the UN. Once all the EMs have set their SC bit, the machine transitions to Execute. Every CM must finish its required actions in the acting state for the machine to transition out of it. Any single CM can hold the entire machine in an acting state if it has not finished its tasks. This is precisely the inverse of the waiting states.

Some logic can also be executed at the UN level directly, rather than inside a particular control module. Any PackML commands sent here are summarized with the equipment modules directly.

The summation of the state machine bits at the EM level is done inside the following FB: EMxx_02_PackML_ModuleSum.

The summation of the state machine bits at the Unit level is done inside the following FB: UN01_01_PackML_ModuleSum.

PackML events can be alarms, warnings, or statuses. They can originate in a control module, or they can take place directly at the UN level. Each EM (and the UN) tracks the events that occur within it. Events are automatically tracked with a timestamp, and a "first fault" is logged separately. Any state machine actions taking place as a result of events are summarized alongside commands coming from the control modules. Event summation at the UN level takes the timestamped events from each EM and produces a single list. This list is automatically displayed on the integrated HMI for analysis. It can be sorted, and the first fault is specially designated.

Determining when events drive state machine actions and resetting faults occurs in: EMxx_02_EventControl (or UN_04_EventControl).

The summation of the events from all EMs and the UN is done in UN_05_EventSummation and the results are stored in the Alarm, Warning, and Status DBs.

1.3.3 Overview of Programming Tasks

The CPG Template implements the PackML standard. It also adds additional features, notably the Make2Pack functional decomposition and the PackML Events alarming concept. The CPG blocks implement this framework and can operate for most users without modification. However, programming a machine requires the user to address several issues. Key questions are outlined in this section, along with what programming steps are required to fulfill them. This section is meant to

provide an introduction and is not exhaustive, please refer to the relevant manual subsections for more information.

- How will the machine be broken down into its functional decomposition?
- Which PackML states and modes will be utilized?
- What machine actions will take place in each PackML state/mode?
- What PackML events will be implemented?

How will the machine be broken down into its functional decomposition?

Though it is always possible to implement a machine inside a single control module, this is seldom desirable. In practice, machines should be broken down into several equipment and control modules. This enhances modularity, promotes debugging, and generally provides an easier way to understand program structure. The number and name of each equipment/control module can be user defined. Typically, equipment modules are a logical subsection of the machine, and control modules are a part of the machine associated with specific actions. Please refer to the labeler demo for further examples.

The CPG Template provides three equipment modules with five control modules each. If not all modules are needed, it is possible to deactivate them. To deactivate a module, navigate to the disable structure in the PackMLCfg datablock. Each module has its own deactivate bit. Set the appropriate bit(s) to TRUE to deactivate the modules and prevent any logic from executing.

If additional control modules inside an EM or additional equipment modules are required, please refer to chapter 4 of this manual.

Which PackML states and modes will be utilized?

The PackML state machine contains 17 states. It is not necessary to use all of them, though four (IDLE, ABORTED, STOPPED, EXECUTE) are required. It is not permitted to add additional states or transitions. Please refer to the PackML documentation for guidelines on the purpose of each state.

It is also necessary to decide which machine modes will be needed. Each mode can rely on its own subset of the 17 PackML states. For example, Production may use all 17 states while Manual only uses the minimum number of states. The mode/state choices are up to the user and depend on machine functionality.

To implement the desired states and modes, navigate to the Main Organization Block (OB1). Two blocks, LPMLV30_ConfigureDisabledUnitModes and LPMLV30_ConfigureDisabledStates are used to control the enabling/disabling of modes and states respectively.

LPMLV30_ConfigureDisabledUnitModes has Boolean inputs for 8 optional PackML user modes. If an input is TRUE, a mode is disabled, if an input is FALSE, the mode is enabled. Modify the inputs according to your needs. If further user modes are required, refer to the LPMLV30 manual directly.

LPMLV30_ConfigureDisabledStates is similar. There must be a call of this block in OB1 for each mode to be used. It is necessary to add additional calls if more than the production and manual modes are used in the program. If an input for a state is TRUE, the state is disabled. If it is FALSE, the state is enabled. Set the inputs according to the machine needs and write the result to the appropriate entry in the CPG_PackMLCfg DB. See section CPG_PackMLCfg for more information.

States and modes can be given customized names. These names are stored in the LPMLV30 constants tab, edit them as needed.

What machine actions will take place in each PackML state/mode?

For each control module, in each mode, in each state, it is necessary to determine what the machine will do. For example, in what states will a servo axis be homed? In what states is it reset? In what states is it enabled? Geared to its master? The needed machine logic should be placed into the control module function blocks, or user blocks called within the control modules.

The concept “states drive outputs” is useful here. This means that logic in the control modules should be executed accordingly to the current state of the machine. If the machine is in [STATE X], then perform [ACTION Y]. If Resetting, set the bit to home the servo axis. If Aborting, signal for a fast stop. Open the valve in any state except Aborting or Aborted, etc. The PackML mode and state determine the outputs that are set in the control module.

To read out the machine state, refer to the *sts* structure in the PackML global data block. In this structure, bits are provided for each PackML state and mode. If the machine is in that state (or mode), the corresponding bit is true.

Furthermore, it is necessary to determine when the state machine must execute a state (or mode) transition. Under what conditions will a start command be sent? This also applies to the state complete (SC) bit. In any acting state, control modules may perform tasks. Feedback that the task is completed should drive the SC bit. For example, consider a CM that is a servo axis in the Stopping state. The CM task is to stop. The feedback that the task is completed is a drive speed of zero, or possibly a standstill signal from the drive. When the PLC receives the proper feedback from its servo, then the CM should set its Stopping SC bit.

To aid in writing this logic, the block `CPG_ModuleInitialize` is provided. This block automatically sets the PackML commands to FALSE and SC bits to TRUE. This is provided so that if a particular control module does not require a command or SC bit, it can be ignored. For example, if a CM has no tasks in the Aborting state, then there is no need to write to the Aborting SC bit; it should be left out. Because of `CPG_ModuleInitialize`, it will be ignored in any summations. Similarly, if a CM will never issue a PackML command, not writing to that bit will result in it always remaining FALSE. Each CM block contains a call of `CPG_ModuleInitialize`, none need to be added.

Each module is provided with its own complete set of PackML command and SC bits. CMs should write to their own bits, not any other module's set. These are located in the appropriately numbered EMxx datablock, in the PackML structure. The provided blocks in the CPG template will handle any remaining summation tasks. If these bits are set when needed, the machine will enter the correct PackML state. If the correct states drive the correct outputs, then the machine will perform the proper actions in each state.

It is also possible to add logic at the machine level, rather than in a control module. Place the logic in the `UN_MiscFunctions` block.

What PackML events will be implemented?

Any alarm, status, or warning used in the machine program must be defined by the user. In the process of configuring/programming events, determine:

- the causes.
- message text (in all languages).
- any state machine actions (such as aborting or stopping) that should occur. This is encoded in the “Category” field.
- the optional value – this is provided for further user-based sorting of events.

The event's data (message texts, category, etc.) is set in the appropriate CPG configuration datablock – CPG_AlarmCfg, CPG_WarningCfg, or CPG_StatusCfg. Each datablock contains an array to hold this information. To set the length of these arrays, modify the constants CPG_NO_OF_CFG_ALARMS_UPPER_LIM, CPG_NO_OF_CFG_WARNINGS_UPPER_LIM, and CPG_NO_OF_CFG_STATUS_UPPER_LIM as appropriate, then recompile. Fill in the information into the datablocks.

To add an event to a CM, insert a multi-instanced call of CPG_Event. Provide the event configuration information for the event to be triggered and set the enable bit while the event should be active. Also, provide that equipment module's event structure (EMxx.Alarm, EMxx.Warning, or EMxx.Status), located in the EM datablock.

Optionally, a message prefix can also be provided. Message prefixes identify the EM/CM the event was triggered inside. If the same alarm is triggered in multiple modules, prefixes can easily differentiate them. This information is prepended to the event text by CPG_Event. The prefix text is set in the configuration datablock, CPG_MessagePrefix.

The CPG Template is fully equipped to summarize events occurring within it. Events are logged in order, timestamped, and alarms which drive state machine actions will do so automatically. Events are also displayed and can be filtered on the integrated HMI without further coding effort by the user.

It is also possible to add machine-level events to the block, via UN_MiscFunctions. Add calls of CPG_Event there.

1.4 Components used

This application example has been created with the following hardware and software components:

Table 1-2 Used components

Component	Number	Article number	Note
CPU 1513-1 PN	1.	6ES7513-1AL01-0AB0	Or other S7-1500 CPU with FW V2.1
or CPU 1515T-2 PN	2.	6ES7515-2TM01-0AB0	Or other S7-1500T CPU with FW V2.1
Step7 Professional V14 SP1	3.	6ES7822-1..04-..	
TP1200 Comfort	4.	6AV2 124-0MC01-0AX0	

This application example consists of the following components:

Table 1-3 Files of CPG Template

Component	File name	Note
CPG Template Project	CPG_Template_SIMATIC_V1_0_x.zip	
CPG Template BaseProject Manual	CPG_Template_SIMATIC_V1_0_en.pdf	

2 Engineering

The CPG Template provides all the necessary structure to implement a PackML compliant Machine. In the following sections all custom data types, constants, and blocks are discussed in detail.

2.1 CPG Template Global Data Blocks

The CPG Template provides several data blocks for global data (navigate to folder *Program blocks* -> *GlobalData*). Their purpose is outlined in the following sections.

Table 2-1 Overview of CPG Template global data blocks

Name	Comment
Alarm	Structure with alarm event summation
HMI_Data	HMI interface data including event view, diagnostics, commands, etc.
HMI_Data_Custom	User specific HMI data
PackML	Additional PackML data e.g. Boolean interface, OEE data, etc.
PackTags	PackML PackTags machine interface data
Status	Structure with status event summation
Warning	Structure with warning event summation

2.1.1 Alarm

This data block contains the final, summarized, alarm information. All data in this block is located inside the "Summation" structure, which is a tag of type `CPG_typeEventSummation`. In this data block, all the events tracked are alarms. This DB is referenced in conjunction with the block *CPG_EventSummation*, *CPG_EventSummationBegin*, and *CPG_EventSummationEnd*. *CPG_EventSummation* is called for each EM and the UN inside of *UN_05_EventSummation*. The purpose of these calls is to summarize the alarm information coming from each EM into a single unified list.

Table 2-2: Alarm Data Block

Name	Data type	Default value	Comment
Summation	<code>CPG_typeEventSummation</code>		Unit Mode – Alarm Summation

Table 2-3: Alarm Data Block (key tags)

Name	Comment
Summation.sts_FirstOutEvent	First out alarm for the entire machine
Summation.sts_FirstOutEventCat	Array of ten alarms. Each array entry is the first tracked alarm for each alarm category
Summation.sts_Events	Time-sorted array of alarms, as they come in. Its length is controlled by the constant <code>CPG_NO_OF_STS_ELEMENTS_EVENT_SUM_UPPER_LIM</code> . Increasing this constant increases the number of alarms tracked
Summation.sts_NumEvents	Number of active alarms

Name	Comment
Summation.sts_Category_#_Latched	Will be true if the category (0-9) is latched. This means either an alarm of that category is active or an alarm was active, but no reset has taken place
Summation.sts_Category_#_NotLatched	Will be true if the category (0-9) is not latched. This means an alarm of that category is currently active
Summation.sts_CurrentActiveEvent	Is true if at least one alarm is active

2.1.2 Warning

This data block contains the final, summarized, warning information. All data in this block is located inside the „Summation“ structure, which is a tag of type *CPG_typeEventSummation*. In this data block, all the events tracked are warnings. This DB is referenced in conjunction with the block *CPG_EventSummation*, *CPG_EventSummationBegin*, and *CPG_EventSummationEnd*. *CPG_EventSummation* is called for each EM and the UN inside of *UN_05_EventSummation*. The purpose of these calls is to summarize the warning information coming from each EM into a single unified list.

Table 2-4: Warning Data Block

Name	Data type	Default value	Comment
Summation	CPG_typeEventSummation		Unit Mode – Warning Summation

Table 2-5: Warning Data Block (key tags)

Name	Comment
Summation.sts_FirstOutEvent	First out warning for the entire machine
Summation.sts_FirstOutEventCat	Array of ten warning. Each array entry is the first tracked warning for each warning category
Summation.sts_Events	Time-sorted array of warnings, as they come in. Its length is controlled by the constant <i>CPG_NO_OF_STS_ELEMENTS_EVENT_SUM_UPPER_LIM</i> . Increasing this constant increases the number of warnings tracked
sSummation.sts_NumEvents	Number of active warnings
Summation.sts_Category_#_Latched	Will be true if the category (0-9) is latched. This means either a warning of that category is active or a warnings was active, but no reset has taken place
Summation.sts_Category_#_NotLatched	Will be true if the category (0-9) is not latched. This means a warning of that category is currently active

2.1.3 Status

This data block contains the final, summarized, status information. All data in this block is located inside the „Summation“ structure, which is a tag of type *CPG_typeEventSummation*. In this datablock, all the events tracked are statuses. This DB is referenced in conjunction with the block *CPG_EventSummation*, *CPG_EventSummationBegin*, and *CPG_EventSummationEnd*. *CPG_EventSummation* is called for each EM and the UN inside of *UN_05_EventSummation*. The purpose of these calls is to summarize the status information coming from each EM into a single unified list.

Table 2-6: Status Data Block

Name	Data type	Default value	Comment
Summation	CPG_typeEventSummation		Unit Mode – Status Summation

Table 2-7: Status Data Block (key tags)

Name	Comment
Summation.sts_FirstOutEvent	First out status for the entire machine
Summation.sts_FirstOutEventCat	Array of ten statuses. Each array entry is the first tracked status for each status category
Summation.sts_Events	Time-sorted array of statuses, as they come in. Its length is controlled by the constant <i>CPG_NO_OF_STS_ELEMENTS_EVENT_SUM_UPPER_LIM</i> . Increasing this constant increases the number of statuses tracked
Summation.sts_NumEvents	Number of active statuses
Summation.sts_Category_#_Latched	Will be true if the category (0-9) is latched. This means either a status of that category is active or a status was active, but no reset has taken place
Summation.sts_Category_#_NotLatched	Will be true if the category (0-9) is not latched. This means a status of that category is currently active
Summation.sts_CurrentActiveEvent	Is true if at least one status is active

2.1.4 HMI_Data

The *HMI_Data* data block contains the data coming to and from the HMI by default. Any future additions to the HMI interface should be placed in *HMI_Data_Custom* or a user defined DB.

Table 2-8: HMI_Data Data Block

Name	Data type	Default value	Comment
resetPackMLTimers	Bool	FALSE	Reset the PackML Timer values when set (1).
languageSelected	Int	0	This is the value of the language selected in the HMI (0=English, 1=DE, 2=SP, 3...)
modeCurrentTime	DInt	0	Time spent in the current mode [seconds](reset each time new mode is entered)
modeCumulativeTime	Dint	0	Accumulative Time spent in the current mode [seconds]
State_Current_Times	Array[0..LPMLV30_STATES_UPPER_LIM] of DInt		Current Time (# of seconds) spent for each state for active mode.
State_Cumulative_Times	Array[0..LPMLV30_STATES_UPPER_LIM] of Dint		Accumulative Time (# of seconds) spend for each state for active mode.
alarmSortedList	Array[0..CPG_NO_OF_HMI_SORT_EVENTS_UPPER_LIM] of CPG_typeEvent		Alarm Sorted List for HMI - summary of Active Events

Name	Data type	Default value	Comment
EM_SortDisable_Alarm	Struct of Bool		Alarm Disable for Equipment Modules (EMs)
alarmFilter	CPG_typeSortFilter		HMI Commands to control the Alarm Sort Filter
warningSortedList	Array[0..CPG_NO_OF_HMI_SORT_EVENTS_UPPER_LIM] of CPG_typeEvent		Warning Sorted List for HMI - summary of Active Events
EMSortDisableWarning	Struct of Bool		Warning Disable for Equipment Modules (EMs)
WarningFilter	CPG_typeSortFilter		HMI Commands to control the Warning Sort Filter
HMIWarningMsgScroll	String[40]	"	Warning Message - scroll of all active warnings
ModeNames	Array[0..LPMLV30_MODES_UPPER_LIM] of String[18]		Mode Names for selected language
StateNames	Array[0..LPMLV30_STATES_UPPER_LIM] of String[16]		State Names for selected modes
PLCDateTime	Struct		Date Time for WinCC Flexible HMI format
diagnosticsBufferIndexForHMI	Int	0	Diagnostics to HMI
diagnosticsBufferEntryForHMI	typeLPMLV30_DiagnosticsEntry		Diagnostics to HMI
HMICoordination	Word	16#0	HMI Coordination area pointer: Bit0 = Startup; 1=OpMode (0=on, 1=off-line), 2=life bit
HMICurrentScreenID	Int	0	HMI Current screen number
CMDs	Struct of Bool		HMI Commands

2.1.5 HMI_Data_Custom

This data block is added to the project for machine specific HMI data not included in *HMI_Data*. By default, it contains only two tags – *machineSpeedSP* and *machineSpeedPV*. These tags by default are not connected anywhere in the CPG Template. Any machine specific HMI data can be added to this block by the programmer.

Table 2-9: HMI_Data_Custom Data Block

Name	Data type	Default value	Comment
machineSpeedSP	Real	20.0	Machine Speed Setpoint (product per min)
machineSpeedPV	Real	0.0	Machine Speed Actual (product per min)

2.1.6 PackML

This data block contains information related to the PackML states and modes. The current PackML state and mode of the machine are stored here in the `PackML.Sts` structure. The summarized Boolean commands which drive the state & mode manager to change state are also located here. `remoteCommandEnable` allows for the enabling and disabling of issuing PackML commands via the remote interface. OEE values are calculated automatically by the CPG Template and are stored in the `PackML.OEE` structure. PackML diagnostics are stored in the `PackML.PackMLDiag` structure.

Table 2-10: PackML Data Block

Name	Data type	Default value	Comment
projectVersion	String[10]	'V1.0.0'	Project specific version
Cmd	Struct of Bool		PackML boolean COMMAND to PackML ModeManager & State Model
Sts	Struct of Bool		Status for current mode and state as boolean
Cmd_Reset	Bool	FALSE	Request for command - RESET
Cmd_Start	Bool	FALSE	Request for command - START
Cmd_Stop	Bool	FALSE	Request for command - STOP
Cmd_Hold	Bool	FALSE	Request for command - HOLD
Cmd_UnHold	Bool	FALSE	Request for command - UNHOLD
Cmd_Suspend	Bool	FALSE	Request for command - SUSPEND
Cmd_UnSuspend	Bool	FALSE	Request for command - UNSUSPEND
Cmd_Abort	Bool	FALSE	Request for command - ABORT
Cmd_Clear	Bool	FALSE	Request for command - CLEAR
Cmd_Complete	Bool	FALSE	Request for command - COMPLETE
remoteCommandEnable	Bool	FALSE	If 0 = local mode for PackML commands, if 1=remote command for PackML
OEE	Struct of Real		Real-Time OEE values
OEE.realTime	Real	0.0	Real-Time value (0-100%)
OEE.availability	Real	0.0	Availability value (0-100%)
OEE.performance	Real	0.0	Performance value (0-100%)
OEE.quality	Real	0.0	Quality value (0-100%)
PackMLDiag	typeLPMLV30_Diagnostics		PackML Diagnostic Data

2.1.7 PackTags

This data block contains the PackTags supported by the CPG Template. PackTags are divided into three structures, „Command“, „Status“, and „Admin“, reflecting the definitions of the PackML PackTags.

Several tags are arrays of configurable length. Change the appropriate constant and recompile to adjust the data structure.

Table 2-11: PackTags Data Block

Name	Data type	Default value	Retain	Comment
Command	Struct			PackTags "Command" tag area. Data is scalable for needs.
Command.UnitMode	DInt	0		Unit Mode Target (1=Production,2=Maintenance,3=Manual,4=UserMode1,...,31=UserMode28)
Command.UnitModeChangeRequest	Bool	FALSE		1=Request Unit Mode Change to commanded value (UnitMode)
Command.MachSpeed	Real	0.0		Setpoint of Machine Speed in Primary Packages per Minute
Command.MaterialInterlock	DWord	16#0		Materials Interlock Command (1=Ready or Not low, 0=Not ready or Low)
Command.CntrlCmd	DInt	0		State Control Command to drive a state change (1=Reset,2=Start,3=Stop,4=Hold,5=Unhold,6=Suspend,7=Unsuspend,8=Abort,9=Clear)
Command.CmdChangeRequest	Bool	FALSE		1=Request Machine State Change to commanded value (CntrlCmd)
Command.Parameter	Array[0..CPG_NO_OF_COMMAND_PARAMETERS_UPPER_LIM] of CPG_typeParameter(Real)			Parameter Structure Array from SCADA / MES
Status	Struct			PackTags "Status" tag area. Data is scalable for needs.
Status.UnitModeCurrent	DInt	3		Current Unit Mode (1=Production,2=Maintenance,3=Manual,4=UserMode1,...,31=UserMode28)
Status.UnitModeCurrentName	String[18]	'Manual'		Current Unit Mode Name (NOT ISA-TR88 - OPTIONAL)
Status.UnitModeRequested	Bool	FALSE		1=Ack. that a unit mode change has been requested
Status.UnitModeChangeInProcess	Bool	FALSE		1=Requested Unit Mode Change In Progress
Status.StateCurrent	DInt	2		Current Machine State (1=Clearing,2=Stopped,3=Starting,4=Idle,5=Suspended,6=Execute,7=Stopping,8=Aborting,9=Aborted,10=Holding,11=Held,12=Unholding,13=Suspending,14=Unsuspending,15=Resetting,16=Completi

Name	Data type	Default value	Retain	Comment
				ng,17=Complete)
Status.State CurrentName	String[16]	'Stopped'		Current Machine State name (NOT ISA-TR88 - OPTIONAL)
Status.State Requested	DInt	0		Target Machine State
Status.State ChangeInProgress	Bool	FALSE		1=Requested State Change in Progress
Status.Mach Speed	Real	0.0		Setpoint of Machine Speed in Primary Packages per Minute
Status.CurM achSpeed	Real	0.0		Actual Value of Machine Speed in Primary Packages per Minute
Status.Mater ialInterlock	DWord	16#0		Materials Interlock Status (1=Ready or Not low, 0=Not ready or Low)
Status.Equip mentInterloc k	Struct			Machine upstream and downstream status
Status.Equip mentInterloc k.Blocked	Bool	FALSE		1=The downstream system is not able to accept products, 0=all OK
Status.Equip mentInterloc k.Starved	Bool	FALSE		1=The upstream system is not able to supply products, 0=all OK
Status.Para meter	Array[0..CPG_NO_OF_ST ATUS_PARAMETERS_UP PER_LIM] of CPG_typeParameter(Real)			Parameter Structure Array for HMI / SCADA / MES
Admin	Struct		X	PackTags "Administration" tag area. Data is scalable for needs.
Admin.Para meter	Array[0..CPG_NO_OF_AD MIN_PARAMETERS_UPP ER_LIM] of CPG_typeParameter(Real)		X	Parameter Structure Array for HMI / SCADA / MES
Admin.Alarm	Array[0..CPG_NO_OF_AD MIN_EVENTS_UPPER_LI M] of CPG_typeEvent		X	Alarm summary of Active Events
Admin.Alarm Extent	DInt	0	X	Number of active alarms
Admin.Alarm History	Array[0..CPG_NO_OF_AD MIN_EVENTS_HISTORY_ UPPER_LIM] of CPG_typeEvent		X	Alarm History
Admin.Alarm HistoryExten t	DInt	0	X	Number of alarms in alarm history
Admin.Stop Reason	CPG_typeEvent		X	First Fault Alarm Code
Admin.Warni ng	Array[0..CPG_NO_OF_AD MIN_EVENTS_UPPER_LI M] of CPG_typeEvent		X	Active Machine Warnings

Name	Data type	Default value	Retain	Comment
Admin.WarningExtent	DInt	0	X	Number of Active Machine Warnings
Admin.Mode CurrentTime	Array[0..LPMLV30_MODE S_UPPER_LIM] of DInt		X	Time spent in the last mode [seconds] (reset each time new mode is entered)
Admin.Mode CumulativeTime	Array[0..LPMLV30_MODE S_UPPER_LIM] of DInt		X	Cumulative Time for all Modes [seconds]
Admin.State CurrentTime	Array[0..LPMLV30_MODE S_UPPER_LIM, 0..LPMLV30_STATES_UPPER_LIM] of DInt		X	Time spent in the last state [seconds] (reset each time new state is entered)
Admin.State CumulativeTime	Array[0..LPMLV30_MODE S_UPPER_LIM, 0..LPMLV30_STATES_UPPER_LIM] of DInt		X	Cumulative Time for all States [seconds]
Admin.Prod ConsumedCount	Array[0..CPG_NO_OF_PROD_CONSUMED_COUNT_UPPER_LIM] of CPG_typeProdCount(DINT)		X	Total Number of Products consumed- good or bad
Admin.Prod ProcessedCount	Array[0..CPG_NO_OF_PROD_PROCESSED_COUNT_UPPER_LIM] of CPG_typeProdCount(DINT)		X	Total Number of Products processed - good or bad
Admin.Prod DefectiveCount	Array[0..CPG_NO_OF_PROD_DEFECTIVE_COUNT_UPPER_LIM] of CPG_typeProdCount(DINT)		X	Total Number of Defective (bad) Products processed
Admin.AccTimeSinceLastReset	DInt	0	X	Accumulated Time Since Last Reset of all mode and state times and counts
Admin.Mach DesignSpeed	Real	0.0	X	Machine Design Speed of the Current Product Configuration
Admin.State sDisabled	DInt	0	X	States 1 to 17 of the PackML State Model can be disabled by turning on the corresponding bit in the DINT
Admin.PLCDateTime	Array[0..6] of DInt		X	Current Date and Time of PLC
Admin.PLCDateDTL	DTL	DTL#1970-01-01-00:00:00	X	Current Date and LTime (DTL) of PLC (date and time-of-day information in nanoseconds since 01/01/1970 0:0)

2.2 CPG Template Configuration Data Blocks

At engineering time, the user must provide some configuration information into the following DBs: *AlarmCfg*, *WarningCfg*, *StatusCfg*, *MessagePrefix*, and *PackMLCfg*. The first three contain the definitions of all PackML events used by the machine. The fourth contains the message prefixes for all modules, this information is also used for PackML events. The final block contains configuration information related to the PackML modes and states to be used by the machine.

Table 2-12 Overview of CPG Template configuration data blocks

Name	Comment
AlarmCfg	Configuration of alarm texts and alarm categories
MessagePrefix	Contains EM message prefixes used for event texts
PackMLCfg	PackML configuration, language dependent texts
StatusCfg	Configuration of status texts
WarningCfg	Configuration of warning texts

To aid in configuration, it is possible to export these blocks as .SCL files, which can be read in a text editor (such as Microsoft's Notepad). This is especially useful for alarm texts. The engineer can specify alarm texts in their primary language, export the block, and provide it to translation bureaus for translation to other languages. The block can be re-imported with the expanded text lists.

To import this file back into TIA Portal its extension should be changed to xxx.SCL.

2.2.1 AlarmCfg

This datablock contains the definitions of all PackML alarms used in the machine. Each alarm must have a unique ID number, which can be freely specified by the user (i.e., it is not necessary for the first alarm listed in AlarmCfg to have ID 1). It's also possible to specify a value, which allows for additional user-defined grouping of alarms.

Table 2-13: AlarmCfg Data Block

Name	Data type	Default value	Comment
Event	Array[0..CPG_NO_OF_CFG_ALARMS_UPPER_LIM] of CPG_typeEventCfg		Configuration of alarm texts and alarm categories

The message text is entered for each language in the *message* field. Set the constant LPMLV30_LANGUAGES_UPPER_LIM to increase or decrease the number of languages that may be entered. By default, event texts may be up to 60 characters in length. This can be changed but has a large impact on the CPU memory usage and should be minimized.

Any state machine actions resulting from the alarm are encoded in the category. By default, the alarm categories are as follows:

Table 2-14 Default category values explanation

Category value	PackML command	Description
0	ABORT	Safety related, Coordinated fast stop
1	ABORT	Coordinated fast stop
2	STOP	Coordinated normal stop
3	STOP	Coordinated normal stop
4	-	Spare
5	-	Spare
6	HOLD	Coordinated slow stop
7	HOLD	Coordinated slow stop
8	SUSPEND	Blocked, Coordinated slow stop
9	SUSPEND	Starved, Coordinated slow stop
Any other number	-	Invalid category

2.2.2 WarningCfg

This datablock contains the definitions of all PackML warnings used in the machine. Each warning must have a unique ID number, which can be freely specified by the user (i.e., it is not necessary for the first warning listed in *WarningCfg* to have ID 1). It's also possible to specify a value, which allows for additional user-defined grouping of warnings.

Table 2-15: WarningCfg Data Block

Name	Data type	Default value	Comment
Event	Array[0..CPG_NO_OF_CFG_WARNINGS_UPPER_LIM] of CPG_typeEventCfg		Configuration of warning texts

The message text is entered for each language in the *message* field. Set the constant LPMLV30_LANGUAGES_UPPER_LIM to increase or decrease the number of languages that may be entered. By default, event texts may be up to 60 characters in length. This can be changed but has a large impact on the CPU memory usage and should be minimized.

For PackML warnings, the category field has no effect.

2.2.3 StatusCfg

This datablock contains the definitions of all PackML statuses used in the machine. Each status must have a unique ID number, which can be freely specified by the user (i.e., it is not necessary for the first status listed in *StatusCfg* to have ID 1). It's also possible to specify a value, which allows for additional user-defined grouping of statuses.

Table 2-16: StatusCfg Data Block

Name	Data type	Default value	Comment
Event	Array[0..CPG_NO_OF_CFG_STATUS_UPPER_LIM] of CPG_typeEventCfg		Configuration of status texts

The message text is entered for each language in the *message* field. Set the constant LPMLV30_LANGUAGES_UPPER_LIM to increase or decrease the number of languages that may be entered. By default, event texts may be up to 60 characters in length. This can be changed but has a large impact on the CPU memory usage and should be minimized.

For PackML statuses, the category field has no effect.

2.2.4 MessagePrefix

When displaying alarm texts, an optional „message prefix“, may be prepended. This message prefix specifies the module the alarm originated it. If the same event is used in different modules, message prefixes offer a method for distinguishing them. By default, message prefixes display the name of the equipment and control modules (e.g. „EM00“ or „CM01“), but the text can be up to thirty characters and is freely definable. It is also possible to change the maximum length of message prefixes but this has a large impact on CPU memory usage and should be minimized.

This datablock consists of structures for the UN and each EM. Within each structure are tags of type *CPG_typeMessagePrefix* for each CM and for event control (alarming). By default, 10 EMs with 10 CMs each are supplied, but more can be added. Extras can be deleted or left unconfigured. Type in the desired message prefix in each project language, for each potential source of events. Connect the appropriate entry to the calls of the CPG_event call where that prefix should be applied.

Table 2-17: MessagePrefix Data Block

Name	Data type	Default value	Comment
UN	Struct		MessagePrefix for Unit
UN.Alarm	CPG_typeMessagePrefix		MessagePrefix for Unit Alarms
EMxx	Struct		MessagePrefix for EquipmentModuleEMxx
EMxx.EventControl	CPG_typeMessagePrefix		MessagePrefixforEventControl
EMxx.CMyy	CPG_typeMessagePrefix		MessagePrefixforControlModule#yy

2.2.5 PackMLCfg

This datablock contains configuration data for the PackML machine. It is divided into the *Cfg* structure, the *Names* structure, and the *disable* structure.

Table 2-18: PackMLCfg Data Block

Name	Data type	Default value	Comment
Cfg	typeLPMLV30_Configuration		PackML Configuration Data
Names	typeLPMLV30_NamesConfiguration		PackML Names Data
disable	Struct		Bits used to disable TR88 UN & EMs
disable.UN	Bool	FALSE	UN (1 = disable, 0 = enable)
disable.EMxx	Struct		EMxx (1 = disable, 0 = enable)
disable.EMxx.EM	Bool	FALSE	EMxx (1 = disable, 0 = enable)
disable.EMxx.CMyy	Bool	FALSE	CMyy (1 = disable, 0 = enable)

The PackMLCfg.Cfg structure should not be modified directly by the user. It is written to directly by two functions, *LPMLV30_ConfigureDisabledUnitModes* and *LPMLV30_ConfigureDisabledStates*. These functions are called in OB1.

To implement the desired modes, navigate to the call of *LPMLV30_ConfigureDisabledUnitModes*. If an input is TRUE a mode is disabled, if an input is FALSE, the mode is enabled. Modify the inputs according to your needs. If further modes are required, refer to the LPMLV30 manual directly.

LPMLV30_ConfigureDisabledStates is similar. There must be a call of this block in Main (OB1) for each mode to be used. If an input for a state is TRUE, the state is disabled. If it is FALSE, the state is enabled. Set the inputs according to the machine needs, and write the result to the appropriate entry in the *PackMLCfg* DB.

The *PackMLCfg.Names* structure holds the user defined names for all modes and states, in all languages. It is possible to rename modes and states with a user-defined text. The same PackML state can also have different names in different modes. For example, the EXECUTE state can be given the default name in the production mode, but be called CLEANING in the cleanout mode. By default, this text can be 18 characters long for modes and 16 characters long for states.

Every module (UN, EM, or CM) can be enabled or disabled in the program. To disable a module, modify its bit in the *PackMLCfg.disable* structure to TRUE. Disabled modules are not executed in the program, and any state machine commands they send are ignored by the summation logic. Modules can be disabled and reenabled as needed during program execution.

2.3 Unit Machine Blocks

The Unit folder contains the OMAC Mode and State Manager, as well as the event summation/sorting logics. A description of the blocks used to implement the UN is found below:

Table 2-19 Unit level blocks

Name	Description
UN_00_Main	This is the Unit's main block call and contains the call of all the underneath described FBs. It is called directly inside of the main scan (OB1)
UN_01_PackML_ModuleSum	This block contains the module summation of all the equipment modules in the project. The state machine commands sent by the EMs are combined into instructions for the overall state machine
UN_02_PackML	This block contains the OMAC mode and state manager to handle all the state transitions. The summarized commands from UN_01_PackML_ModuleSum are validated and passed into the state manager, which determines the machine state/mode
UN_03_LineComm	This block integrates the remote PackTags interface. If a command (Abort, Stop, Suspend, or Hold) is sent to the machine via the PackTags, it is handled here using several unit machine level events
UN_04_EventControl	This block transforms UN level PackML events into state machine actions. If (for example) a category 0 alarm is active, then the UN will receive an abort command because of logic present in this block
UN_05_EventSummation	This block contains the alarm summary logic, together with the sorting logic. All the events that are generated inside the EMs are collected here and are transferred to the global Alarm/Warning/Status DBs
UN_06_MiscFunctions	This block can be used for Unit auxiliary functions, such as filling in the PDI DB or OEE calculations. By default, it contains logic to update any statuses communicated via the PackTags and to update the real time OEE values
UN DB	This block contains the Unit's PackML status bits, together with a list of active events that are triggered on the Unit level
InstUN_00_Main	This block is the instance datablock for UN_00_Main. It also contains the instance data for all other UN level blocks. This data is typically not accessed or read directly so it is located inside a sub folder

Table 2-20 Elements in UN DB

Name	Data type	Description
PackML_Sts	CPG_typePackML_ModuleSts	PackML Module Status - UN
PackML_Sts_Alarm	CPG_typePackML_ModuleSts	PackML Module Status – UN alarm status
Alarm	CPG_typeEventStatus	Alarm Event Status – UN Status
Warning	CPG_typeEventStatus	Warning Event Status – UN Status
Status	CPG_typeEventStatus	Event Status – UN Status

For more information on the UN DB, refer to section [2.6](#).

2.4 Equipment Module Blocks

Each EM folder contains blocks for its CMs, as well as an EM DB, which contains all the information to be provided to the Unit level (PackML status and Alarms & Events). It also contains the module summation data coming from its CMs.

Table 2-21 Description of the blocks below an EM Folder, exemplary here EM00

Name	Description
EM00_00_Main	This block contains calls for all the other EM blocks listed. It is called directly inside of the main scan (OB1).
EM00_01_EventControl	This block transforms PackML events into state machine actions. If (for example) a category 0 alarm is active, then the EM will receive an Abort command because of logic present in this block.
EM00_02_PackML_ModuleSum	This block summarizes the PackML commands Start, Abort, State complete, etc.) coming from the control modules and from the PackML events in those control modules.
EM00_CMyy_ssss	These blocks each represent a control module contained in EM00. They are each given a unique number and a user definable name. Any control module logic is placed here.
EM00 DB	This block is a datablock containing local data for the equipment module. If the equipment module or any control module under it needs to issue a state machine command or execute any PackML event, it uses data present in this datablock.
InstEM00_00_Main	This block is the instance datablock for EM00_00_Main. It also contains the instance data for all other EM blocks in this folder. This data is typically not accessed or read directly so it is located inside a subfolder.

Table 2-22 Elements in EM DB

Name	Data type	Description
PackML_Sts	Struct of CPG_typePackML_ModuleSts	PackML Module Status – EM and all control modules
Alarm	CPG_typeEventStatus	Alarm Event Status
Warning	CPG_typeEventStatus	Warning Event Status
Status	CPG_typeEventStatus	Event Status

EMxx_00_Main

This block contains calls of each CM block inside the EM. Calls of a CM are conditional on the *disable* bits – if a CM's disable bit is false, the module is executed. If the *disable* bit is true, the CM logic is not executed. *Disable* bits are located in the PackMLCfɡ datablock.

CM blocks may have inputs or outputs if desired. If the CM executes without programming faults, it's ENO sets a "module active" bit. This bit is used to determine if the CM should be included in the summation. If moduleActive is not set, any commands (or lack thereof) from that CM are ignored. *ModuleActive* bits are located in the EM's datablock, in the *PackML_Sts* structure.

The EM's eventControl and ModuleSum blocks are also called. EventControl has a *moduleActive* bit set by its ENO as well. To prevent alarms from this equipment module from affecting the state machine, ensure the *EMxx.PackML_Sts.Alarm.moduleActive* bit is not set. "ModuleSum" does not have a *moduleActive* bit, and neither of these FBs is associated with a *disable* bit or possess any inputs or outputs.

EMxx_01_EventControl

This block contains logic that transforms alarms occurring in the CMs into state machine actions. Data is passed from the CM FBs into the *EMxx_01_EventControl* FB via the EMxx datablock. In order to influence the state machine, *EMxx_01_EventControl* is equipped with its own complete set of PackML commands (Abort, Reset, Start, State Complete, etc.). These are located in the *EMxx.PackML_Sts.Alarm* structure. The commands are set and reset according to the presence of alarms with varying categories. For example the *EMxx.Alarm* structure reports a category 0 alarm, then the *EMxx.PackML_Sts.Alarm.cmd_Abort* bit will be set. The logic in *EMxx_01_EventControl* fully implements the category structure outlined in section [2.2.1](#).

EMxx_01_EventControl also contains logic for the resetting of faults. By default, faults are reset when the state machine is in Resetting, Clearing, Unsuspending, or Unholding, but additional conditions may be added. A call of *CPG_Event* triggers an alarm if faults do not clear. When transitioning into Execute, a command is sent to clear the first fault, if present. The clearing of faults is handled by the *CPG_EventManager* FB, which is called for alarms, warnings, and statuses.

EMxx_02_PackML_ModuleSum

This block contains logic to summarize all commands for the state machine in the manner outlined in section [1.3.2](#). The state machine command bits from all CMs and from *01_EventControl* are combined with the *moduleActive* bits. All tags referenced are located in the EMxx block. If a module is not active, its PackML command bits are ignored in the summation. The results of the summation are stored in the *EMxx.PackML_Sts.UN* structure.

If the state is a waiting state, any control module can cause the overall EM bit to be set. If the state is a waiting state, every control module must set its state complete bit to set the overall EM state complete bit.

For the control module blocks, refer to section [2.5](#)

For the EM datablock, refer to section [2.6](#)

Instance Data

The instance data for *EMxx_00_Main* and any functions called inside main (or called inside those functions, etc.) will be located here. This is achieved by always selecting "Multi-Instance" when calling an FB in the PackML program. The user will not need data located here, but it may be useful to monitor this block when debugging.

2.5 Control Module Blocks

Each control module has its own function block. Each control module must have a unique number and should have a user-defined name. Program the machine logic for that control module here. Machine logic includes the control module's actions (outputs), and any PackML commands, state complete bits, or events.

Each CM also has its own structure underneath *EMxx.PackML_Sts*, where EMxx is the equipment module number. This structure contains a complete set of SC and command bits. When a particular CM sets one of these bits, it should write to its own structure. These bits are automatically summarized into actionable commands for the UN by the *PackML_ModuleSum* blocks.

Each CM contains by default a call of *CPG_ModuleInitialize*. If a command is not used in the module, it will be set to its 'ignorable' value by this block. Commands are ignored if false, SC bits are ignored if true. By default, this is done on the first scan and each time the machine enters "Stopped".

Calls of *CPG_Event* can be placed in the CM for PackML alarms, warnings, and statuses. Connect the condition on which the alarm should trigger, the configured event, the optional prefix, and the event status information. The eventStatus information is located in the EMxx datablock. If the event in question is an alarm, it is *EMxx.Alarm*. If it is a warning, use *EMxx.Warning*, and for statuses use *EMxx.Status*. This connection allows data to flow from the CM FB into the *EMxx_01_EventControl* FB and *UN_05_EventSummation* FB.

If technology objects are used, it is recommended to add them as inputs into the block. This enhances modularity. It may be desirable for other inputs to be added.

If complex, lengthy, or often reused logic is to be implemented, additional function blocks can be defined and called within a CM. It is recommended to make all function calls Multi-Instance.

2.5.1 Control Module Programming Examples

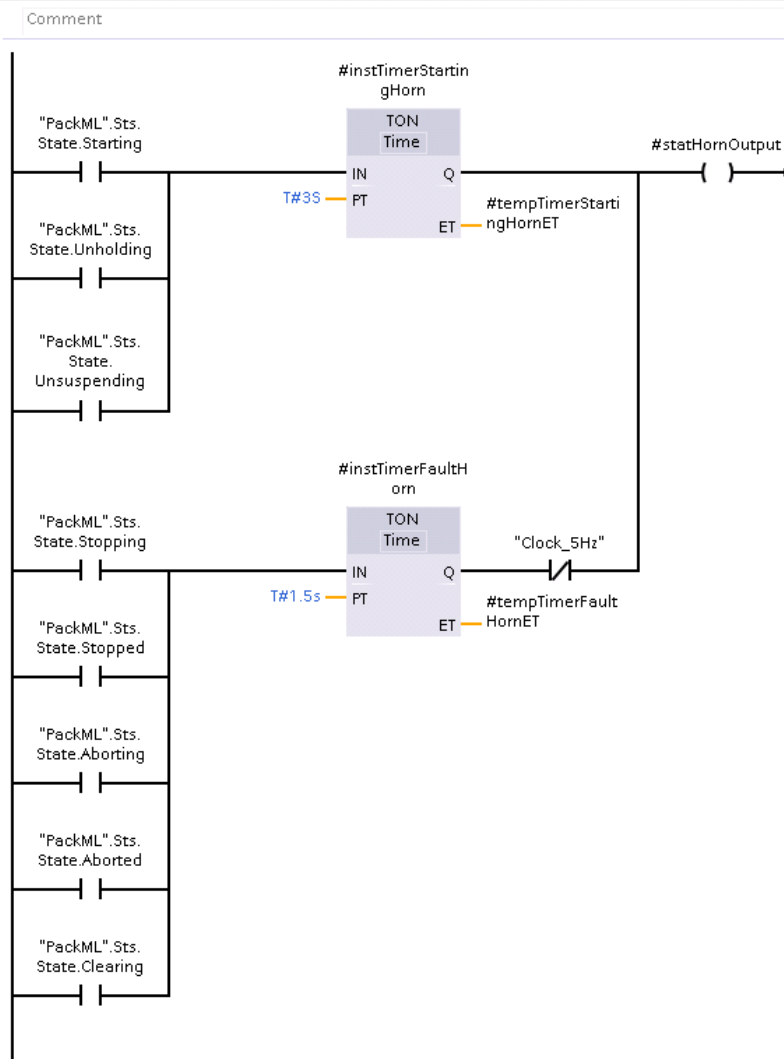
There are four main tasks the control modules accomplish:

1. Setting outputs (%Q bits, or otherwise external signals)
2. Sending commands to change the state machine
3. Sending SC bits to change the state machine
4. Alarm programming

Setting Outputs

The user should determine the PackML states and modes in which the output is set. If there are any extra or alternative conditions for setting the output, these should be determined as well. In the example image, the output is a horn. It sounds 3s after the machine enters Starting, Unholding, or Unsuspending, and 1.5s after entering Stopping, Stopped, Aborting, Aborted, or Clearing.

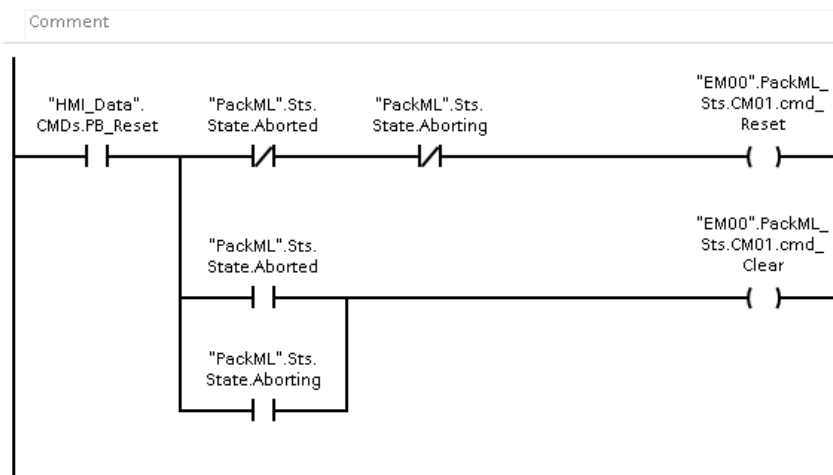
Figure 2-1 Setting a CM output



Sending commands to change the state machine

If a control module could change the state machine, it is necessary to write to the bits in that control module's *PackML_Sts* structure (*EMxx.PackML_Sts.CMyy*). In this example, a reset signal from an HMI sends either a command to clear the state machine, or reset it, depending on the current state. Note that only one CM in the project setting a command bit (Clear, Reset, Start, etc.) is sufficient to change the entire machine state.

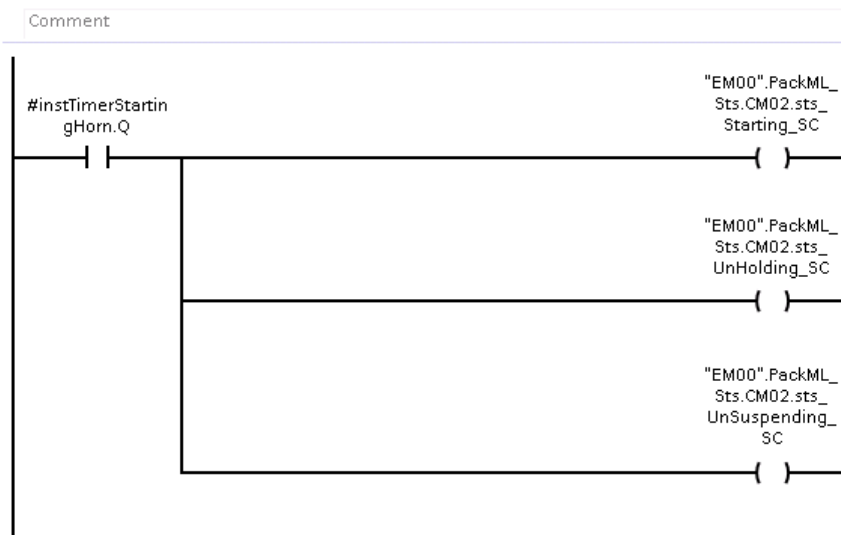
Figure 2-2 Setting an CM State Machine Command



Sending SC bits to change the state machine

If a control module has a task to accomplish in an acting state, then the control module should set the SC bit for that state when the task is accomplished. In this example, the CM is a horn. The horn must sound before the machine starts (enters EXECUTE). Only when the horn output is set will the SC bit for the states STARTING, UNHOLDING, and UNSUSPENDING is set. This way, the machine will not proceed from STARTING until the horn sounds. All CMs must set their SC bits to move the machine onward – though it is possible to never write to an SC bit at all. If that is the case, then CPG_ModuleInitialize will ensure the SC bit is always set to TRUE.

Figure 2-3 Setting an SC bit

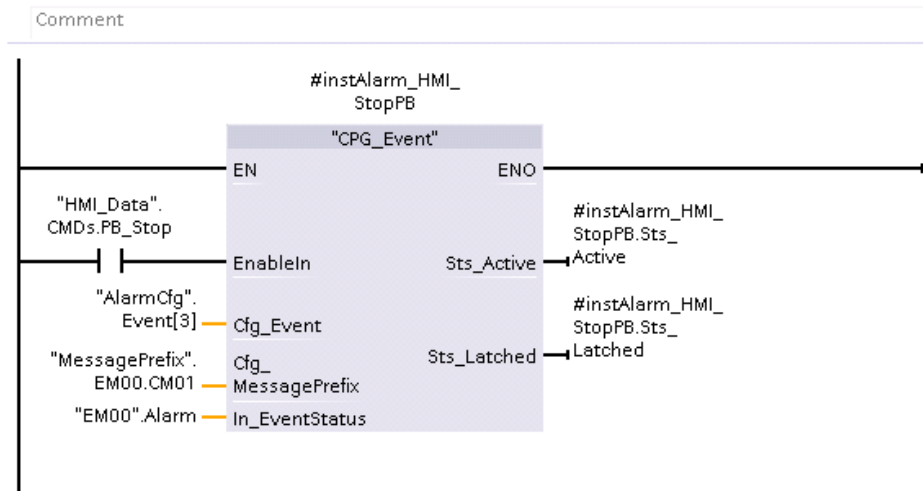


Alarm Programming

Each alarm requires a multi-instanced call of *CPG_Event* FB. Define the alarm in the *AlarmCfg* global DB. Attach the defined alarm to the input *Cfg_Event*. Optionally, attach a prefix to the input *MessagePrefix*. Identify when the alarm should occur and attach that signal to the input *EnableIn*. Lastly, attach the containing EM's *EMxx.Alarm* structure to the *In_EventStatus* input. In this example,

a stop signal from the HMI is used to trigger a category 2 alarm, which will stop the machine. This is preferable to writing to the cmd_Stop bit because it automatically logs the time of the event and provides operators with a reason the machine has stopped.

Figure 2-4 Setting an Alarm



2.6 The EM Data Block

Table 2-23 Content of the global EMxx Data block

Name	Data type	Description
PackML_Sts	Struct	PackML Module Status – EM and all control modules
PackML_Sts.Unit	CPG_typePackML_ModuleSts	PackML Module Status – EM – master status
PackML_Sts.Alarm	CPG_typePackML_ModuleSts	PackML Module Status – EM – alarm status
PackML_Sts.CMyy	CPG_typePackML_ModuleSts	PackML Module Status – CM yy
Alarm	CPG_typeEventStatus	Alarm Event Status – EMxx Status
Warning	CPG_typeEventStatus	PackML Event Status – EMxx Status
Status	CPG_typeEventStatus	PackML Event Status – EMxx Status

The PackML Status structure has 3 main subtypes:

Unit

This holds the EM's master status. The status is the result of the module summation of every CM and the alarms in this EM. The result is used by the Unit level to check if an EM is fully 'State Complete' in order to make the transition to the next OMAC state.

Additionally, if a CM requests a certain state, the information is moved to this structure to notify to the Unit to switch to a certain state.

In the EM's *EMxx_02_PackML_ModuleSum* block, these bits are effectively the output – the alarms and CM bits are summarized to calculate the Unit bits. In the UN's *UN_01_PackML_ModuleSum*, these bits are effectively the input – the EM's state machine commands are combined to give a single command to the overall program.

Alarm

If an alarm in a CM occurs, it may trigger a command for a state transition (e.g. moving to the "Aborted" state). The alarm handling sets the transition command in this structure to TRUE to make sure that the OMAC machine state will move to the required state. The Setting of these bits is done in the *EMxx_01_EventControl* FB in the EM.

CMyy

This structure contains the command and state complete bits for each CM. The programmer must set these bits as needed in the CM FBs. The programmer should only set commands and SC bits that are necessary for their machine control. If a command or SC bit is not written inside a CM, it is ignored by the module summation logic. These bits are the only ones requiring programming in the CM blocks.

NOTE

Both alarm and CMyy structures are read and checked for active bits (state complete or command bits), the result is then written to the Unit structure.

The Unit structure inside the EMxx DB is thus the result of the underlying CMs.

In the *CPG_typePackML_ModuleSts* structure both the CMD (Command) and SC (State complete) bits can be found.

The Command bits can request a certain state transition, while the SC bits will signal that a certain state transition has been completed.

NOTE

The module active bit will state that the current module (EM/CM) is used in the project. If this is set to FALSE, then the Command and State Complete bits of this EM/CM are no longer evaluated in the module summation. To deactivate this bit, set the appropriate bit in the *PackMLCfg.disable* structure to TRUE. This disables the module.

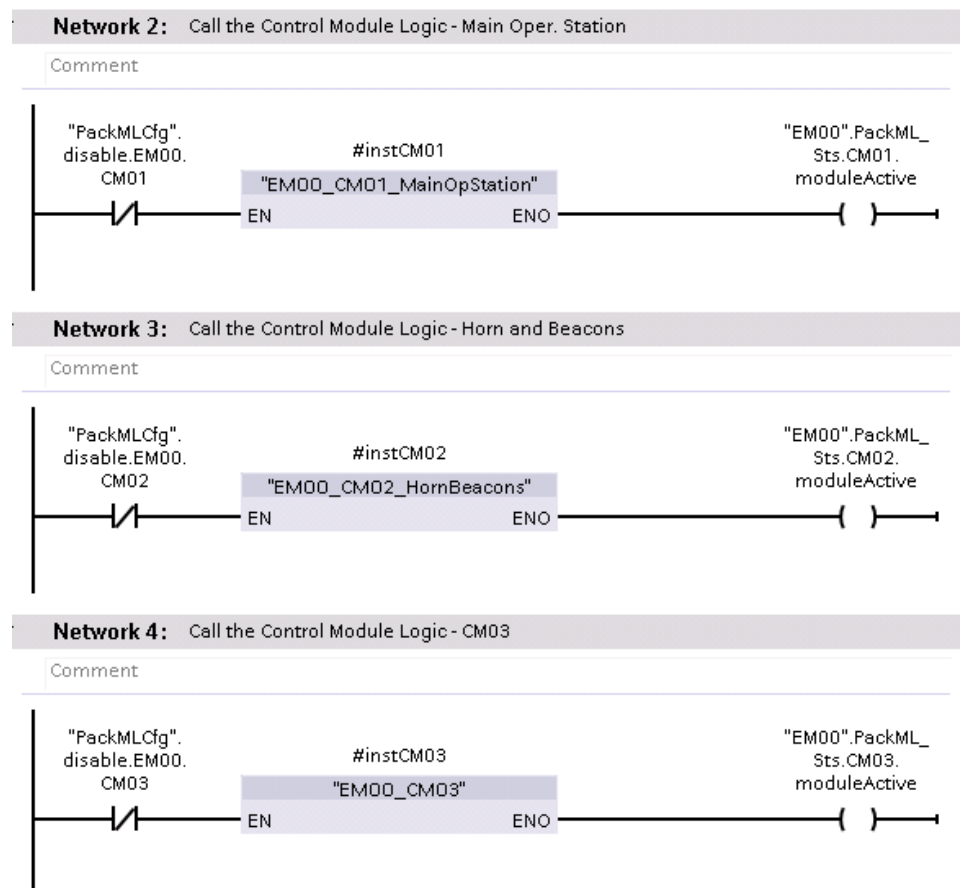
The module ONS bit is a one-shot flag, used to initialize the module status of the CMs in the EM. This occurs with every entry into the stopped state in the *CPG_ModuleInitialize* FB.

In the same EMxx DB the Alarm, Warning and Status structures can be found; all of these structures use the *CPG_typeEventStatus* data type.

These entries are used to collect all the messages (Alarm/Warning/Status) that occur within a single EM.

Inside the main block of the EM *EMxx_00_Main* all the underlying CM blocks are called and the CMs corresponding module active bits are set.

Figure 2-5: FB call structure in the Main EM Block



NOTE

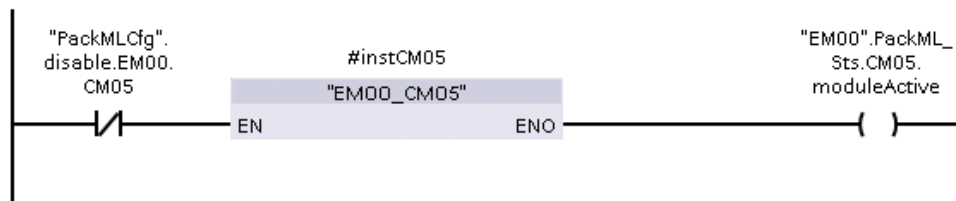
If the *moduleActive* bit is not set, then the commands created in this EM don't take effect. Also, the State model will not wait on this EM to proceed in the OMAC state machine, because the module summation will not evaluate the SC bits from this EM.

The event control block is called together with the module summation block right below CMs calls.

Figure 2-6: FB call structure in the Main EM Block (2)

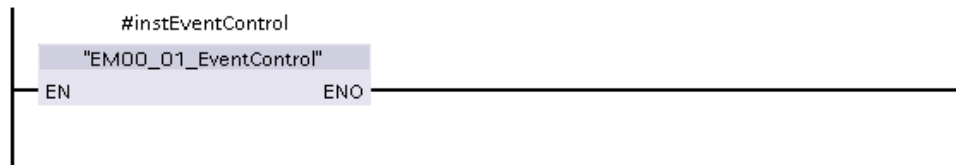
Network 6: Call the Control Module Logic - CM05

Comment



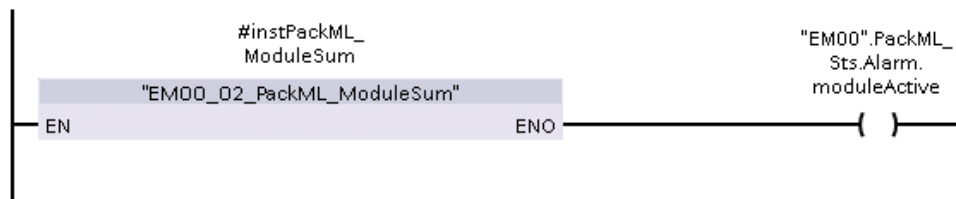
Network 7: Call the Event Control Logic

Comment



Network 8: Call the PackML Module Summation Logic

Comment



The event control block summarizes all the events of a specific category (alarm, warning or event) that occurred within this EM. Later on, this data will be treated by the Unit.

The module summation block summarizes all the state complete bits and commands coming from each CM into the EM structure.

2.7 LMPLV30 Supporting Blocks

The CPG Template is constructed using several blocks from the LPMLV30 library. Refer to its documentation for more information.

Table 2-24 Used blocks from LPMLV30 library

Name	Description
LPMLV30_UnitModeStateManager	The LPMLV30_UnitModeStateManager FB is the main part of the block library LPMLV30 and manages the transitions between the unit modes and states according to the OMAC standard
LPMLV30_UnitModeStateTimes	The LPMLV30_UnitModeStateTimes FB counts the time in seconds for every state in every unit mode.
LPMLV30_ConfigureDisabledStates	This function allows the user to set the state configuration for every unit mode in LPMLV30_UnitModeStateManager FB.
LPMLV30_ConfigureDisabledUnitModes	This function allows the user to set the unit mode configuration for the LPMLV30_UnitModeStateManager FB.
LPMLV30_GetUnitModeStateNamesAsString	The function LPMLV30_GetUnitModeStateNamesAsString provides the unit mode and state names as strings. The default names can be edited in the PLC data type typeLPMLV30_NamesConfiguration.

2.8 CPG Supporting Blocks

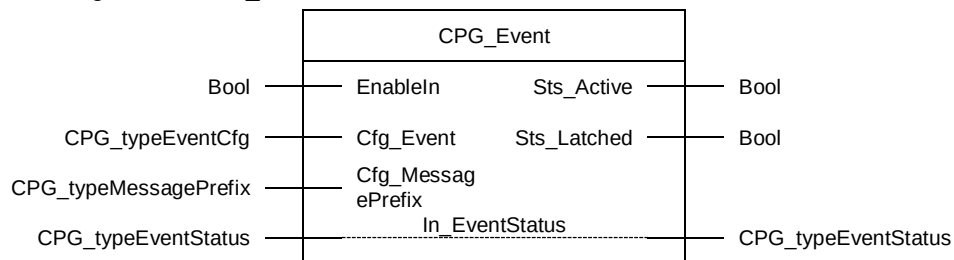
The CPG supporting blocks implement the summation logic for events and states.

2.8.1 CPG_Event

This function is required in control modules to signal an event. It does this by copying the data from the active alarm to the EM's event collection data block.

This function block is called for every Alarm/Warning/Status that must be created.

Figure 2-7: CPG_Event



Input parameters

Table 2-48 CPG_Event input parameters

Name	Data type	Default value	Comment
EnableIn	Bool	FALSE	Enable the FB call.
Cfg_Event	CPG_typeEventCfg		configuration data structure for this event
Cfg_MessagePrefix	CPG_typeMessagePrefix		Message Prefix value for event messages

Output parameters

Table 2-48 CPG_Event output parameters

Name	Data type	Default value	Comment
Sts_Active	Bool	FALSE	event is currently active
Sts_Latched	Bool	FALSE	event has been active since last reset command

InOut parameters

Table 2-48 CPG_Event inOut parameters

Name	Data type	Default value	Comment
In_EventStatus	CPG_typeEventStatus		Working event list data section

When the *EnableIn* input is set to TRUE, the configuration data for the configured event, together with the selected prefix and time of occurrence, will be transferred to the *In_EventStatus* collection DB.

The input *cfg_Event* holds a structure that contains all the info about the triggered event. This info is then collected in the *In_EventStatus* parameter.

The following structure is taken over from the *Cfg_Event* input:

ID

It holds a unique alarm number, this value is used in the PackTags DB to transfer it to the upper level systems (e.g. MES), in order to have a log of the times a certain alarm occurred.

Value

It can be used to supply some extra information about the alarm, for example where the alarm occurred physically in the machine.

Message

It contains an array of strings that hold the alarm message in several languages.

Category

This encodes the state machine action that the alarm will trigger, refer to section [2.2.1](#). In the template the categories have the following default configuration, but users can change it according to their needs:

The message prefix structure adds a string before the alarm text; in this way extra information can be added to the alarm string.

NOTE

The size of the strings has a substantial effect on the program size: try to keep this as low as possible.

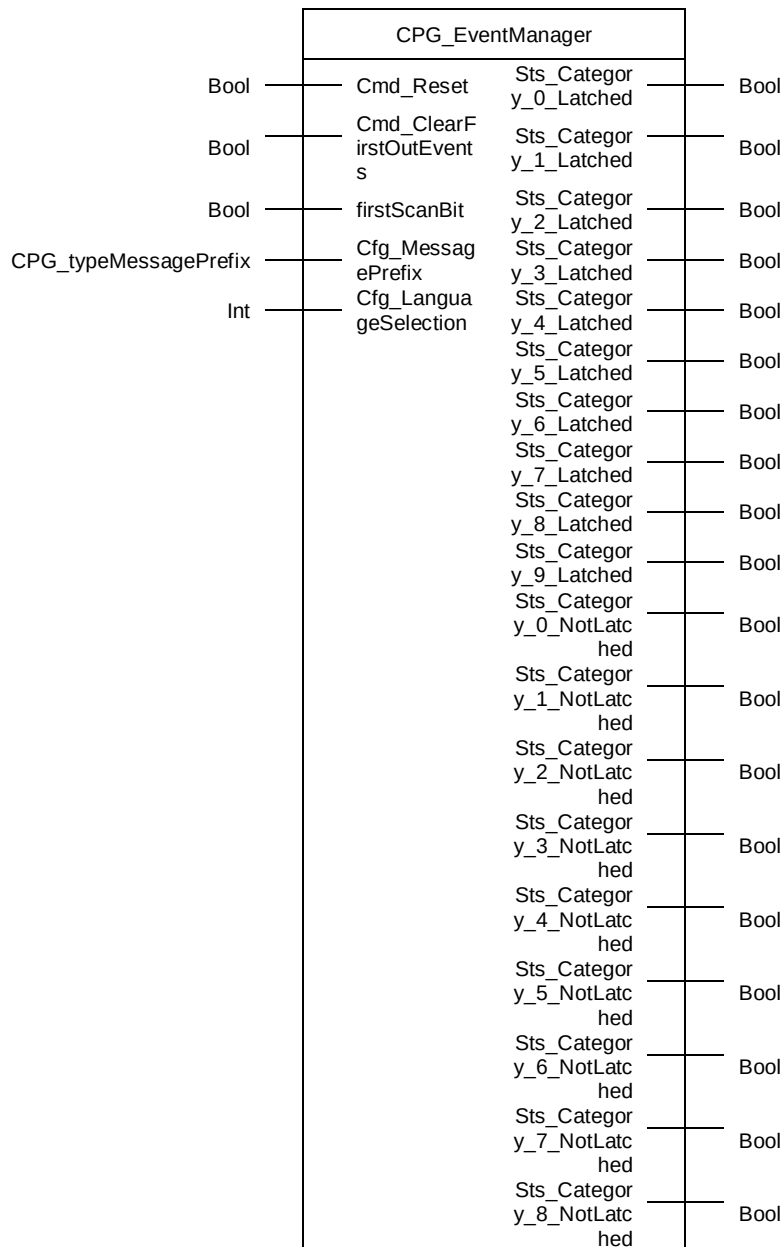
2.8.2 CPG_EventManager

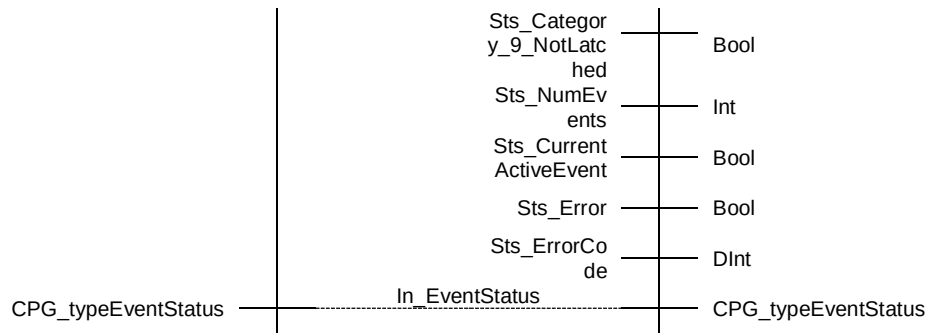
The event manager function block checks whether an alarm of a certain category is active within an EM. A function block call is needed for each event structure inside the EM data block (Alarms, Warnings, Events). It also takes care of resetting the active event structure. This block is called within each EM, in the FB *EMxx_01_EventControl*.

The output values of the block have two different categories, either latched or not latched.

A *Latched* output remains set to TRUE, even when the alarm is no longer active, until a *Cmd_Reset* is performed. *NotLatched* output is set to TRUE as long as an alarm of that category is active.

Figure 2-8: CPG_EventManager





Input parameters

Table 2-48 CPG_EventManager input parameters

Name	Data type	Default value	Comment
Cmd_Reset	Bool	FALSE	Reset Command
Cmd_ClearFirstOutEvents	Bool	FALSE	Command to clear First Out events
firstScanBit	Bool	FALSE	Bit used to tell logic that this is the first scan of PLC
Cfg_MessagePrefix	CPG_typeMessagePrefix		Message Prefix value for event messages multi-lingual
Cfg_LanguageSelection	Int	0	Current Language: 0=EN, 1=DE, 2=SP/ES, 3=IT, 4=FR

Output parameters

Table 2-48 CPG_EventManager output parameters

Name	Data type	Default value	Comment
Sts_Category_0_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 0
Sts_Category_1_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 1
Sts_Category_2_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 2
Sts_Category_3_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 3
Sts_Category_4_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 4
Sts_Category_5_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 5
Sts_Category_6_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 6
Sts_Category_7_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 7
Sts_Category_8_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 8
Sts_Category_9_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 9
Sts_Category_0_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event)

Name	Data type	Default value	Comment
			status) - CAT 0
Sts_Category_1_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 1
Sts_Category_2_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 2
Sts_Category_3_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 3
Sts_Category_4_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 4
Sts_Category_5_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 5
Sts_Category_6_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 6
Sts_Category_7_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 7
Sts_Category_8_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 8
Sts_Category_9_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 9
Sts_NumEvents	Int	0	Number of events in the In_EventStatus list
Sts_CurrentActiveEvent	Bool	FALSE	Status - TRUE = if any active events
Sts_Error	Bool	FALSE	FB Error exist
Sts_ErrorCode	DInt	0	FB Error code

InOut parameters

Table 2-48 CPG_EventManager inOut parameters

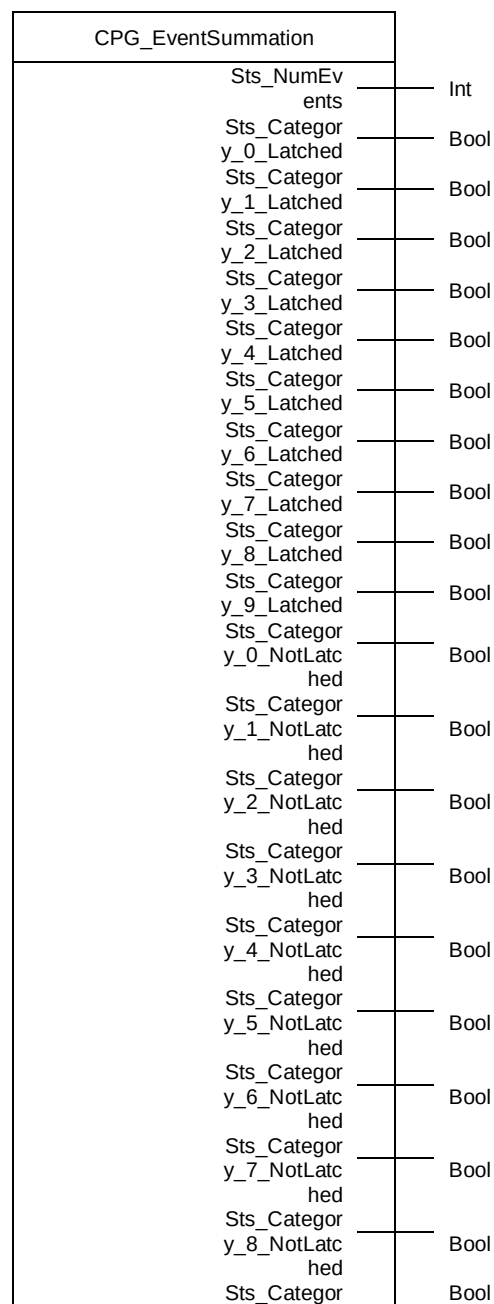
Name	Data type	Default value	Comment
In_EventStatus	CPG_typeEventStatus		Working event list data section

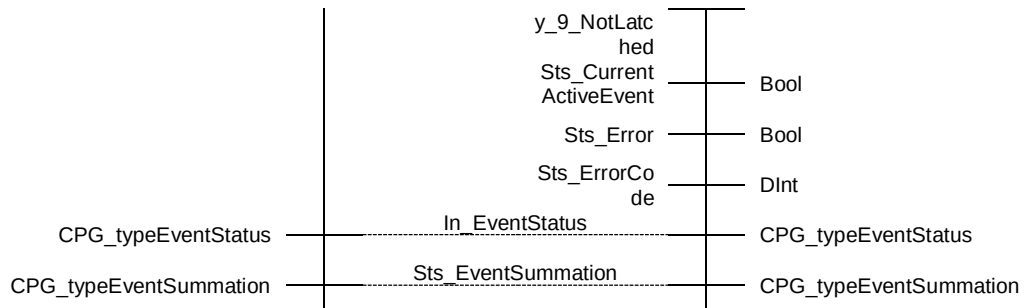
2.8.3 CPG_EventSummation

This block summarizes the data from the equipment modules (from *FB_EventManager*) and creates one list of events for the Unit machine. It summarizes the data by finding the Unit machine first out event and summarizes which categories of events are active. *CPG_EventSummationBegin* initializes the summation and *CPG_EventSummationEnd* finalizes the summation.

The block has to be called for each EM inside the project. The sequence of blocks instances must be respected (*CPG_SummationBegin*, *CPG_EventSummation*, *CPG_SummationEnd*) and there should be a call for each type (Alarms, Warnings, Events).

Figure 2-9: CPG_EventSummation





Output parameters

Table 2-48 CPG_EventSummation output parameters

Name	Data type	Default value	Comment
Sts_NumEvents	Int	0	Number of active events
Sts_Category_0_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 0
Sts_Category_1_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 1
Sts_Category_2_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 2
Sts_Category_3_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 3
Sts_Category_4_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 4
Sts_Category_5_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 5
Sts_Category_6_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 6
Sts_Category_7_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 7
Sts_Category_8_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 8
Sts_Category_9_Latched	Bool	FALSE	Event Category Status (on until reset cmd) - CAT 9
Sts_Category_0_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 0
Sts_Category_1_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 1
Sts_Category_2_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 2
Sts_Category_3_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 3
Sts_Category_4_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 4
Sts_Category_5_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 5
Sts_Category_6_NotLatched	Bool	FALSE	Event Cat Status, not

Name	Data type	Default value	Comment
ed			latched (=active event status) - CAT 6
Sts_Category_7_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 7
Sts_Category_8_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 8
Sts_Category_9_NotLatched	Bool	FALSE	Event Cat Status, not latched (=active event status) - CAT 9
Sts_CurrentActiveEvent	Bool	FALSE	Status - TRUE = if any active events
Sts_Error	Bool	FALSE	FB Error exist
Sts_ErrorCode	DInt	0	FB Error code

InOut parameters

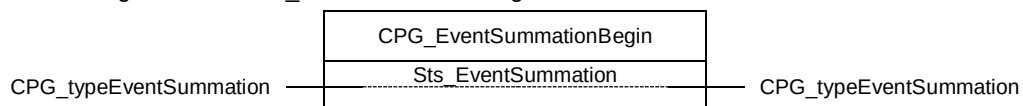
Table 2-48 CPG_EventSummation inOut parameters

Name	Data type	Default value	Comment
In_EventStatus	CPG_typeEventStatus		Working event list data section
Sts_EventSummation	CPG_typeEventSummation		Working event list data section

2.8.4 CPG_EventSummationBegin

This block initializes the variables in the Alarm/Warning/Status data block during the first scan of the PLC program. This block is called in the unit level in *UN_05_EventSummation* once for every event type (Alarm/Warning/Status).

Figure 2-10: CPG_EventSummationBegin



InOut parameters

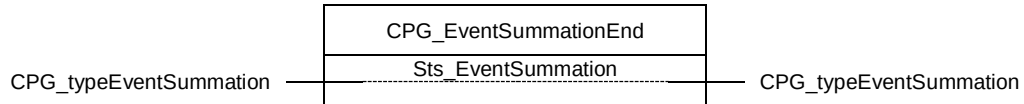
Table 2-48 CPG_EventSummationBegin inOut parameters

Name	Data type	Default value	Comment
EventSummation	CPG_typeEventSummation		Working event list data section

2.8.5 CPG_EventSummationEnd

A call of the *CPG_EventSummationEnd* block is necessary after all *CPG_EventSummationBegin* and *CPG_EventSummation* calls. This block collects the results of the *CPG_EventSummation* blocks and writes the result to each category. This block is called for every event type (Alarm/Warning/Status).

Figure 2-11: CPG_EventSummationEnd



InOut parameters

Table 2-48 CPG_EventSummationEnd inOut parameters

Name	Data type	Default value	Comment
EventSummation	CPG_typeEventSummation		Working event list data section

2.8.6 CPG_EventSortFilter

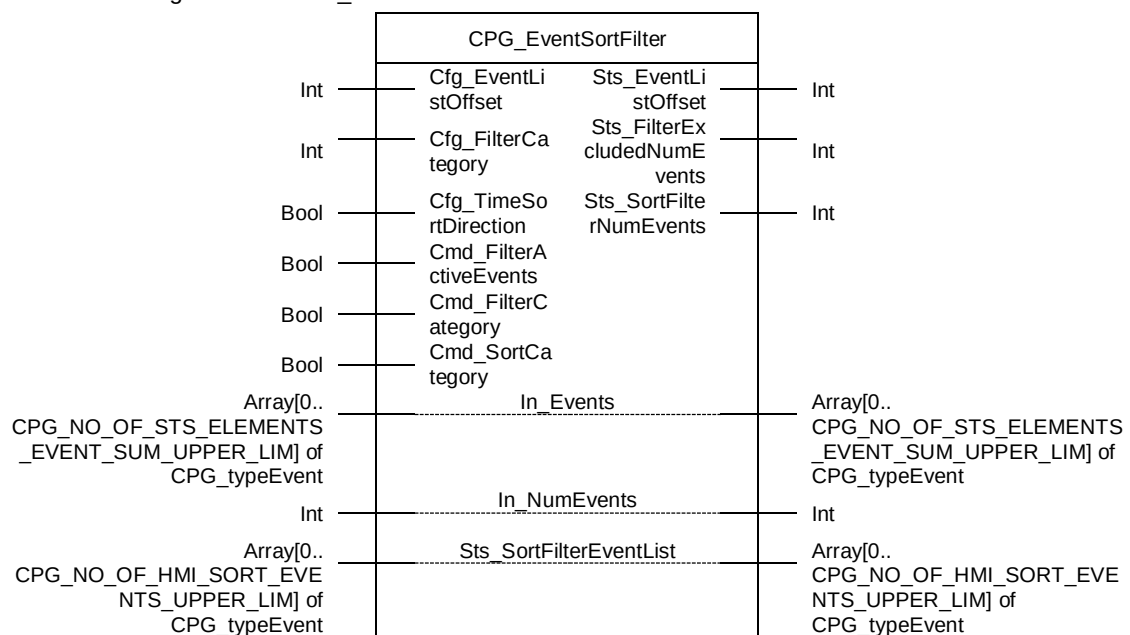
The *CPG_EventSortFilter* block allows the sorting/filtering of Alarm & Warning event types. This block is called in the *UN_05_EventSummation* within the Unit level in the project.

For each event type, a separate FB call has to be made. Sorting can be done on active events, sorting by category, or display only a specific category.

The function block reads the active events from the Unit machine alarm/warning collection data block and moves the sorted result list to the *HMI_Data* sorted list.

The *HMI_Data* data block is then used to display the sorted alarms on the HMI.

Figure 2-12: CPG_EventSortFilter



Input parameters

Table 2-48 CPG_EventSortFilter input parameters

Name	Data type	Default value	Comment
Cfg_EventListOffset	Int	0	Commanded event list offset (used for scrolling)
Cfg_FilterCategory	Int	0	Filtered category number
Cfg_TimeSortDirection	Bool	FALSE	1=newest first; 0=oldest first
Cmd_FilterActiveEvents	Bool	FALSE	1=only show active events; 0=show all events
Cmd_FilterCategory	Bool	FALSE	Filter events to show only selected category number
Cmd_SortCategory	Bool	FALSE	1=Sort events by ascending category number

Output parameters

Table 2-48 CPG_EventSortFilter output parameters

Name	Data type	Default value	Comment
Sts_EventListOffset	Int	0	Actual event list offset
Sts_FilterExcludedNumEvents	Int	0	The number of events excluded by filtering
Sts_SortFilterNumEvents	Int	0	Number of events in the sorted or filtered event list

InOut parameters

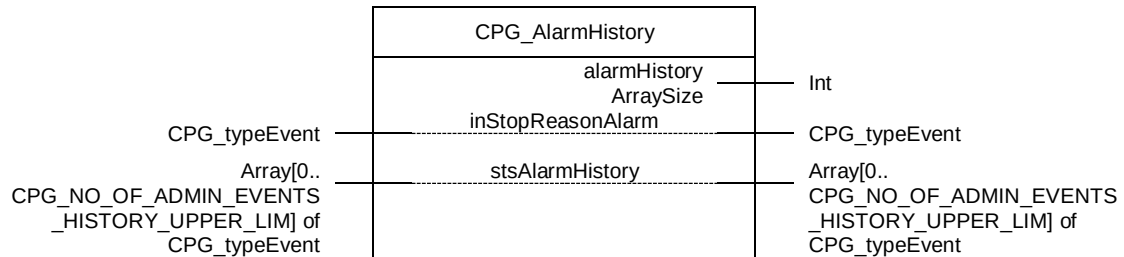
Table 2-48 CPG_EventSortFilter inOut parameters

Name	Data type	Default value	Comment
In_Events	Array[0..CPG_NO_OF_STS_ELEMENTS_EVENT_SUM_UPPER_LIM] of CPG_typeEvent		Input Event list to be sorted and/or filtered
In_NumEvents	Int	0	Number of events to be sorted and/or filtered
Sts_SortFilterEventList	Array[0..CPG_NO_OF_HMI_SORT_EVENTS_UPPER_LIM] of CPG_typeEvent		Output sorted and/or filtered event list

2.8.7 CPG_AlarmHistory

This function block keeps track of the first fault of the Unit, it copies it to the PackTags data block. It should be called within the Unit level in the *EventSummation* for the Alarms and Warnings structures.

Figure 2-13: CPG_AlarmHistory



Output parameters

Table 2-48 CPG_AlarmHistory output parameters

Name	Data type	Default value	Comment
alarmHistoryArraySize	Int	0	Size of Alarm History - number of alarm array elements

InOut parameters

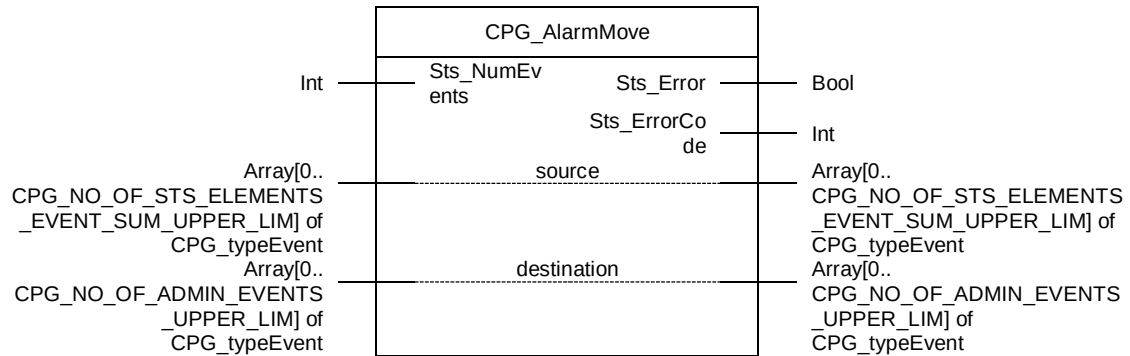
Table 2-48 CPG_AlarmHistory inOut parameters

Name	Data type	Default value	Comment
inStopReasonAlarm	CPG_typeEvent		Input - Stop Reason Alarm (structure)
stsAlarmHistory	Array[0..CPG_NO_OF_ADMIN_EVENTS_HISTORY_UPPER_LIM] of CPG_typeEvent		Status- Alarm History Log

2.8.8 CPG_AlarmMove

This function block copies the alarms from the CPG Template to the PackTags data block. It should be called within the Unit level in the *EventSummation* for the Alarms and Warnings structures.

Figure 2-14: CPG_AlarmMove



Input parameters

Table 2-48 CPG_AlarmMove input parameters

Name	Data type	Default value	Comment
Sts_NumEvents	Int	0	Number of Active Alarms

Output parameters

Table 2-48 CPG_AlarmMove output parameters

Name	Data type	Default value	Comment
Sts_Error	Bool	FALSE	Error Status. 0=no error, 1=error
Sts_ErrorCode	Int	0	Error code / number

InOut parameters

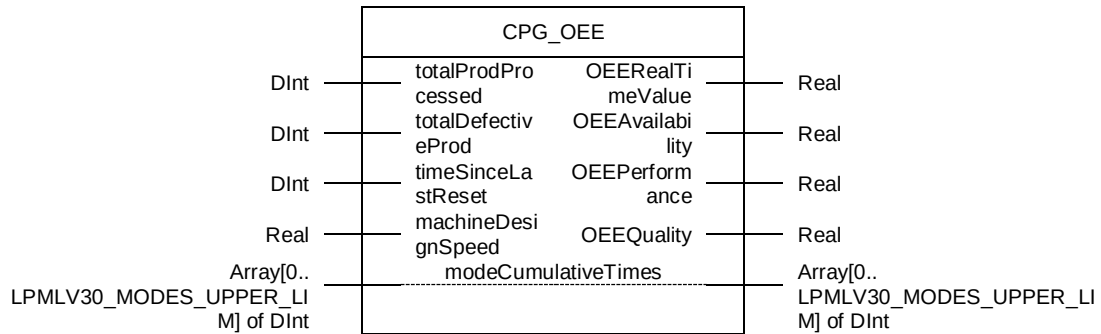
Table 2-48 CPG_AlarmMove inOut parameters

Name	Data type	Default value	Comment
source	Array[0..CPG_NO_OF_STS_ELEMENTS_EVENT_SUM_UPPER_LIM] of CPG_typeEvent		source location of the machine alarms
destination	Array[0..CPG_NO_OF_ADMIN_EVENTS_UPPER_LIM] of CPG_typeEvent		destination location of the PackTag alarms

2.8.9 CPG_OEE

This optional function block is used to calculate the overall equipment effectiveness (OEE) values of the Unit. This function block uses the *LPMLV30_UnitModeStateTimes* FB from the LPMLV30 library.

Figure 2-15: CPG_OEE



Input parameters

Table 2-48 CPG_AlarmMove input parameters

Name	Data type	Default value	Comment
totalProdProcessed	DInt	0	Total Number OF Products Processed
totalDefectiveProd	DInt	0	Total Number OF Defective Products
timeSinceLastReset	DInt	0	Total Accumulative TIME since Last RESET
machineDesignSpeed	Real	0.0	Machine Design Speed

Output parameters

Table 2-48 CPG_AlarmMove output parameters

Name	Data type	Default value	Comment
OEERealTimeValue	Real	0.0	REAL TIME OEE value (availability x performance x quality)
OEEAvailability	Real	0.0	OEE - Availability value
OEEPerformance	Real	0.0	OEE - Performance value
OEEQuality	Real	0.0	OEE - Quality value

InOut parameters

Table 2-48 CPG_AlarmMove inOut parameters

Name	Data type	Default value	Comment
modeCumulativeTimes	Array[0..LPMLV30_MODES_UPPER_LIM] of DInt		Mode Cumulative Times

2.8.10 CPG_ModuleInitialize

Inside every CM block a module initialization function block is called, in order to initialize the CM's data structure. Not every CM uses all the State Complete bits or Command bits that are available in the structure.

For example, consider a CM where only a pneumatic cylinder is controlled, and no actions are required for this in the holding state. Then this CM should permanently set the Holding State Complete bit to TRUE in order to proceed in the OMAC machine state.

The module initialization FB is called at the first scan of the PLC and when the state machine state enters the "Stopped" state. The FB sets all the SC bits of a certain CM to TRUE. If a certain SC bit is not used in the CM, this will not make the application wait.

Figure 2-16 Call of the module initialization block example

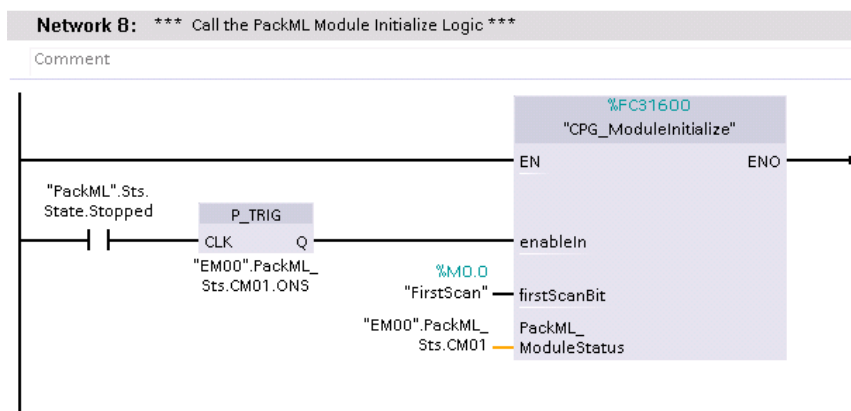
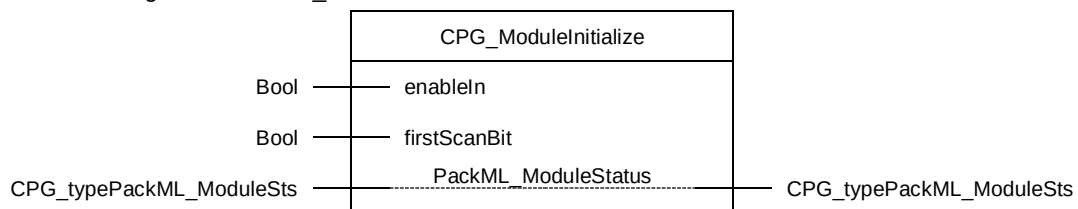


Figure 2-17: CPG_ModuleInitialize



Input parameters

Table 2-48 CPG_ModuleInitialize input parameters

Name	Data type	Default value	Comment
enableIn	Bool	FALSE	Enable the FB call.
firstScanBit	Bool	FALSE	Bit used to tell logic that this is the first scan of PLC

InOut parameters

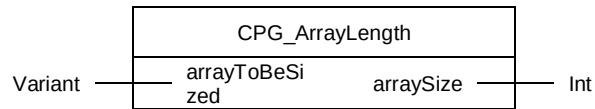
Table 2-48 CPG_ModuleInitialize inOut parameters

Name	Data type	Default value	Comment
PackML_ModuleStatus	CPG_typePackML_ModuleSts		PackML Module Status UDT

2.8.12 CPG_ArrayLength

Function to determine the length of any array using optimized S7-1200/1500 data blocks.

Figure 2-19: CPG_ArrayLength



Input parameters

Table 2-48 CPG_ArrayLength input parameters

Name	Data type	Default value	Comment
arrayToBeSized	Variant		Array to be sized

Output parameters

Table 2-48 CPG_ArrayLength output parameters

Name	Data type	Default value	Comment
arraySize	Int	0	Number of elements (size) in an array

2.9 CPG Template Data types

The CPG Template provides the following CPG user data types.

Table 2-25: Overview of CPG user data types (UDTs)

Name	Comment
CPG_typeEvent	This UDT defines the PackML event structure (ID, Value, Text, Category, time of occurrence, time of acknowledgement)
CPG_typeEventCfg	This UDT defines the PackML event definition. The message text is an array to hold the desired message in all project languages. This type is used for the multilingual configuration of events, but the events passed throughout the program are CPG_typeEvent
CPG_typeEventStatus	This UDT defines the events occurring in each EM (and the UN). It holds what events have occurred, how many, and what categories are present
CPG_typeEventSummation	This UDT defines the structure of the fully summarized events for the machine
CPG_typeMessagePrefix	This UDT aids in the multi-lingual configuration of message prefixes
CPG_typePackML_ModuleSts	This UDT defines a complete set of PackML commands. Each module in the project has its own instance
CPG_typeParameter(DINT)	This UDT defines a PackTag parameter where the value is a double integer
CPG_typeParameter(REAL)	This UDT defines a PackTag parameter where the value is a real
CPG_typeParameter(STRING)	This UDT defines a PackTag parameter where the value is a string
CPG_typeProductCount(DINT)	This UDT defines the product count structure in the PackTags
CPG_typeSortFilter	This UDT aids in the sorting and display of events on the HMI

The LPMLV30 blocks also come with a set of data types. For further explanation of the LPMLV30 data types, refer to the LPMLV30 manual directly.

Table 2-26: Overview of LPMLV30 user data types (UDTs)

Name	Comment
typeLPMLV30_Configuration	This UDT helps track which modes and states are active
typeLPMLV30_Diagnostics	This UDT defines diagnostics structures for the state manager
typeLPMLV30_DiagnosticsEntry	This UDT defines diagnostics to be displayed on the HMI
typeLPMLV30_NamesConfiguration	This UDT aids in the multilingual naming of modes and states

2.9.1 CPG_typeEvent

Table 2-27: Parameter of CPG_typeEvent

Name	Data type	Default value	Comment
Trigger	Bool	FALSE	1=Alarm is currently active
ID	DInt	0	Alarm Message Identification Number, see PackTags Reason Code (e.g. 1=E-Stop pushed, 3=Mains turned off, etc.)
Value	DInt	0	Alarm Message Value to break down the Alarm.ID to greater detail
Message	String[60]	"	Alarm Message Text (= message prefix + alarm message)
Category	DInt	0	Alarm Category Value (0-9)
DateTime	Array[0..6] of DInt		Date and Time the Event Occurred
DateTimeDTL	DTL	DTL#1970-01-01-00:00:00	Date and LTime (DTL) the event was initially triggered (date and time-of-day information in nanoseconds since 01/01/1970 0:0)
AckDateTime	Array[0..6] of DInt		Date and Time the Event was Acknowledged
AckDateTimeDTL	DTL	DTL#1970-01-01-00:00:00	Date and LTime (DTL) the event was acknowledged (date and time-of-day information in nanoseconds since 01/01/1970 0:0)

2.9.2 CPG_typeEventCfg

Table 2-28: Parameter of CPG_typeEventCfg

Name	Data type	Default value	Comment
ID	Int	0	ID Value - from PackTags Reason Code, 1=E-Stop pushed, 3=Main off, etc.
value	Int	0	Reason Code 'Value' - allows for detailed breakdown of ID, ie 1=Discharge
message	Array[0..LPMLV30_LANGUAGES_UPPER_LIM] of String[60]		Alarm Message - 0=English, 1=German, 2=Spanish, 3=Italian, 4=French
category	Int	0	Category Value (0-9)

2.9.3 CPG_typeEventStatus

Table 2-29: Parameter of CPG_typeEventStatus

Name	Data type	Default value	Comment
sts_FirstOutEvent	CPG_typeEvent		Event Info of the very first active event (irregardless of the category)
sts_FirstOutEvent Cat	Array[0..9] of CPG_typeEvent		[No comment]
sts_Events	Array[0..CPG_NO_OF_ELEMENTS_EVENT_STATUS_UPPER_LIM] of		[No comment]

Name	Data type	Default value	Comment
	CPG_typeEvent		
sts_NumEvents	Int	0	Number of all current active events
sts_NumAllEvents	Int	0	Number of all current events
out_ZeroEvent	CPG_typeEvent		null event information
sts_Category_0_Latched	Bool	FALSE	Category 0 Event - latched (require reset cmd)
sts_Category_1_Latched	Bool	FALSE	Category 1 Event - latched (require reset cmd)
sts_Category_2_Latched	Bool	FALSE	Category 2 Event - latched (require reset cmd)
sts_Category_3_Latched	Bool	FALSE	Category 3 Event - latched (require reset cmd)
sts_Category_4_Latched	Bool	FALSE	Category 4 Event - latched (require reset cmd)
sts_Category_5_Latched	Bool	FALSE	Category 5 Event - latched (require reset cmd)
sts_Category_6_Latched	Bool	FALSE	Category 6 Event - latched (require reset cmd)
sts_Category_7_Latched	Bool	FALSE	Category 7 Event - latched (require reset cmd)
sts_Category_8_Latched	Bool	FALSE	Category 8 Event - latched (require reset cmd)
sts_Category_9_Latched	Bool	FALSE	Category 9 Event - latched (require reset cmd)
sts_Category_0_NotLatched	Bool	FALSE	Category 0 Event - NotLatched (currently active)
sts_Category_1_NotLatched	Bool	FALSE	Category 1 Event - NotLatched (currently active)
sts_Category_2_NotLatched	Bool	FALSE	Category 2 Event - NotLatched (currently active)
sts_Category_3_NotLatched	Bool	FALSE	Category 3 Event - NotLatched (currently active)
sts_Category_4_NotLatched	Bool	FALSE	Category 4 Event - NotLatched (currently active)
sts_Category_5_NotLatched	Bool	FALSE	Category 5 Event - NotLatched (currently active)
sts_Category_6_NotLatched	Bool	FALSE	Category 6 Event - NotLatched (currently active)
sts_Category_7_NotLatched	Bool	FALSE	Category 7 Event - NotLatched (currently active)
sts_Category_8_NotLatched	Bool	FALSE	Category 8 Event - NotLatched (currently active)
sts_Category_9_NotLatched	Bool	FALSE	Category 9 Event - NotLatched (currently active)
sts_CurrentActiveEvent	Bool	FALSE	Active # of Event

2.9.4 CPG_typeEventSummation

Table 2-30: Parameter of CPG_typeEventSummation

Name	Data type	Default value	Comment
sts_FirstOutEvent	CPG_typeEvent		First Out (Stop Reason) Event
sts_FirstOutEvent Cat	Array[0..9] of CPG_typeEvent		First Out (Stop Reason) Event by Category
sts_Events	Array[0..CPG_NO_OF_STS_ELEMENTS_EVENT_SUM_UPPER_LIM] of CPG_typeEvent		All Active Events
sts_NumEvents	Int	0	Number of Active Events
sts_Category_0_Latched	Bool	FALSE	Category 0 Event - latched (require reset cmd)
sts_Category_1_Latched	Bool	FALSE	Category 1 Event - latched (require reset cmd)
sts_Category_2_Latched	Bool	FALSE	Category 2 Event - latched (require reset cmd)
sts_Category_3_Latched	Bool	FALSE	Category 3 Event - latched (require reset cmd)
sts_Category_4_Latched	Bool	FALSE	Category 4 Event - latched (require reset cmd)
sts_Category_5_Latched	Bool	FALSE	Category 5 Event - latched (require reset cmd)
sts_Category_6_Latched	Bool	FALSE	Category 6 Event - latched (require reset cmd)
sts_Category_7_Latched	Bool	FALSE	Category 7 Event - latched (require reset cmd)
sts_Category_8_Latched	Bool	FALSE	Category 8 Event - latched (require reset cmd)
sts_Category_9_Latched	Bool	FALSE	Category 9 Event - latched (require reset cmd)
sts_Category_0_NotLatched	Bool	FALSE	Category 0 Event - NotLatched (currently active)
sts_Category_1_NotLatched	Bool	FALSE	Category 1 Event - NotLatched (currently active)
sts_Category_2_NotLatched	Bool	FALSE	Category 2 Event - NotLatched (currently active)
sts_Category_3_NotLatched	Bool	FALSE	Category 3 Event - NotLatched (currently active)
sts_Category_4_NotLatched	Bool	FALSE	Category 4 Event - NotLatched (currently active)
sts_Category_5_NotLatched	Bool	FALSE	Category 5 Event - NotLatched (currently active)
sts_Category_6_NotLatched	Bool	FALSE	Category 6 Event - NotLatched (currently active)
sts_Category_7_NotLatched	Bool	FALSE	Category 7 Event - NotLatched (currently active)
sts_Category_8_NotLatched	Bool	FALSE	Category 8 Event - NotLatched (currently active)
sts_Category_9_NotLatched	Bool	FALSE	Category 9 Event - NotLatched (currently active)

Name	Data type	Default value	Comment
sts_CurrentActive Event	Bool	FALSE	Event Active = 1, not active = 0

2.9.5 CPG_typeMessagePrefix

Table 2-31: Parameter of CPG_typeMessagePrefix

Name	Data type	Default value	Comment
messagePrefix	Array[0..LPMLV30_LAN GUAGES_UPPER_LIM] of String[30]		Multi-Lingual Message for Prefix (i.e 0=EN, 1=DE, 2=SP/ES, 3=IT, 4=FR)

2.9.6 CPG_typePackML_ModuleSts

Table 2-32: Parameter of CPG_typePackML_ModuleSts

Name	Data type	Default value	Comment
cmd_Reset	Bool	FALSE	Reset
sts_Resetting_SC	Bool	FALSE	Resetting State Complete
cmd_Start	Bool	FALSE	Start
sts_Starting_SC	Bool	FALSE	Starting State Complete
cmd_Stop	Bool	FALSE	Stop
sts_Stopping_SC	Bool	FALSE	Stopping State Complete
cmd_Hold	Bool	FALSE	Hold
sts_Holding_SC	Bool	FALSE	Holding State Complete
cmd_UnHold	Bool	FALSE	UnHold
sts_UnHolding_SC	Bool	FALSE	UnHolding State Complete
cmd_Suspend	Bool	FALSE	Suspend
sts_Suspending_SC	Bool	FALSE	Suspending State Complete
cmd_UnSuspend	Bool	FALSE	UnSuspend
sts_UnSuspending_SC	Bool	FALSE	UnSuspending State Complete
cmd_Abort	Bool	FALSE	Abort
sts_Aborting_SC	Bool	FALSE	Aborting State Complete
cmd_Clear	Bool	FALSE	Clear
sts_Clearing_SC	Bool	FALSE	Clearing State Complete
sts_Execute_SC	Bool	FALSE	Execute State Complete
sts_Completing_SC	Bool	FALSE	Completing State Complete
moduleActive	Bool	FALSE	Module Active Bit
ONS	Bool	FALSE	OneShot bit

2.9.7 CPG_typeParameterDINT

Table 2-33: Parameter of CPG_typeParameterDINT

Name	Data type	Default value	Comment
ID	DInt	0	Parameter ID - This is the arbitrary (user defined) ID value of the parameter
Name	String[24]	“	Parameter Name - The literal parameter name is used to describe the parameter number and its associated value (e.g. Glue Temp)
Unit	String[5]	“	Parameter Unit of Measure - The parameter unit is used to describe the unit of measure of the parameter and its associated value (e.g. DegF)
Value	DInt	0	Parameter Value - This is the numeric or string value of the parameter (e.g. 100)

2.9.8 CPG_typeParameterReal

Table 2-34: Parameter of CPG_typeParameterReal

Name	Data type	Default value	Comment
ID	DInt	0	Parameter ID - This is the arbitrary (user defined) ID value of the parameter
Name	String[24]	“	Parameter Name - The literal parameter name is used to describe the parameter number and its associated value (e.g. Glue Temp)
Unit	String[5]	“	Parameter Unit of Measure - The parameter unit is used to describe the unit of measure of the parameter and its associated value (e.g. DegF)
Value	Real	0.0	Parameter Value - This is the numeric or string value of the parameter (e.g. 100.2)

2.9.9 CPG_typeParameterString

Table 2-35: Parameter of CPG_typeParameterString

Name	Type	Default value	Comment
ID	DInt	0	Parameter ID - This is the arbitrary (user defined) ID value of the parameter
Name	String[24]	“	Parameter Name - The literal parameter name is used to describe the parameter number and its associated value (e.g. Glue Temp)
Unit	String[5]	“	Parameter Unit of Measure - The parameter unit is used to describe the unit of measure of the parameter and its associated value (e.g. DegF)
Value	String[24]	“	Parameter Value - This is the numeric or string value of the parameter (e.g. '6ES7518-4AP00-0AB0')

2.9.10 CPG_typeProdCountDINT

Table 2-36: Parameter of CPG_typeProdCountDINT

Name	Data type	Default value	Comment
ID	DInt	0	Product ID - This is the arbitrary (user defined) ID value of the consumed production material
Name	String[24]	“	Descriptive name - The name is used to literally describe the material ID, and its associated material
Unit	String[5]	“	Unit of measure - The unit tag is used to describe the names associated with a specific material used by the machine
Count	DInt	0	Value - The count value is used as a variable for displaying information about the amount of consumed
AccCount	DInt	0	Value - The accumulative count value is used as a variable for displaying information about the total amount of consumed production material

2.9.11 CPG_typeSortFilter

Table 2-37: Parameter of CPG_typeSortFilter

Name	Data type	Default value	Comment
cmdFilterActiveEvents	Bool	FALSE	Filter for Active events: 1=only show active events; 0=show all events
cmdFilterCategory	Bool	FALSE	Filter by Category: filter events to show only selected category number
cmdSortCategory	Bool	FALSE	Sort by Category: 1=sort events by ascending category number
cfgTimeSortDirection	Bool	FALSE	Sort by Time: 1=newest first; 0=oldest first
cfgEventListOffset	Int	0	Commanded event list offset (used for scrolling)
cfgFilterCategory	Int	0	Filter by Category: filtered category number

2.10 CPG Template Constants

To aid in configuration, a list of constants is provided. These constants define the size of arrays throughout the CPG Template. To view and edit the values of these constants, navigate to *PLC Tags* -> *CPG_Tags* -> *CPG_Constants*, and then click the "User Constants" tab in the tag table. The constant values are global and can be set in engineering but cannot be changed at runtime. After changing a constant, perform a recompile to automatically resize all arrays.

By convention, constants are named in capital letters to distinguish them from other tags.

Table 2-38: CPG Constants

Name	Data type	Default value	Comment
CPG_NO_OF_ELEMENTS_EVENT_STATU S_UPPER_LIM	Int	29	Number of Alarms, Warning and PackML Status events used by Event Status UDT (Array size of 0..x)
CPG_NO_OF_ADMIN_EVENTS_UPPER_LI M	Int	9	Number of Alarms and Warning events found in the PackTags Admin Area - active alarms / warnings. (Array size of 0..x)
CPG_NO_OF_ADMIN_EVENTS_HISTORY_ UPPER_LIM	Int	9	Number of Alarms and Warning events found in the PackTags Admin Area - historical alarms / warnings. (Array size of 0..x)
CPG_NO_OF_PROD_CONSUMED_COUN T_UPPER_LIM	Int	0	Number of Prod Consumed Counter Array Size (PackTags Admin) (Array size of 0..x)
CPG_NO_OF_PROD_PROCESSED_COUN T_UPPER_LIM	Int	0	Number of Prod Processed Counter Array Size (PackTags Admin) (Array size of 0..x)
CPG_NO_OF_PROD_DEFECTIVE_COUNT _UPPER_LIM	Int	0	Number of Prod Defective Counter Array Size (PackTags Admin) (Array size of 0..x)
CPG_NO_OF_STS_ELEMENTS_EVENT_S UM_UPPER_LIM	Int	99	Number of Alarms, Warning and PackML Status events used by Event Summation UDT (Array size of 0..x)
CPG_NO_OF_HMI_SORT_EVENTS_UPPE R_LIM	Int	19	Number of Alarms and Warning events found for the HMI area - sorted HMI alarms / warnings. (Array size of 0..x)
CPG_NO_OF_CFG_ALARMS_UPPER_LIM	Int	39	Number of Cfg Alarm event configurations (Array size of 0..x)
CPG_NO_OF_CFG_WARNINGS_UPPER_L IM	Int	29	Number of Cfg Warning event configurations (Array size of 0..x)
CPG_NO_OF_CFG_STATUS_UPPER_LIM	Int	19	Number of Cfg Status event configurations (Array size of 0..x)
CPG_NO_OF_STATUS_PARAMETERS_UP PER_LIM	Int	0	Number of Status Parameters (Array size of 0..x)
CPG_NO_OF_COMMAND_PARAMETERS_ UPPER_LIM	Int	0	Number of Command Parameters (Array size of 0..x)
CPG_NO_OF_ADMIN_PARAMETERS_UPP ER_LIM	Int	0	Number of Admin Parameters (Array size of 0..x)

The LPMLV30 blocks also come with a set of constants. Open the User Constants tab of the LPMLV30 tags table to access them. For further explanation of the LPMLV30 tags, refer to the LPMLV30 manual directly.

Table 2-39: LPMLV30 Constants

Name	Data type	Default value	Comment
LPMLV30_MODE_INVALID	DInt	0	OMAC PackMLV30 unit mode Invalid
LPMLV30_MODE_PRODUCTION	DInt	1	OMAC PackMLV30 unit mode Production
LPMLV30_MODE_MAINTENANCE	DInt	2	OMAC PackMLV30 unit mode Maintenance
LPMLV30_MODE_MANUAL	DInt	3	OMAC PackMLV30 unit mode Manual
LPMLV30_MODE_USER_01	DInt	4	OMAC PackMLV30 user-defined unit mode 01
LPMLV30_MODE_USER_02	DInt	5	OMAC PackMLV30 user-defined unit mode 02
LPMLV30_MODE_USER_03	DInt	6	OMAC PackMLV30 user-defined unit mode 03
LPMLV30_MODE_USER_04	DInt	7	OMAC PackMLV30 user-defined unit mode 04
LPMLV30_MODE_USER_05	DInt	8	OMAC PackMLV30 user-defined unit mode 05
LPMLV30_MODE_USER_06	DInt	9	OMAC PackMLV30 user-defined unit mode 06
LPMLV30_MODE_USER_07	DInt	10	OMAC PackMLV30 user-defined unit mode 07
LPMLV30_MODE_USER_08	DInt	11	OMAC PackMLV30 user-defined unit mode 08
LPMLV30_MODE_USER_09	DInt	12	OMAC PackMLV30 user-defined unit mode 09
LPMLV30_MODE_USER_10	DInt	13	OMAC PackMLV30 user-defined unit mode 10
LPMLV30_MODE_USER_11	DInt	14	OMAC PackMLV30 user-defined unit mode 11
LPMLV30_MODE_USER_12	DInt	15	OMAC PackMLV30 user-defined unit mode 12
LPMLV30_MODE_USER_13	DInt	16	OMAC PackMLV30 user-defined unit mode 13
LPMLV30_MODE_USER_14	DInt	17	OMAC PackMLV30 user-defined unit mode 14
LPMLV30_MODE_USER_15	DInt	18	OMAC PackMLV30 user-defined unit mode 15
LPMLV30_MODE_USER_16	DInt	19	OMAC PackMLV30 user-defined unit mode 16
LPMLV30_MODE_USER_17	DInt	20	OMAC PackMLV30 user-defined unit mode 17
LPMLV30_MODE_USER_18	DInt	21	OMAC PackMLV30 user-defined unit mode 18
LPMLV30_MODE_USER_19	DInt	22	OMAC PackMLV30 user-defined unit mode 19
LPMLV30_MODE_USER_20	DInt	23	OMAC PackMLV30 user-defined unit mode 20
LPMLV30_MODE_USER_21	DInt	24	OMAC PackMLV30 user-defined unit mode 21
LPMLV30_MODE_USER_22	DInt	25	OMAC PackMLV30 user-defined unit mode 22
LPMLV30_MODE_USER_23	DInt	26	OMAC PackMLV30 user-defined unit mode 23
LPMLV30_MODE_USER_24	DInt	27	OMAC PackMLV30 user-defined unit mode 24
LPMLV30_MODE_USER_25	DInt	28	OMAC PackMLV30 user-defined unit mode 25
LPMLV30_MODE_USER_26	DInt	29	OMAC PackMLV30 user-defined unit mode 26
LPMLV30_MODE_USER_27	DInt	30	OMAC PackMLV30 user-defined unit mode 27
LPMLV30_MODE_USER_28	DInt	31	OMAC PackMLV30 user-defined unit mode 28
LPMLV30_STATE_UNDEFINED	DInt	0	OMAC PackMLV30 state Undefined
LPMLV30_STATE_CLEARING	DInt	1	OMAC PackMLV30 state Clearing
LPMLV30_STATE_STOPPED	DInt	2	OMAC PackMLV30 state Stopped
LPMLV30_STATE_STARTING	DInt	3	OMAC PackMLV30 state Starting
LPMLV30_STATE_IDLE	DInt	4	OMAC PackMLV30 state Idle
LPMLV30_STATE_SUSPENDED	DInt	5	OMAC PackMLV30 state Suspended
LPMLV30_STATE_EXECUTE	DInt	6	OMAC PackMLV30 state Execute
LPMLV30_STATE_STOPPING	DInt	7	OMAC PackMLV30 state Stopping
LPMLV30_STATE_ABORTING	DInt	8	OMAC PackMLV30 state Aborting

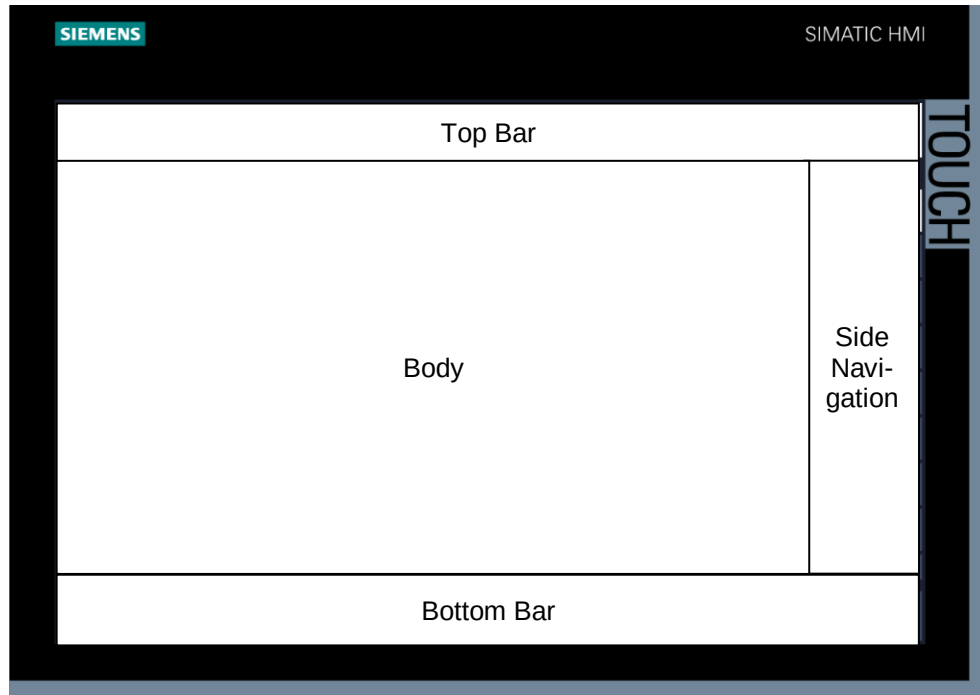
Name	Data type	Default value	Comment
LPMLV30_STATE_ABORTED	DInt	9	OMAC PackMLV30 state Aborted
LPMLV30_STATE_HOLDING	DInt	10	OMAC PackMLV30 state Holding
LPMLV30_STATE_HELD	DInt	11	OMAC PackMLV30 state Held
LPMLV30_STATE_UNHOLDING	DInt	12	OMAC PackMLV30 state Unholding
LPMLV30_STATE_SUSPENDING	DInt	13	OMAC PackMLV30 state Suspending
LPMLV30_STATE_UNSPENDING	DInt	14	OMAC PackMLV30 state Unsuspending
LPMLV30_STATE_RESETTING	DInt	15	OMAC PackMLV30 state Resetting
LPMLV30_STATE_COMPLETING	DInt	16	OMAC PackMLV30 state Completing
LPMLV30_STATE_COMPLETE	DInt	17	OMAC PackMLV30 state Complete
LPMLV30_CMD_UNDEFINED	DInt	0	OMAC PackMLV30 control command Undefined
LPMLV30_CMD_RESET	DInt	1	OMAC PackMLV30 control command Reset
LPMLV30_CMD_START	DInt	2	OMAC PackMLV30 control command Start
LPMLV30_CMD_STOP	DInt	3	OMAC PackMLV30 control command Stop
LPMLV30_CMD_HOLD	DInt	4	OMAC PackMLV30 control command Hold
LPMLV30_CMD_UNHOLD	DInt	5	OMAC PackMLV30 control command Unhold
LPMLV30_CMD_SUSPEND	DInt	6	OMAC PackMLV30 control command Suspend
LPMLV30_CMD_UNSPEND	DInt	7	OMAC PackMLV30 control command Unsuspend
LPMLV30_CMD_ABORT	DInt	8	OMAC PackMLV30 control command Abort
LPMLV30_CMD_CLEAR	DInt	9	OMAC PackMLV30 control command Clear
LPMLV30_CMD_COMPLETE	DInt	10	OMAC PackMLV30 control command Complete
LPMLV30_LANGUAGE_1	Int	0	Texts in 1st language (default English)
LPMLV30_LANGUAGE_2	Int	1	Texts in 2nd language (default German)
LPMLV30_DIAG_BUFFER_UPPER_LIM	Int	15	Diagnostics buffer array upper boundary (0-based)
LPMLV30_LANGUAGES_UPPER_LIM	Int	2	(Number of languages - 1) -> Array[0..LPMLV30_LANGUAGES_UPPER_LIM]
LPMLV30_MODES_UPPER_LIM	Int	4	(Number of unit modes - 1) -> Array[0..LPMLV30_MODES_UPPER_LIM]
LPMLV30_STATES_UPPER_LIM	Int	17	(Number of states - 1) -> Array[0..LPMLV30_STATES_UPPER_LIM] (SHOULD ALWAYS BE 17!)
LPMLV30_MAX_MODES_UPPER_LIM	Int	31	(Maximum number of unit modes - 1) -> Array[0..LPMLV30_MAX_MODES_UPPER_LIM]

3 HMI

3.1 Screen layout

The template has the following screen layout. The top and bottom bars are always visible with the same buttons. The body and side navigation change depending on the current screen.

Figure 3-1 Screen layout



Top Bar

The Top Bar has the following functions:

- OEM logo
- Date and time display
- Feedback on PLC or PLC's connections
- Screen name
- Current Mode/State name
- First Fault
- Warning scroll message

Body

The Body is the main viewable section of the screen. It contains all commands and detailed information of the equipment.

Bottom Bar

The Bottom Bar has the following functions:

- Main navigation
- Start/Stop/Reset pushbuttons
- Back button
- User Logon button

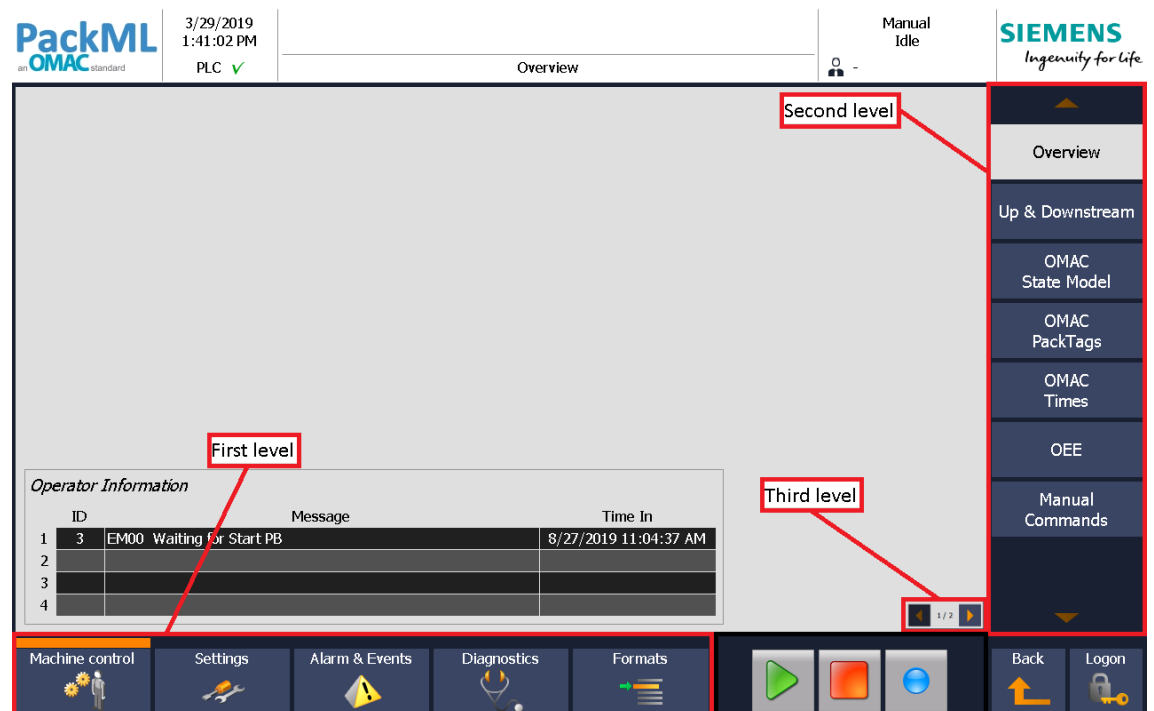
Side Navigation

The Side Navigation is the second level of navigation available. The choices for screens shown depend on the button pushed on the bottom bar. Touch a button on the Side Navigation to move to that screen.

3.2 Navigation

There are three navigation levels available when using the HMI template.

Figure 3-2 Navigation



3.2.1 First level (primary navigation)

This level includes the following preconfigured main topics:

- Machine control
- Settings
- Alarm & Events
- Diagnostics
- Formats

Primary navigation takes place via the Bottom Bar. The user clicks on the appropriate button to change to the first screen of the relevant main topic. The orange marking above the buttons indicates the active main topic.

Machine Control

In this section, the user is able to control the equipment and gain access to upstream and downstream key equipment data. This section contains the primary

screens to control the machine while also providing OMAC PackML and machine OEE / KPI information.

Settings

This section is dedicated to the settings adjustments for each station/equipment module, such as the selected HMI language and User Management / Access.

Alarms & Events

All alarms, warning, and HMI events, including safety feedback, are grouped into this section.

Diagnostics

This section is dedicated to machine diagnostics. It enables diagnostics for each individual station.

Formats

Format management section.

3.2.2 Second level (secondary navigation)

The Side Navigation is an object with up to 24 buttons or tabs for each main topic in order to provide a structured overview of the topics belonging to a main topic. The user clicks on a tab to change the view in the tab. A light grey background indicates an active status.

The tab object is configured in the appropriate template screen of a main topic.

For reasons of space, the number of currently visible tabs is limited to eight. However, to allow you to select from 24 tabs, they are each divided over three levels with eight tabs per level. If more than 8 tabs are defined, a button is automatically displayed above and below the tab selections. This enables the user to switch between the different levels without having to change the screen.

3.2.3 Third level (optional)

If a Side Navigation topic has multiple sub-topics, or if more than 24 screens are needed in a single main topic, a third level of navigation can be used. This makes it possible to switch between a maximum of 40 sub-screens.

With this navigation, the user remains in the same main topic and in the same tab. The user only navigates between screens which belong to one topic.

3.3 Numbering and naming screens

A screen name and screen number must be defined in order to be able to open the previous screen and mark the primary and secondary navigation in which the user is currently located. It is **important** that newly inserted screens are numbered according to this method.

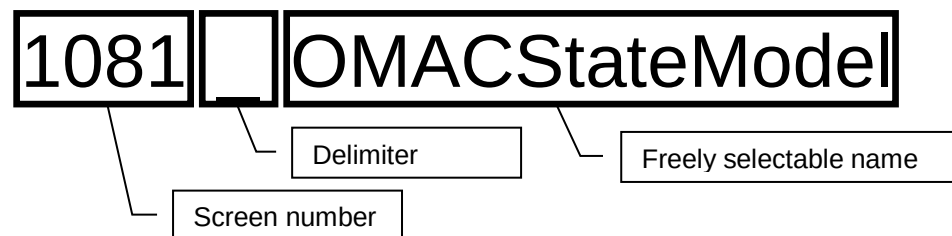
Screen number

The screen number must always be a four-digit number. The first digit defines the main topic and the next three digits define the current tab object and the sub-screen contained there. Each tab, for each main topic, can have 40 sub-screens

No.	Main topic	Tab	Number range
1	Machine Control	1	001 – 040
2	Settings	2	041 – 080
3	Alarms & Events	3	081 – 120
6	Diagnostics	4	121 – 160
7	Formats	5	161 – 200
		6	201 – 240
		7	241 – 280
		8	281 – 320
		9	321 – 360
		10	361 – 400
		11	401 – 440
		12	441 – 480
		13	481 – 520
		14	521 – 560
		15	561 – 600
		16	601 – 640
		17	641 – 680
		18	681 – 720
		19	721 – 760
		20	761 – 800
		21	801 – 840
		22	841 – 880
		23	881 – 920
		24	921 – 960

Screen name

In addition to correct numbering, the correct structure of the screen name must be observed. This **must always** begin with the defined screen number, followed by the underscore "_" as delimiter and the freely selectable screen name.



The screen number and name are entered in the screen properties when the screen is opened.

Figure 3-3 Numbering and naming screen – example #1 screen 1081_OMACStateModel

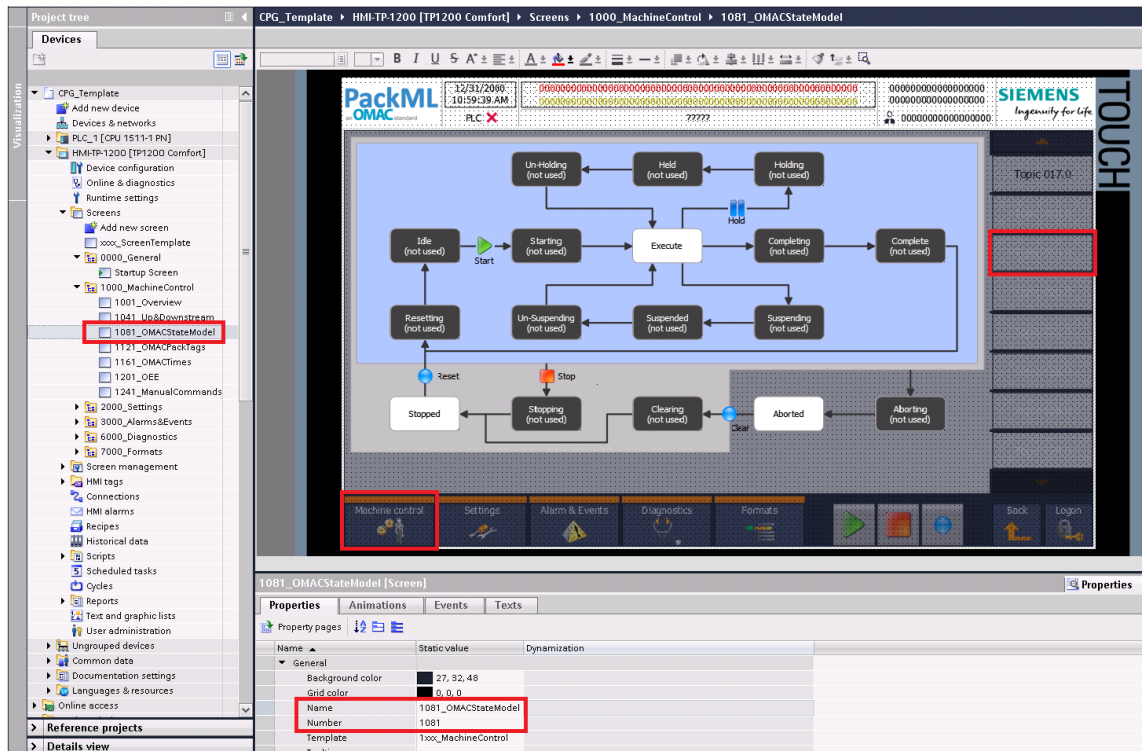
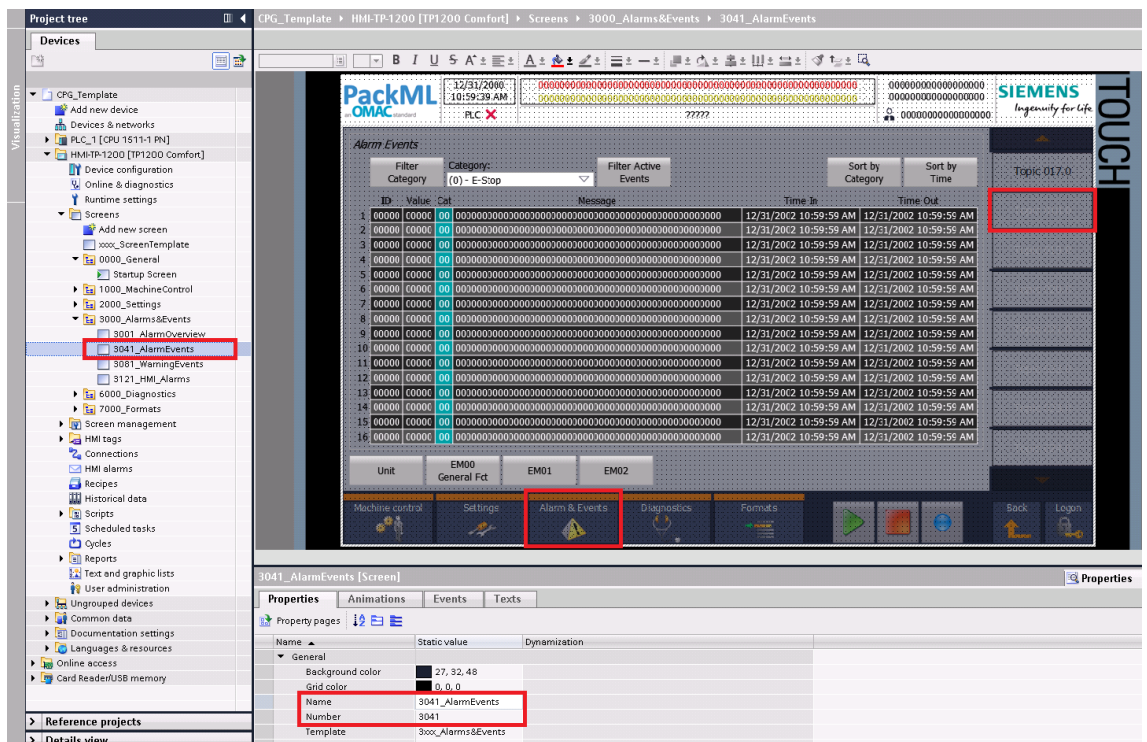


Figure 3-4 Numbering and naming screen – example #2 screen 3041_AlarmEvents

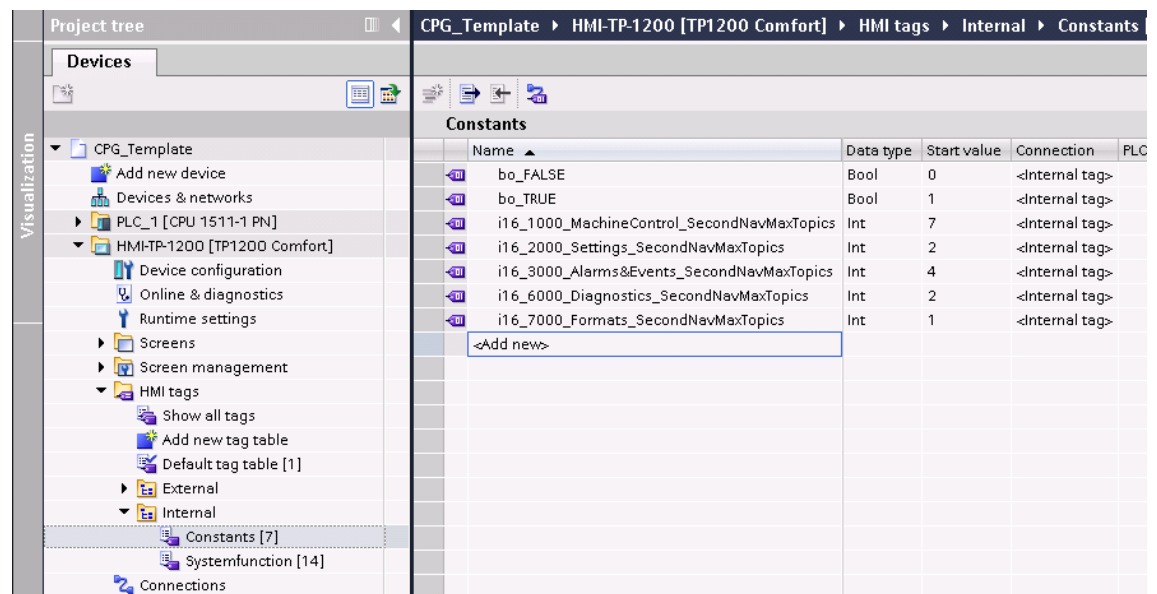


3.4 Adjusting the Navigation

3.4.1 Adjusting the number of second level topic buttons

The number of displayed second level tabs for each main topic (first level) button is determined respective HMI Constant. By setting the start value with the tags in the tag list **HMI Tags > Internal > Constants**, the number of available tabs in the individual main topics can be defined. Since the maximum number of tabs available for each second level navigation is 24, the maximum value for the constant start value is also 24.

Figure 3-5 Max topics in second level (secondary navigation)



Name	Data type	Start value	Connection	PLC
bo_FALSE	Bool	0	<Internal tag>	
bo_TRUE	Bool	1	<Internal tag>	
i16_1000_MachineControl_SecondNavMaxTopics	Int	7	<Internal tag>	
i16_2000_Settings_SecondNavMaxTopics	Int	2	<Internal tag>	
i16_3000_Alarms&Events_SecondNavMaxTopics	Int	4	<Internal tag>	
i16_6000_Diagnostics_SecondNavMaxTopics	Int	2	<Internal tag>	
i16_7000_Formats_SecondNavMaxTopics	Int	1	<Internal tag>	
<Add new>				

The default second level tabs configured for each main topic are:

- Machine Control (i16_1000_MachineControl_SecondNavMaxTopics) = 7
- Settings (i16_2000_Settings_SecondNavMaxTopics) = 2
- Alarms & Events (i16_3000_Alarm&Events_SecondNavMaxTopics) = 4
- Diagnostics (i16_6000_Diagnostics_SecondNavMaxTopics) = 2
- Formats (i16_7000_Formats_SecondNavMaxTopics) = 1

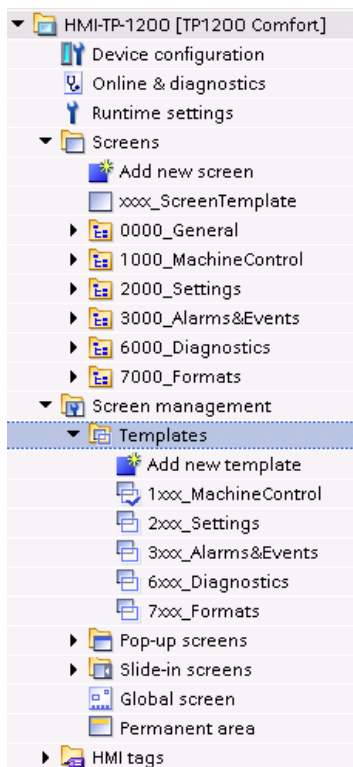
3.4.2 Screen Templates

Once the first and second level navigation has been defined, the corresponding second level buttons for each main topic must be properly connected to the appropriate HMI screen using the standardized HMI screen numbering and naming convention.

The screen template can be found in **Screen Management > Templates**.

Each of the templates consists of the areas Top Bar (see **Screen Management > Templates->Permanent area**), Body Bar and Bottom Bar.

Figure 3-6 Screen Templates



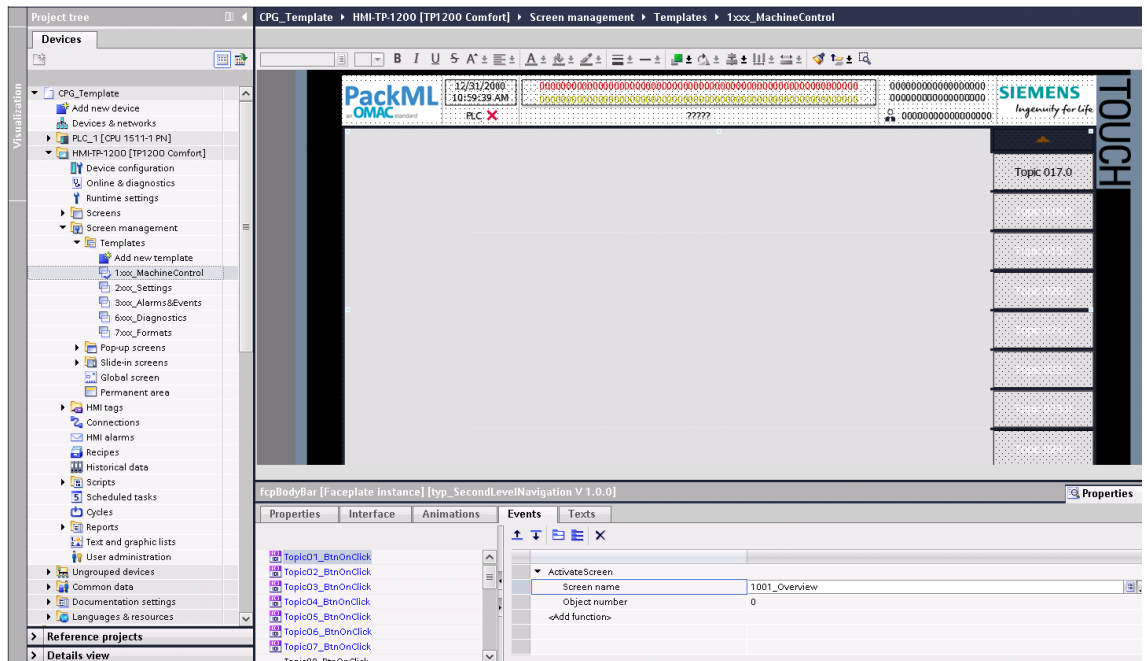
The Templates folder contains a unique screen template for each of the main topic items: Machine Control, Settings, Alarms & Events, Diagnostics, and Formats.

3.4.3 Adjusting the second level topic button screen activation

For each main topic template, the second level (secondary navigation) button must be configured to activate the appropriate screen as determined by the screen numbering and naming convention found in section [3.3](#)

This is accomplished by editing the **TopicXY_BtnOnClick** items found in the **Properties > Events** section for the items in **Screen Management > Templates**. Every visible button that determines the start value of the second level navigation constants, must be connected to the corresponding HMI screen using the **ActiveScreen** command.

Figure 3-7 HMI Template – Second level navigation screen configuration

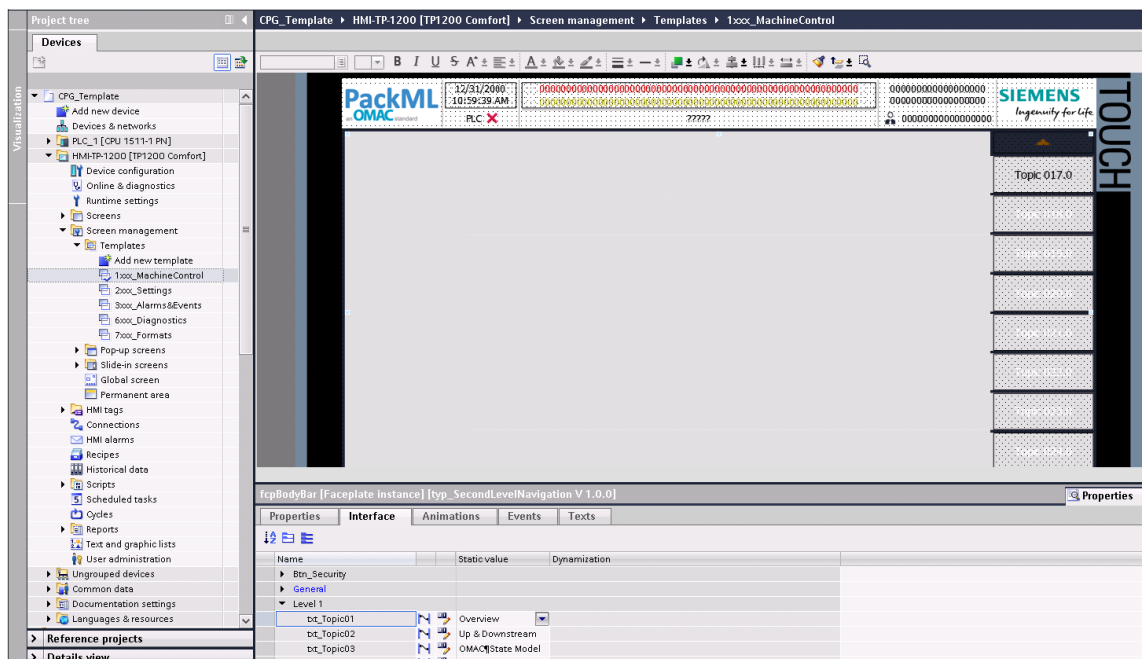


3.4.4 Adjusting the second level topic button text

The button text for each second level screen navigation button must be defined for each configured language. This is accomplished by editing the „text“ tab of the corresponding machine topic (first level) template. The screen templates can be found in **Screen Management > Templates**.

The actual text displayed during runtime for the second level tabs is determined by the **Properties > Interface** area for each main topic template.

Figure 3-8 Second level tab text value configuration



Within the **Properties > Interface** area there are 3 Level sub-areas – Level 1 for tabs 1-8, Level 2 for tabs 9-16, and Level 3 for tabs 17-24. Edit the *Static value* text for each configured tab and each HMI Runtime language.

Figure 3-9 Second level tab text value configuration for all 24 tabs

fcpBodyBar [Faceplate instance] [typ_SecondLevelNavigation V 1.0.0]				
Properties	Interface	Animations	Events	Texts
Name			Static value	Dynamization
▶ Btn_Security				
▶ General				
▼ Level 1				
txt_Topic01			Overview	
txt_Topic02			Up & Downstream	
txt_Topic03			OMAC State Model	
txt_Topic04			OMAC PackTags	
txt_Topic05			OMAC Times	
txt_Topic06			OEE	
txt_Topic07			Manual Commands	
txt_Topic08			Topic 008.0	
▼ Level 2				
txt_Topic09			Topic 009.0	
txt_Topic10			Topic 010.0	
txt_Topic11			Topic 011.0	
txt_Topic12			Topic 012.0	
txt_Topic13			Topic 013.0	
txt_Topic14			Topic 014.0	
txt_Topic15			Topic 015.0	
txt_Topic16			Topic 016.0	
▼ Level 3				
txt_Topic17			Topic 017.0	
txt_Topic18			Topic 018.0	
txt_Topic19			Topic 019.0	
txt_Topic20			Topic 020.0	
txt_Topic21			Topic 021.0	
txt_Topic22			Topic 022.0	
txt_Topic23			Topic 023.0	
txt_Topic24			Topic 024.0	

When the HMI Runtime is configured for more than 1 language, then the text field for each language needs to be entered. This is managed by editing the **Texts** data for each configured Runtime language.

Figure 3-10 – Mult-lingual text configuration

fcpBodyBar [Faceplate instance] [typ_SecondLevelNavigation V 1.0.0]			
Properties	Interface	Animations	Events
Texts			
German (Germany)	English (United States)	Reference	
Manuelle Kommandos	Manual Commands	fcpBodyBar/G	
OEE	OEE	fcpBodyBar/G	
OMAC PackTags	OMAC PackTags	fcpBodyBar/G	
OMAC Statusmodell	OMAC State Model	fcpBodyBar/G	
OMAC Zeiten	OMAC Times	fcpBodyBar/G	
Übersicht	Overview	fcpBodyBar/G	
Up & Downstream	Up & Downstream	fcpBodyBar/G	

3.5 Functions/screens and their integration

3.5.1 HMI Tags

The tags configured in the template are divided into external and internal tags. All the tags which are connected to a controller can be found under the external tags. Internal tags, however, are tags which are only used in the panel.

External tags

With external tags, please note that the addressing may have to be adapted to the customer project.

Table 3-1 HMI External Tag Tables

Tag table	Description
PackTags	OMAC PackTags
PackTags_Times	Pack Tags - Mode/State Times Data
EM_CM_Module Active	Module Active status
Event_Alarm	Alarm Data
Event_Alarm_SortFilter	Alarm Filter Data
Event_AlarmHistory	Alarm History Data
Event_Status	Status is in PLC code, but not HMI
Event_Warning	Warning Data
Event_Warning_SortFilter	Warning Filter Data

Internal tags

Internal tags are only valid in the panel and are required primarily for the implementation of different system functions and to display the OMAC Mode-State Manager.

Table 3-2 HMI Internal Tag Tables

Tag table	Description
Constants	Project constants
System Functions	Tags in conjunction with system functions

3.5.2 User Administration

Different users, user groups and authorization levels are already provided in the template. If the screen objects are configured accordingly, this ensures that certain functions can only be executed by authorized and trained users.

In order to provide important settings and areas with additional protection, the position of the key-operated switch on the panel is also requested in addition to the password when a user wants to logs on. It must be inserted and in the appropriate position.

If a new user is logged on, the array for opening the previous screen is initialized. This prevents screens for which the current user is not authorized from being opened via this function.

If incorrect logon data is used, the logon process is canceled, and the user can try to log on again.

These functions are executed by the script "sub_UserChanged". This script is called up by the scheduler when the user changes.

User groups

Various user groups are already created and assigned a number in the user administration of the current template. If required, additional user groups can be inserted.

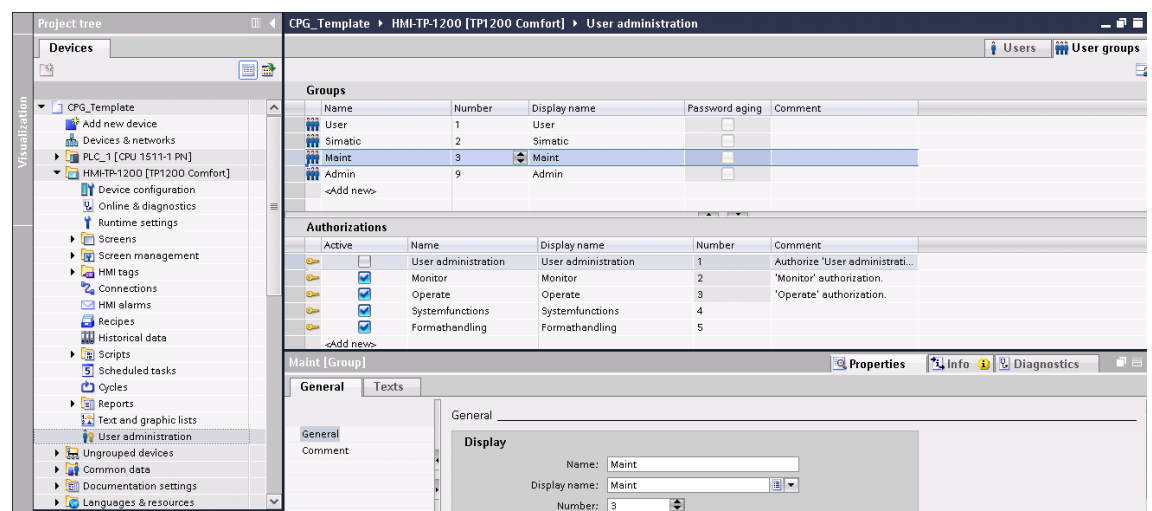
Users

Different users with different passwords can be created in the panel. Each user is assigned to a user group and thus inherits its authorizations.

Authorizations

One or more authorization levels can be defined for a user group and then selected as user authorization for screen objects.

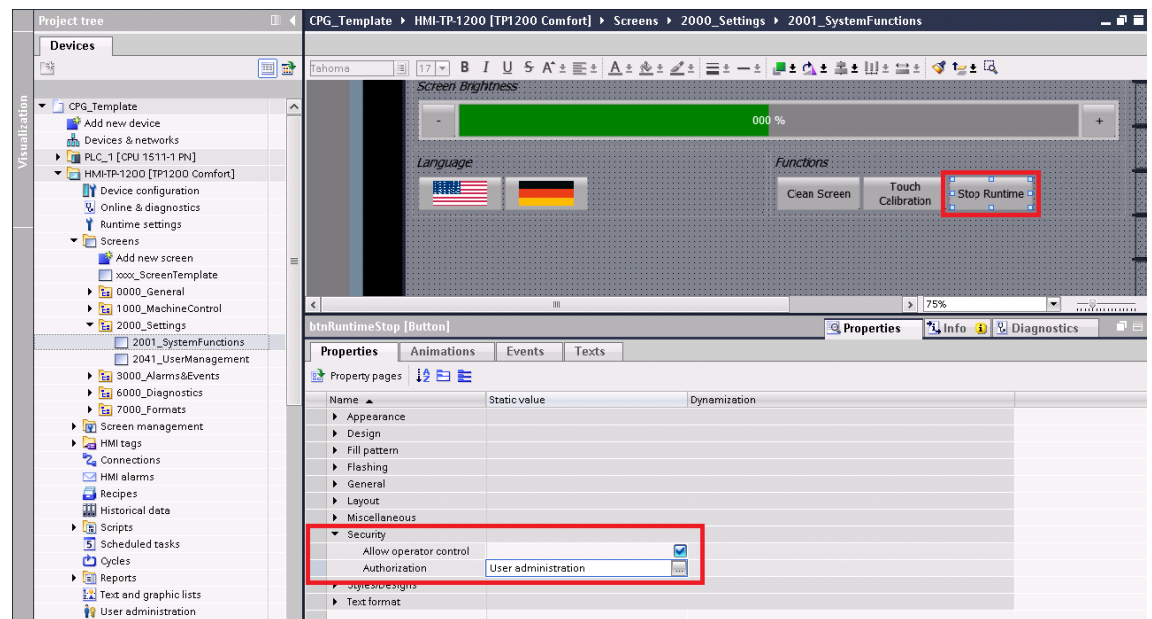
Figure 3-11 Authorizations



Using authorizations

An appropriate authorization level can be configured for an operable screen element in the properties if it is required for carrying out the action. A check is thus carried out in Runtime as to whether the logged-on user also has authorization for this.

Figure 3-12 Using authorization level



3.5.3 Scripts

The following table lists and describes all the configured scripts.

Table 3-3

Script name	Description
sub_OpenPrevScreen	Open previous screen. You can scroll backwards in accordance with the array size with the help of a screen number buffer.
sub_RuntimeStart	Actions to be performed at Runtime start
sub_ScreenChanged	Filter screen number from the name of the current screen and save it in the screen number buffer for opening the previous screen.
sub_SetScreenBrightness	Set screen brightness of panel
sub_UserChanged	Determine current user name and group number. A '0' is then entered in the screen memory for the return function at the current pointer position, so that the new user cannot jump to the screen last opened by the previous user.

3.5.4 Text Lists

Some text lists are available in the template project. Most of these do not need to be changed, however.

Table 3-4

Text list	Description
CPG_CategoryList	Alarm Event – Category List
Local_Remote_Control	Local or remote control mode 0: local control only 1: remote control enabled
LPMLV30_CntrlCmd	Commands for state change in accordance with OMAC
LPMLV30_Message	OMAC messages for diagnostics
LPMLV30_States	OMAC states for display
LPMLV30_UnitModes	OMAC modes for display
MaterialInterlock_BottleSupply_NotLow	Material Interlock - BottleSupply – NotLow 0: Low; 1: Not low
MaterialInterlock_BottleSupply_Ready	Material Interlock - BottleSupply – Ready 0: Not Ready; 1: Ready
MaterialInterlock_LabelSupply_NotLow	Material Interlock - LabelSupply – NotLow 0: Low; 1: Not low
MaterialInterlock_LabelSupply_Ready	Material Interlock - LabelSupply – Ready 0: Not Ready; 1: Ready
ScreenName	The screen Name is related to the Screen number

3.5.5 Graphic Lists

The following table lists and describes all the configured graphic lists.

Table 3-5

Graphic list	Description
MaterialInterlocksReadyOrNotLow	Graphic to show whether the material supply is ready or not low 0: Not Ready or Low; 1: Ready or Not Low
PLCConnectionStatus	Graphic to show whether the connection to the PLC has been established 0: Not OK; 1: OK

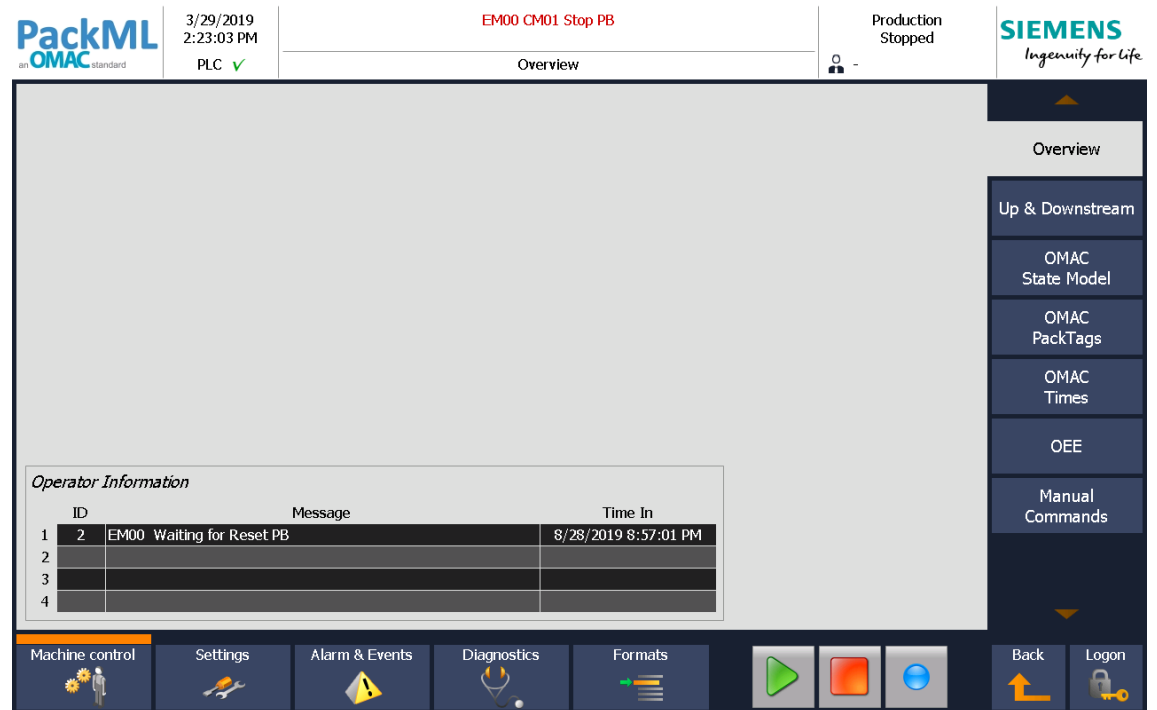
3.6 Preconfigured HMI Screens

3.6.1 Machine Control

Overview

This screen provides an overview of the machine status. By default, it contains only the „Operator Information“ window, which shows the current statuses.

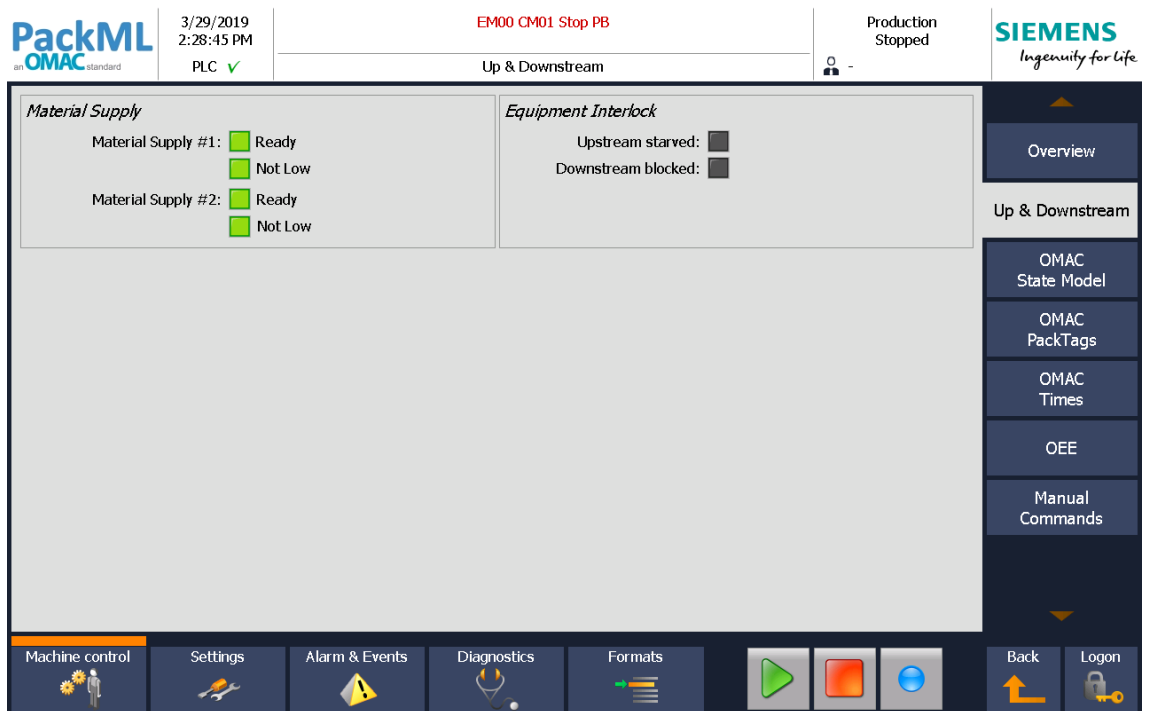
Figure 3-13 Machine Control - Overview



Up & Downstream

This screen summarizes information of machines up and downstream from the UN. It displays if input materials are ready from upstream, and if there are blocked or starved conditions. By default, the *PackTags.Status.materialInterlocks* control the material supply LEDs. These are unused in the template. The starved & blocked bits are controlled by the *PackTags.Status.EquipmentInterlock* bits, which ultimately are set/reset by the presence of a category 8 or 9 alarm in the UN.

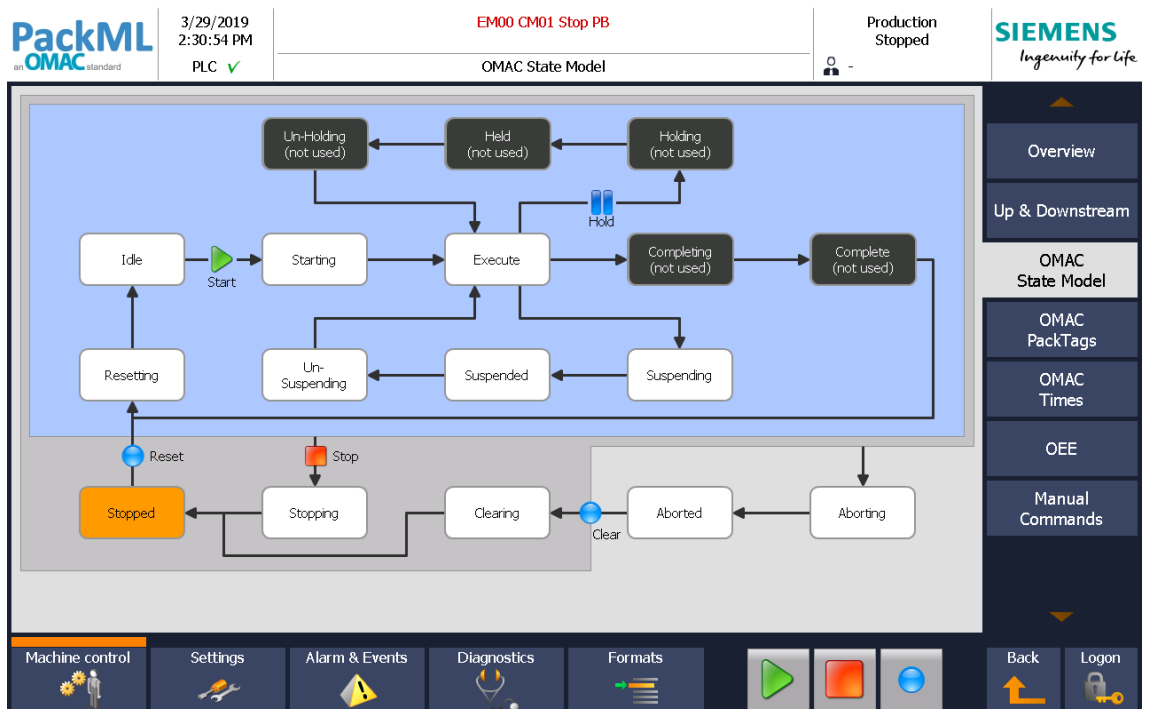
Figure 3-14 Machine Control - Up & Downstream Information



OMAC State Model

This screen displays the current PackML state. It also shows the commands which will transition the states (e.g. to move from IDLE -> STARTING, press the Start button).

Figure 3-15 Machine Control - OMAC PackML State Model



OMAC PackTags

This screen summarizes the information in the PackTags global DB. It is for display only, no fields are editable.

Figure 3-16 Machine Control - OMAC PackTags

PackML on OMAC standard

3/29/2019 2:32:12 PM
PLC ✓

EM00 CM01 Stop PB

Production Stopped

SIEMENS
Ingenuity for Life

OMAC PackTags

Status

Unit mode current: Production

State current: Stopped

Current machine speed: 110.0 pcs/min

Machine speed: 120.0 pcs/min

Equip. interlock blocked: 0

Equip. interlock starved: 0

Command

Machine speed: 120.0 pcs/min

Unit mode: Invalid

Mode change request: 0

State control command: Undefined

State change request: 0

Admin

	Counts	Acc. Count
Consumed #1	+397	+4180 count
Processed #1	+412	+5989 count
Defective #1	+11	+278 count

Machine design speed: 125.0 pcs/min

Stop reason ID: 33

Stop reason message: EM00 CM01 Stop PB

Active alarms: 1

Active warnings: 0

Machine control | Settings | Alarm & Events | Diagnostics | Formats | [Play] [Stop] [Reset]

Back | Logon

PackML Times

This screen summarizes how long the machine has stayed in each state/mode. This information is stored in the *PackTags.Admin* structure, which is ultimately controlled by the LPMLV30 block *LPMLV30_UnitModeStateTimes*, called in *UN_02_PackML*. This screen is read only, except for a Reset signal.

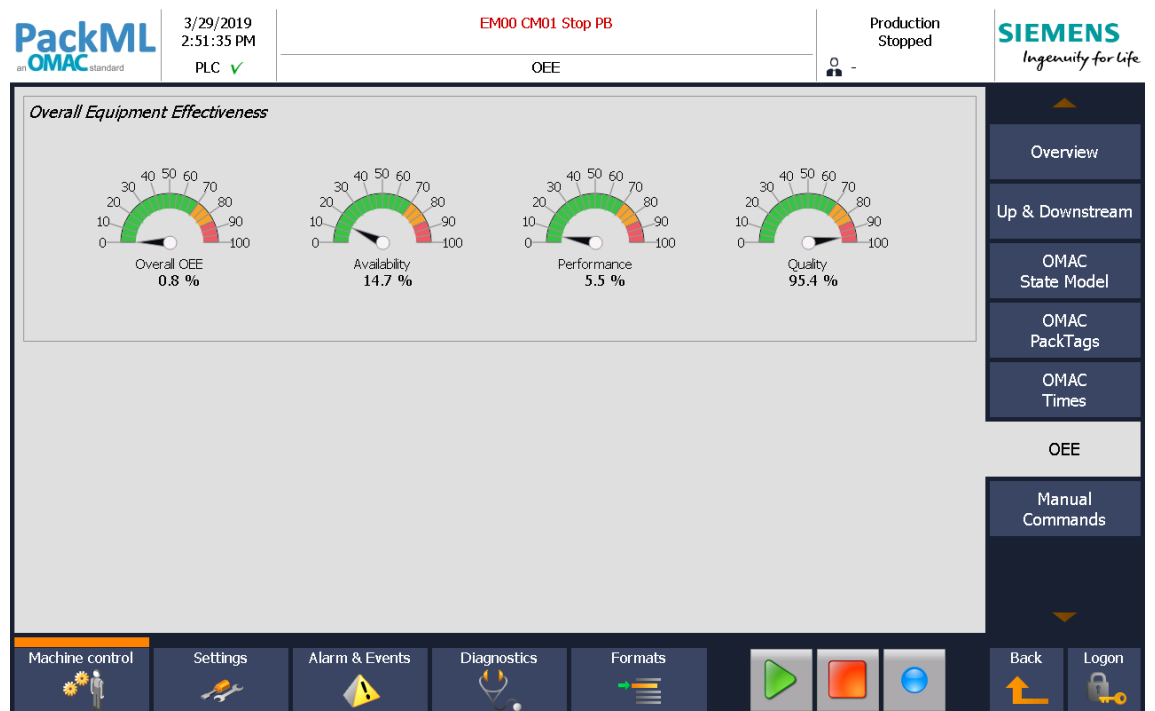
Figure 3-17 Machine Control - OMAC Mode and State Times



OEE

This screen displays the built in OEE calculations. These values are taken from the *PackML.OEE* structure, which is calculated by the block *CPG_OEE*, called in *UN_06_MiscFunctions*. This screen is read only, no values are editable.

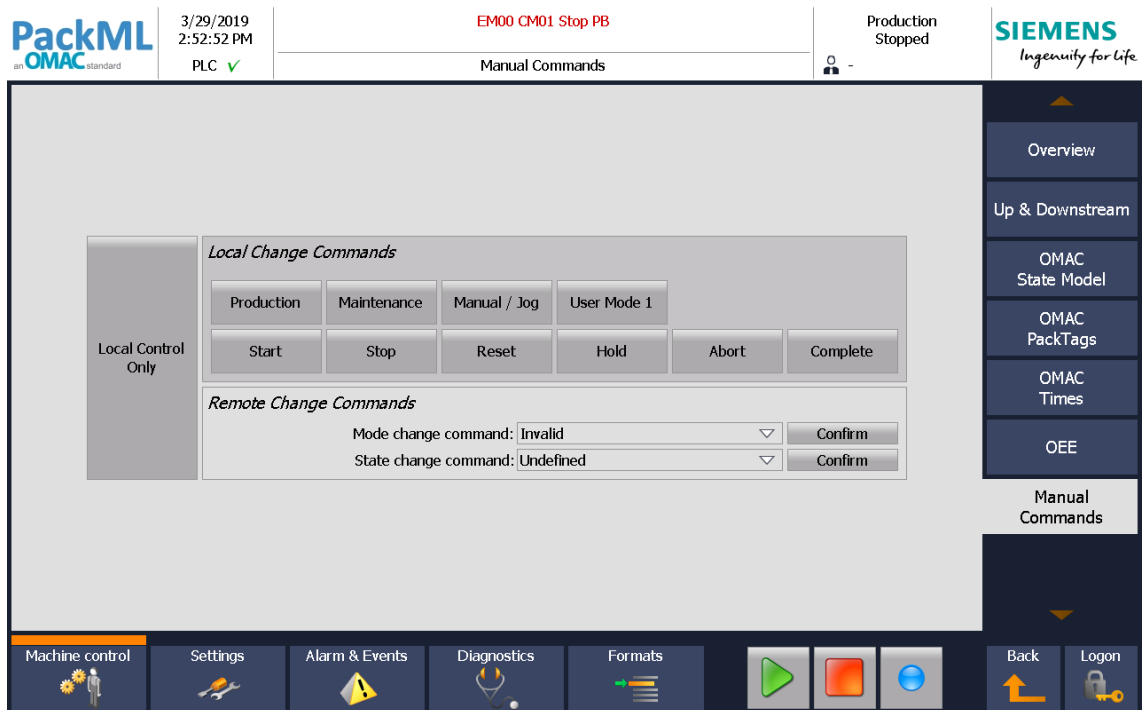
Figure 3-18 Machine Control - OEE Values



Manual Commands

In this screen the OMAC Mode/State Manager can be controlled manually, e.g. switching from Manual mode to Production mode in Stopped state.

Figure 3-19 Machine Control - Manual Commands



3.6.2 Settings

System Functions

This screen contains some settings for the HMI itself. It is possible to set the time, adjust the brightness, change the language, or execute some system functions.

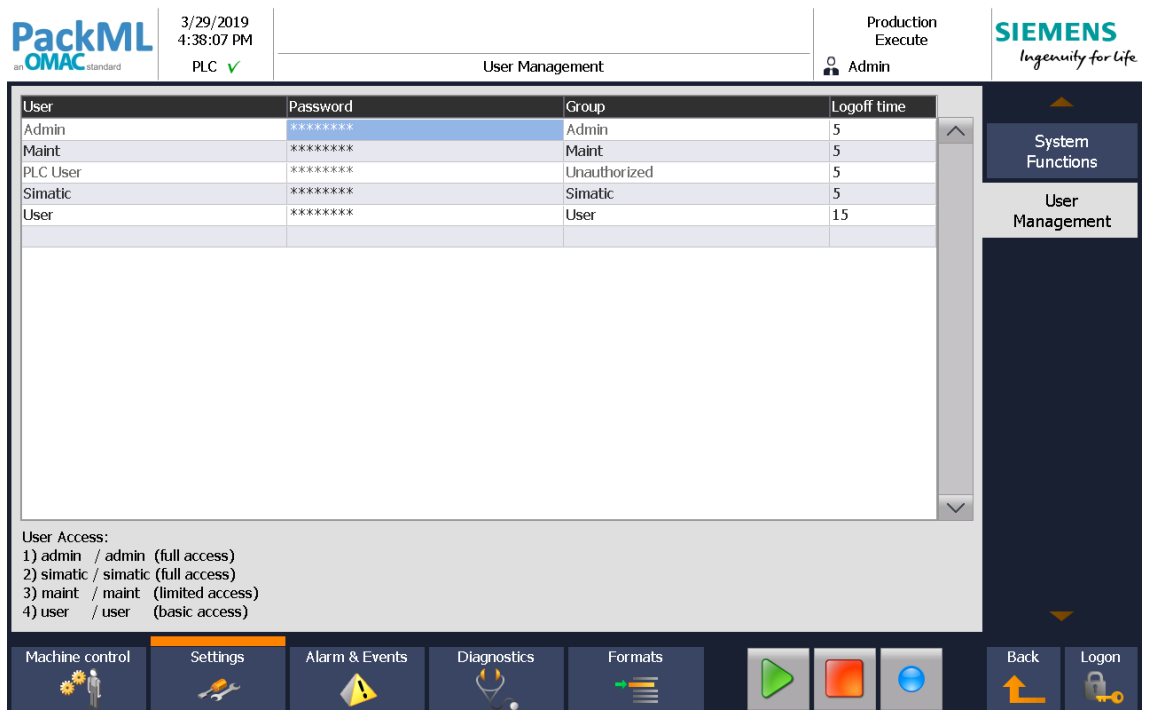
Figure 3-20 Settings - System Functions



User Management

This screen contains provides functionality to manage users and user rights.

Figure 3-21 Settings – User Management



3.6.3 Alarms and Events

Event Overview

This screen overviews the alarms and warnings. The first window shows the active alarms. The second shows previous “first faults”. The third shows the active warnings. This screen is read only.

Figure 3-22 Alarms & Events - Event Overview

Active Alarms

ID	Value	Cat	Message	Time In	Time Out	
1	33	2	2	EM00 CM01 Stop PB	8/28/2019 10:17:52 PM	8/28/2019 10:17:52 PM
2						
3						
4						
5						
6						

Stop Reason Alarm History

ID	Value	Cat	Message	Time In	Time Out	
1	33	2	2	EM00 CM01 Stop PB	8/28/2019 10:17:52 PM	1/1/1990 12:00:00 AM
2	33	2	2	EM00 CM01 Stop PB	8/28/2019 9:03:15 PM	8/28/2019 10:17:50 PM
3	33	2	2	EM00 CM01 Stop PB	8/28/2019 8:56:59 PM	8/28/2019 9:01:17 PM
4	33	2	2	EM00 CM01 Stop PB	7/24/2019 11:15:00 AM	8/28/2019 1:48:26 PM

Active Warnings

ID	Cat	Message	Time In	Time Out
1				
2				
3				
4				

Alarm Events

This screen provides a much more detailed view of the alarms. More alarms are displayed, and it is possible to sort and filter them. Filter by category, remove inactive (acknowledged) events, sort by category or time of occurrence, and filter out alarms by the EM in which they occurred.

Figure 3-23 Alarm & Events - Alarm Events

Alarm Events

ID	Value	Cat	Message	Time In	Time Out
1	33	2	EM00 CM01 Stop PB	8/28/2019 10:17:52 PM	8/28/2019 10:17:52 PM
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					
16					

Unit: EM00 General Fct, EM01, EM02

Machine control | Settings | **Alarm & Events** | Diagnostics | Formats

Back | Logon

Warning Events

This screen provides a much more detailed view of the warnings. More warnings are displayed, and it is possible to sort and filter them. Filter by category, remove inactive (acknowledged) events, sort by category or time of occurrence, and filter out warnings by the EM in which they occurred.

Figure 3-24 Alarm & Events - Warning Events

Warning Events

ID	Cat	Message	Time In	Time Out
1	2	EM00 CM01 Stop PB	8/28/2019 10:17:52 PM	8/28/2019 10:17:52 PM
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				

Unit: EM00 General Fct, EM01, EM02

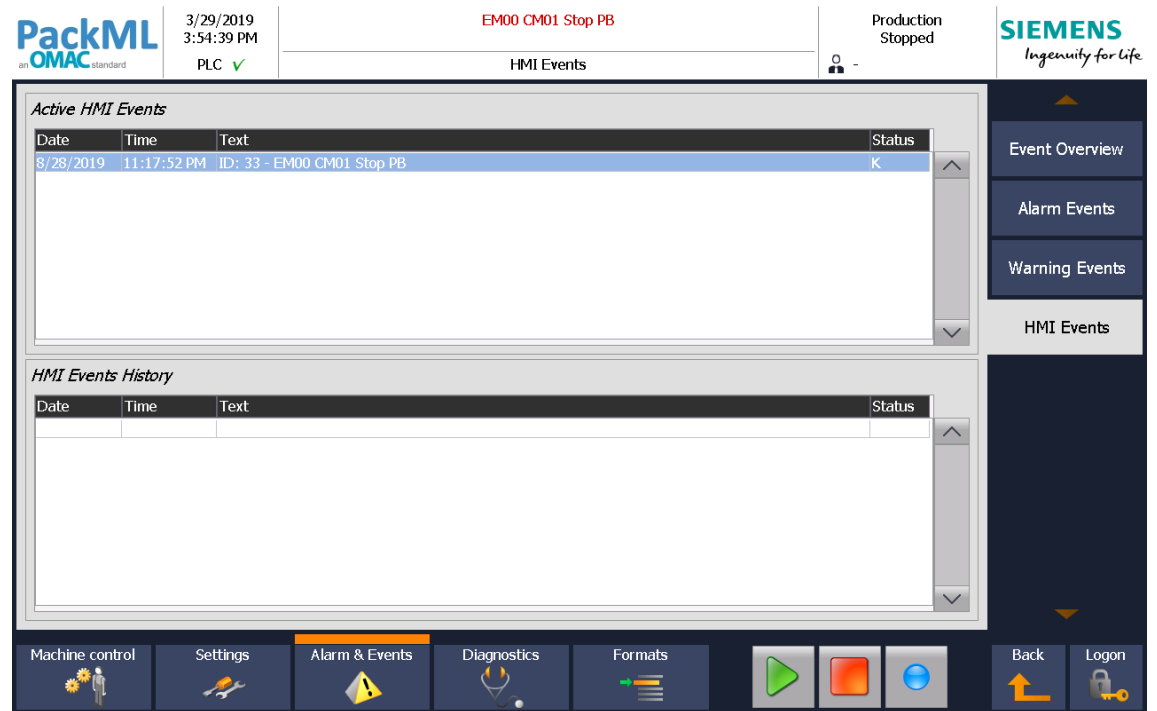
Machine control | Settings | **Alarm & Events** | Diagnostics | Formats

Back | Logon

HMI Events

This screen provides a much more detailed view of HMI events. Any events generated by the HMI are displayed here, as well as a history of previous events.

Figure 3-25 Alarm & Events - HMI Events

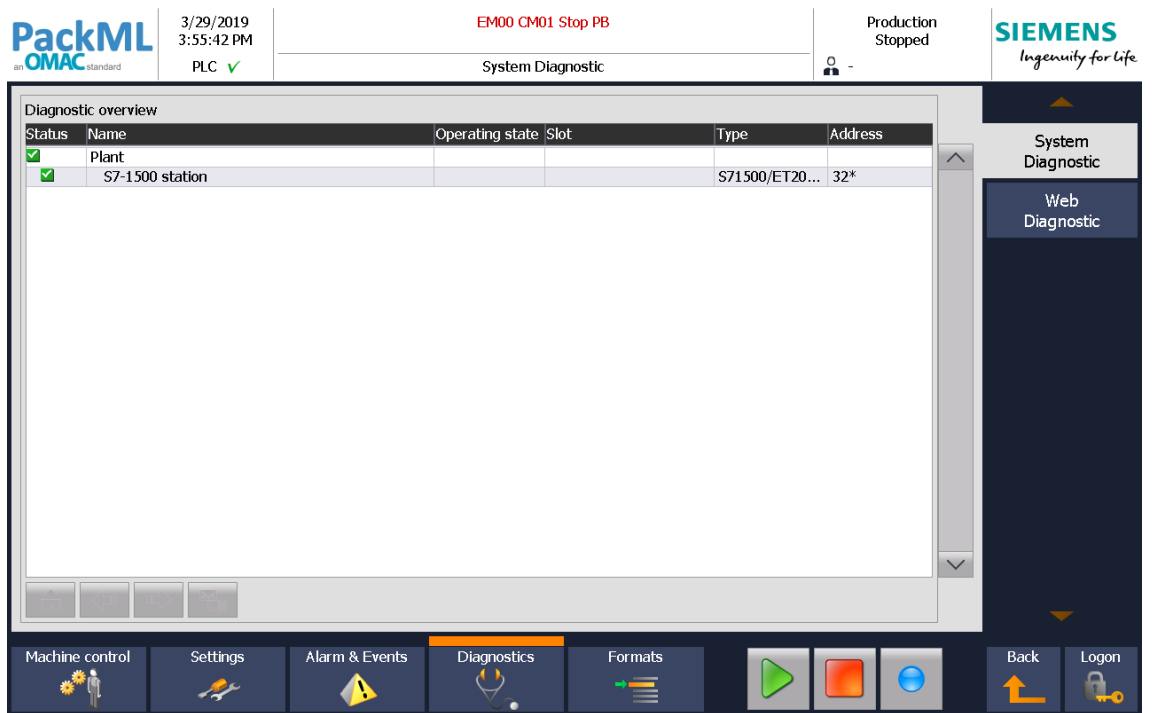


3.6.4 Diagnostics

System Diagnostic

This built-in system diagnostics are displayed here. This information is also available in the diagnostics of the PLC but it is displayed automatically on this screen. Information about the status of devices is shown.

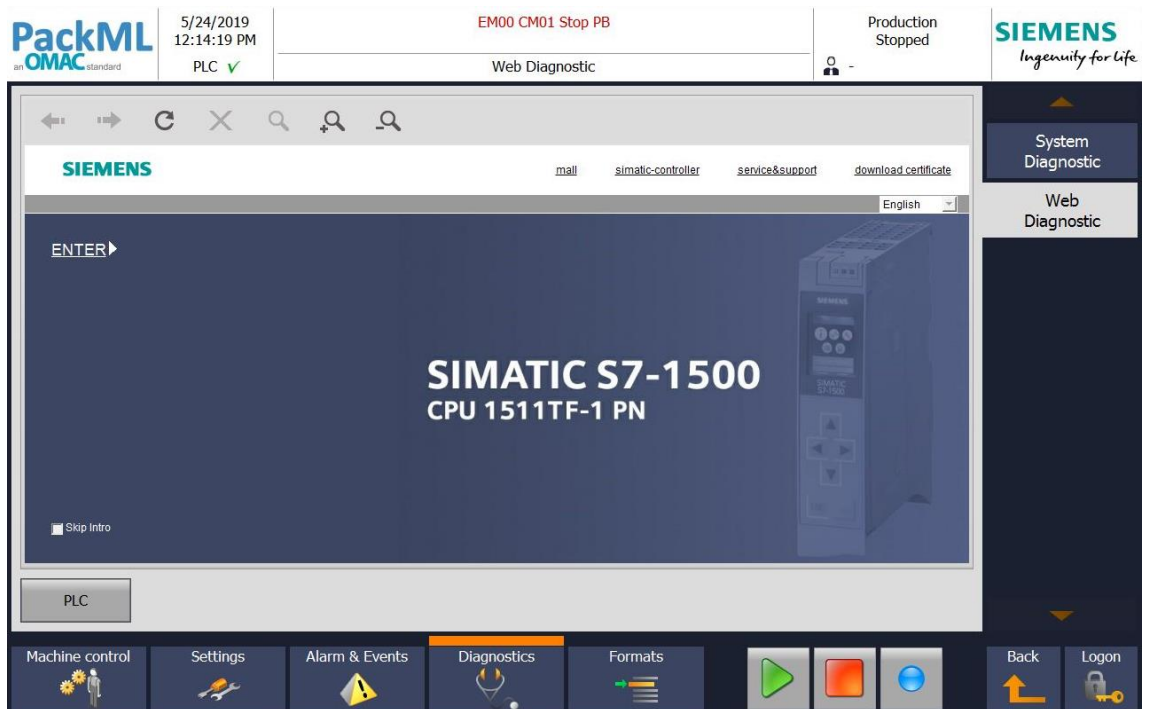
Figure 3-26 – Diagnostics - Integrated PLC System Diagnostics



Web Diagnostic

The web page is connected to the web browser of the project PLC. It is possible to access the webpage here to view diagnostic information.

Figure 3-27 Diagnostics - S7-1500's Web Diagnostics

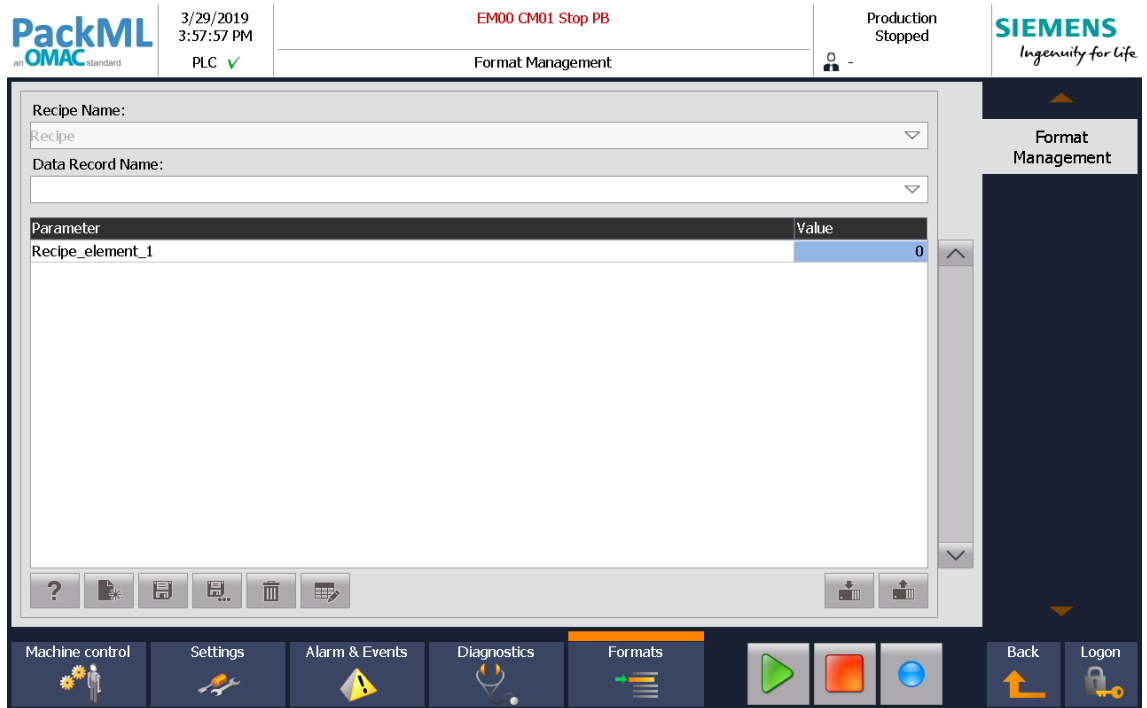


3.6.5 Formats

System Diagnostic

In this screen the recipe handling can be managed.

Figure 3-28 Formats – Format Management



3.7 Multi-language

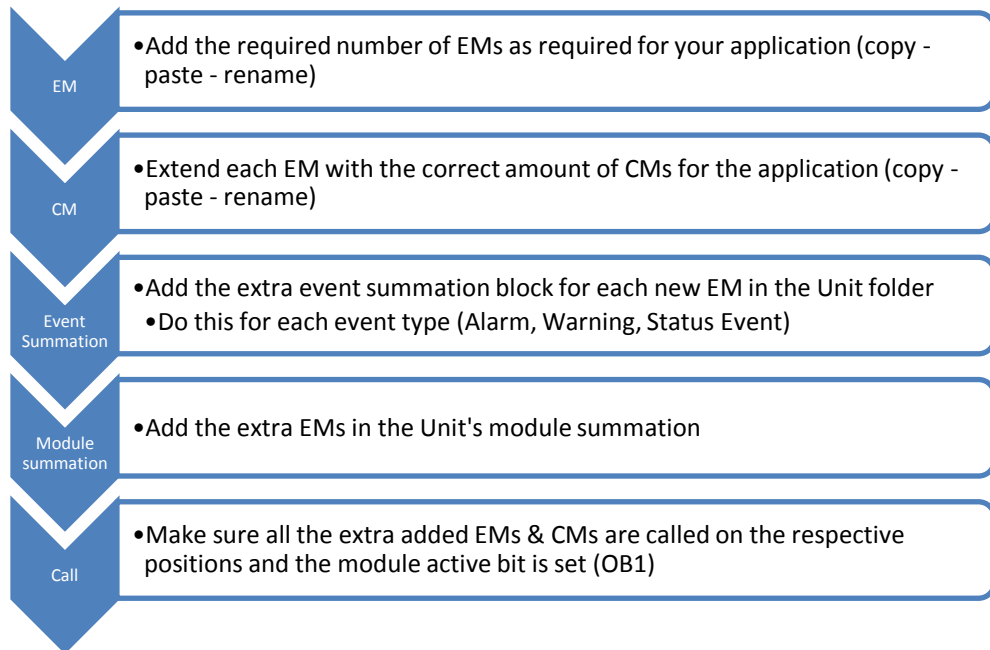
The CPG Template is delivered with texts in the languages English (US) and German (Germany).

The selected HMI language has to be aligned with the texts in the configuration data blocks AlarmCfg, MessagesPrefix, PackMLCf, StatusCfg and WarningCfg.

4 Extending the template

The CPG Template is pre-programmed with 3 Equipment Modules. Each Equipment Module has 5 Control Modules. If fewer modules are needed, it is possible to “deactivate” the unused EMs or CMs. To do this, set the bits for those modules to TRUE in the *PackMLCfg.disable* structure in the PackMLCfg global data block. For more details, refer to section [2.2.5](#).

In some applications, it is necessary to add control or equipment modules. That is the subject of this chapter.



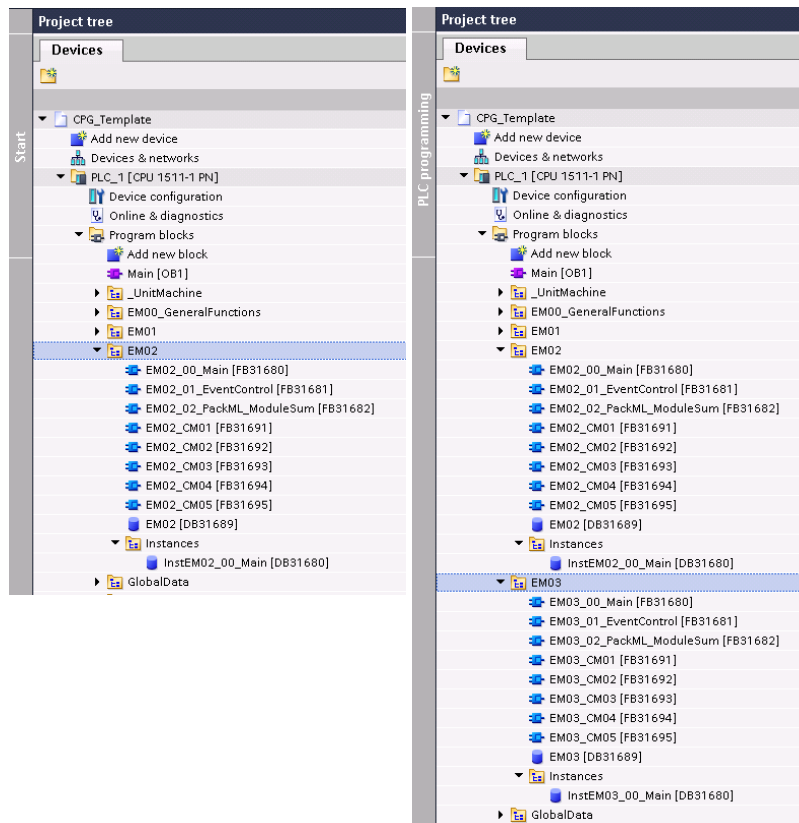
4.1 Adding a new Equipment Module

NOTE As an example, the figures show a fourth EM being added after EM02.

NOTE The new EM is created by copy/paste of an existing EM. It is highly advisable that the copied EM is blank. If no blank EM exists in the project, re-download the CPG Template from SIOS and import EM02 from the blank template.

First, in the project tree, duplicate an existing equipment module. Click on the folder and copy the entire structure and paste it to the *Program blocks* folder. Rename all blocks with the new EM number and delete the appended “_1” on all of them. Be sure to also rename the instance data block in the *Instances* folder.

Figure 4-1 EM folder duplication



Open Main (OB1). This is the block where all equipment modules are called in the program. Navigate to the network of the last EM call and duplicate it. There are four pieces in this network that must be updated with the new EM number:

1. The network comments
2. The single instance call of the EM
3. The *disable* bit for the EM
4. The *moduleActive* bit of the EM

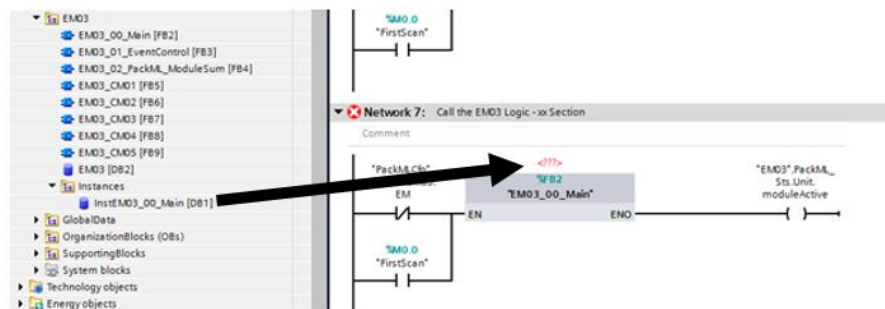
Figure 4-2 Copy the call of an EM inside OB1



Rewrite the network comment to anything desired.

Delete the call of the copied EM and replace it with a call of the new block. When prompted to add instance data, click “cancel”. Then drag the instance DB from the *Instances* folder in the EM to the <??> in the programming window.

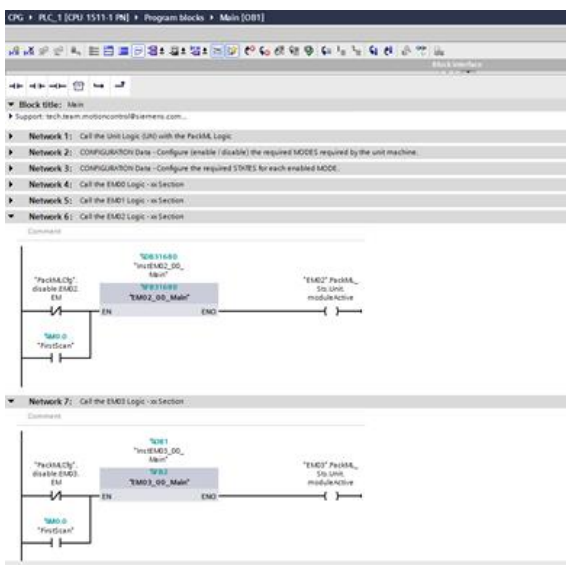
Figure 4-3 Use the already created instance data as the instance data of the new EM



Change the equipment module in the name of *disable* bit from the old EM to the new one – in this example EM02 is replaced with EM03 in the copied network. It is possible this bit will not exist, depending on how many EMs are present in the project already.

Repeat this replacement for the *moduleActive* bit (this bit should always exist in the renamed EM data block).

Figure 4-4 Renamed call of the new EM



Ensure that the new EM has a set of message prefixes. Open the config DB *MessagePrefix*. Duplicate an existing EM's structure, and rename it for the current EM.

Figure 4-5 Message prefixes config DB

CPG_Template > PLC_1 [CPU 1511-1 PN] > Program blocks > GlobalData > MessagePrefix [DB31604]									
MessagePrefix									
	Name	Data type	Start value	Retain	Accessible...	Write...	Visible in...	Setpoint	Supervisi...
1	Static								
2	UN	Struct							MessagePrefix for Unit
3	EM00	Struct							MessagePrefix for EquipmentModuleEM00
4	EM01	Struct							MessagePrefix for EquipmentModuleEM01
5	EM02	Struct							MessagePrefix for EquipmentModuleEM02
6	EM03	Struct							MessagePrefix for EquipmentModuleEM03
7	EM04	Struct							MessagePrefix for EquipmentModuleEM04
8	EM05	Struct							MessagePrefix for EquipmentModuleEM05
9	EM06	Struct							MessagePrefix for EquipmentModuleEM06
10	EM07	Struct							MessagePrefix for EquipmentModuleEM07
11	EM08	Struct							MessagePrefix for EquipmentModuleEM08
12	EM09	Struct							MessagePrefix for EquipmentModuleEM09

The new EM's EventControl block contains four references to the old EM's message prefixes. They are located in networks 12, 16, 17, and 18. Update these references to the new EM.

These message prefixes can now be used in the CM's user program.

If the *disable* bit does not exist in the user program:

Each EM has its own *disable* bit, which deactivates the EM. If the *disable* bit does not exist, it must be created. *Disable* bits are located in the PackMLCfG global DB. Open this DB and open the *disable* structure. Duplicate and rename an existing EM's data structure. The *disable* bit should be defined in the program after this addition.

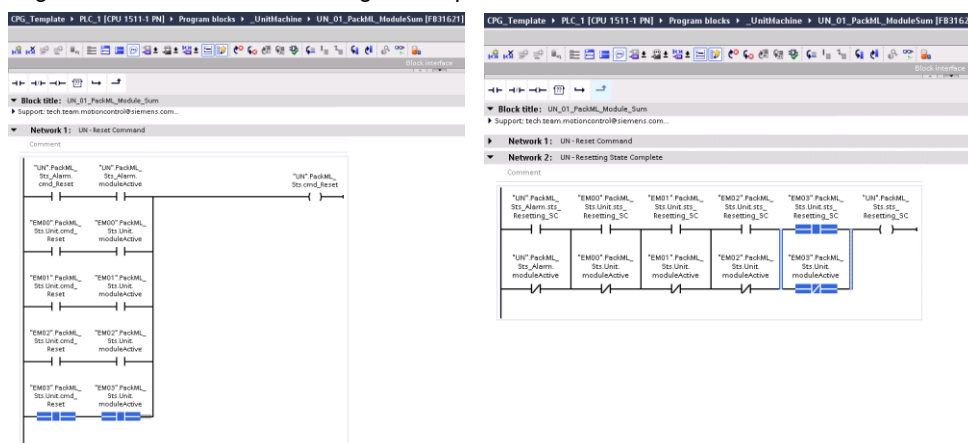
Figure 4-6 PackMLCfG DB with disable bits

CPG_Template > PLC_1 [CPU 1511-1 PN] > Program blocks > GlobalData > Configuration > PackMLCfG [DB31603]									
PackMLCfG									
	Name	Data type	Start value	Retain	Accessible...	Write...	Visible in...	Setpoint	Supervision
1	Static								
2	Cfg	"typePMLV30_Configuration"							PackML Configuration Data
3	Names	"typePMLV30_NamesConfiguration"							PackML Names Data
4	disable	Struct							Bits used to disable TR88 UN & EMs
5	UN	Bool	false						UN (1 = disable, 0 = enable)
6	EM00	Struct							EM00 (1 = disable, 0 = enable)
7	EM01	Struct							EM01 (1 = disable, 0 = enable)
8	EM02	Struct							EM02 (1 = disable, 0 = enable)
9	EM03	Struct							EM03 (1 = disable, 0 = enable)
10	EM04	Struct							EM04 (1 = disable, 0 = enable)
11	EM05	Struct							EM05 (1 = disable, 0 = enable)
12	EM06	Struct							EM06 (1 = disable, 0 = enable)
13	EM07	Struct							EM07 (1 = disable, 0 = enable)
14	EM08	Struct							EM08 (1 = disable, 0 = enable)
15	EM09	Struct							EM09 (1 = disable, 0 = enable)
16	EM10	Struct							EM10 (1 = disable, 0 = enable)

4.1.1 Adding the EM into the module summation

Open the block *UN_01_PackML_ModuleSum*. In this block the EM's state machine commands are combined into a single set of commands for the unit machine. The state machine commands from the new CM will need to be added in each of the networks of this block. Copy and paste the existing logic, copying the patterns already present in the block.

Figure 4-7 Module summation logic examples



4.1.2 Adding the EM into the event summation

Open the block *UN_05_EventSummation*. In this block the event information from all EMs is incorporated into the global Alarm, Warning, and Status DBs. A call of *CPG_EventSummation* must be added for the new EM's alarms, warnings and statuses. The calls must be located in between the calls of *CPG_EventSummationBegin* and *CPG_EventSummationEnd*.

Compile the program. After a successful compile the new EM is integrated and can be used. User logic can be added to the control modules. Additional control modules can be added, per section [4.2](#).

4.2 Adding a new Control Module to an existing EM

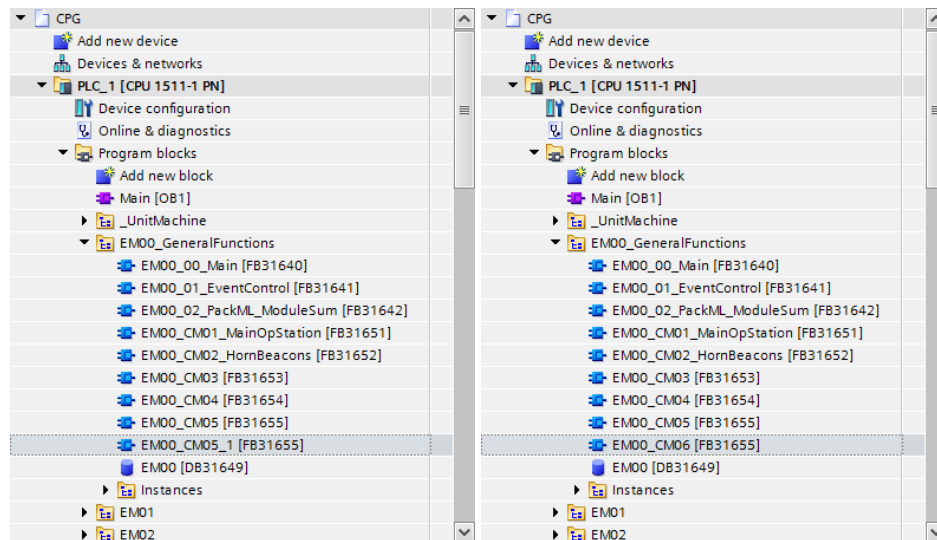
NOTE

As an example, the figures show a sixth control module being added to EM00.

First, navigate to the equipment module in which the new control module will reside. If the EM does not yet exist, refer to section [4.1](#) and create it. Copy any existing control module from the EM, paste it into the EM's folder, and rename it. For future steps in this tutorial, we will refer to the "Main" block (FB31640 in the image), the "ModuleSum" block (FB31642), and the "EM Datablock" (DB31649).

4 Extending the template

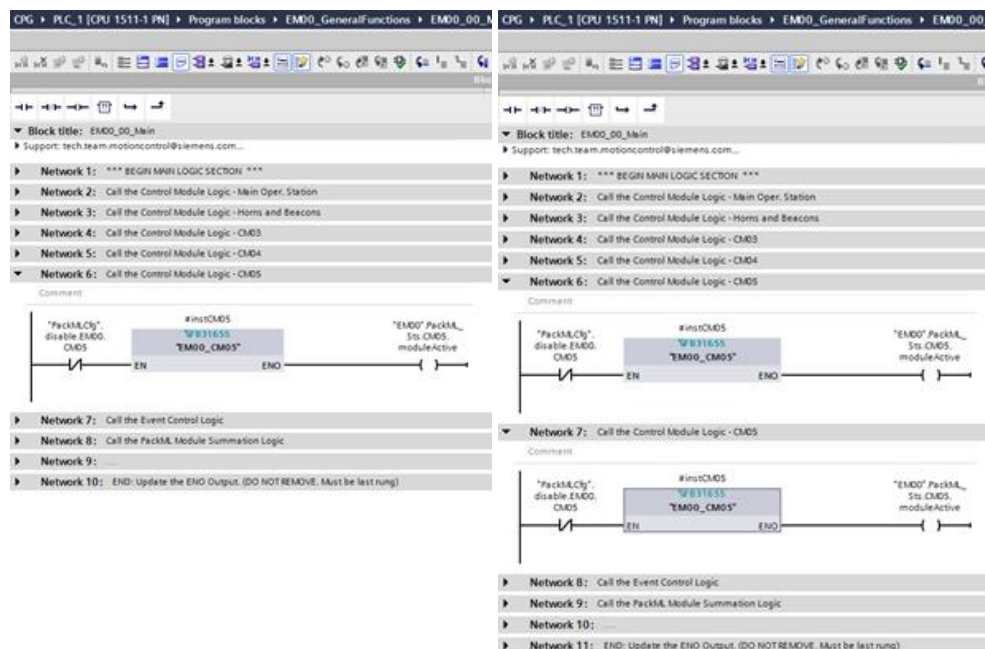
Figure 4-8: Copy and paste an existing control module to make the new CM FB



Open your EM's main block. Each control module FB is called here, and it is necessary to add a call of the new CM. Copy and paste the call of the final CM listed. This duplicate network has 4 separate pieces, all of which need to be updated for the new CM:

1. The network comments
2. The multi-instanced call of the CM
3. The disable bit for the CM
4. The moduleActive bit for the CM

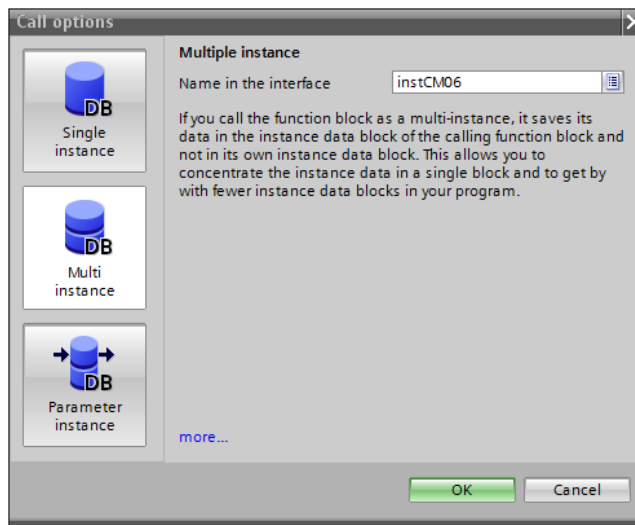
Figure 4-9: Duplicate an existing CM call in EM_Main.



Rename the network comment to anything desired.

Delete the call of the old CM and replace it with a call of the new block. When creating instance data, be sure to select multi-instance.

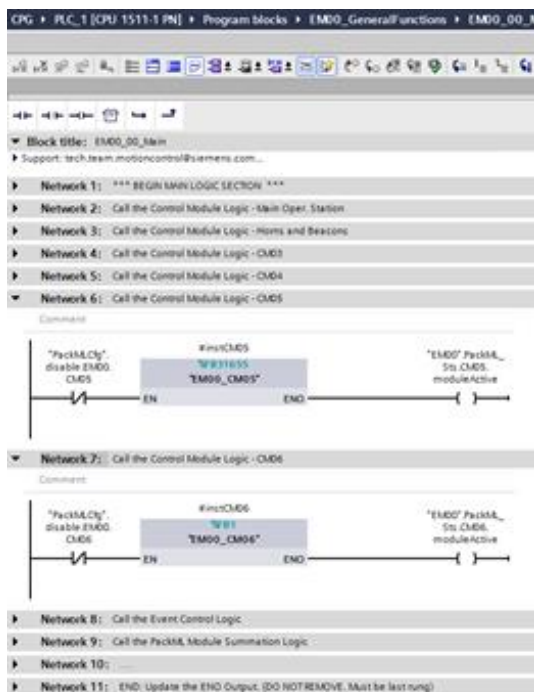
Figure 4-10: Multi-instance the call of the new CM.



Change the control module in the name of *disable* bit from the old CM to the new one - in this example CM05 is replaced with CM06 in the copied network. It is possible this bit will not exist, depending on how many CMs are present in the project already.

Repeat this replacement for the *moduleActive* bit.

Figure 4-11: Call of the new CM, with tags edited.



Attempt to compile the software program. If prompted, renumber any blocks. If the program does not compile continue following the instructions below to create the missing tags.

If the compile was successful, it is now possible to write user logic in the new CM. Any state machine commands can be sent using the status bits for the new CM. Alarms can be added, and the CM can be given a unique message prefix.

NOTE

If user logic existed in the old CM before it was copied, any references to these status bits or message prefixes will require updating!

4.2.1 Creating supporting tags for the new CM

Each CM has its own *disable* bit, which deactivates the CM. If the *disable* bit does not exist, it must be created. *Disable* bits are located in the PackMLCfg global DB. Open this DB and open the *disable* structure. The structure contains all its CM's disable bits. Add a new Boolean tag for the new CM.

Figure 4-12: *Disable* bits for the modules.

Name	Data type	Start value	Retain	Accessible	Write	Visible	Setpoint	Supervision	Comment
CM01	Bool	false							ENM01 (1 = disable, 0 = enable)
CM02	Bool	false							ENM02 (1 = disable, 0 = enable)
CM03	Bool	false							ENM03 (1 = disable, 0 = enable)
CM04	Bool	false							ENM04 (1 = disable, 0 = enable)
CM05	Bool	false							ENM05 (1 = disable, 0 = enable)
CM06	Bool	false							ENM06 (1 = disable, 0 = enable)
CM07	Bool	false							ENM07 (1 = disable, 0 = enable)
CM08	Bool	false							ENM08 (1 = disable, 0 = enable)
CM09	Bool	false							ENM09 (1 = disable, 0 = enable)
CM10	Bool	false							ENM10 (1 = disable, 0 = enable)

Each CM has its own set of “status bits”, which are used to control the state machine. They consist of the full set of state machine commands which are sent to the UN to start and stop the entire machine. They also include the moduleActive bit. If these status bits do not exist, they must be created.

Open the equipment module's data block. The status bits are located in the *PackML_Sts* structure. Copy, paste, and rename a tag of type *CPG_typePackML_ModuleSts*. The bits used in the *Main* block should exist after performing this operation.

4 Extending the template

Figure 4-13: Status bits for EM00.

Name	Data type	Start value	Retain	Accessible F.	Write	Visible in	Setpoint	Supervision	Comment
EM00									
Static									
PackAA_Sts	Struct								PackAA Module Status - EM and all control m...
Unit	"CPG_typePackAA_ModuleSts"								PackAA Module Status - EM-master status
Alarm	"CPG_typePackAA_ModuleSts"								PackAA Module Status - alarm status
CM01	"CPG_typePackAA_ModuleSts"								PackAA Module Status - CM01
CM02	"CPG_typePackAA_ModuleSts"								PackAA Module Status - CM02
CM03	"CPG_typePackAA_ModuleSts"								PackAA Module Status - CM03
CM04	"CPG_typePackAA_ModuleSts"								PackAA Module Status - CM04
CM05	"CPG_typePackAA_ModuleSts"								PackAA Module Status - CM05
CM06	"CPG_typePackAA_ModuleSts"								PackAA Module Status - CM06
CM07	"CPG_typePackAA_ModuleSts"								PackAA Module Status - CM07
CM08	"CPG_typePackAA_ModuleSts"								PackAA Module Status - CM08
Alarm	"CPG_typeEventStatus"								Alarm Event Status - EM00 Status
Warning	"CPG_typeEventStatus"								Warning Event Status - EM00 Status
Status	"CPG_typeEventStatus"								Event Status - EM00 Status

If the *status* bits did not exist previously, it will be necessary to add them into the EM's *ModuleSum* block. The state machine commands from the new CM will need to be added in each of the networks. Copy and paste the existing logic, copying the patterns already present in the block.

Figure 4-14: Module sum for networks with commands (Reset, Stop, Start, etc.)

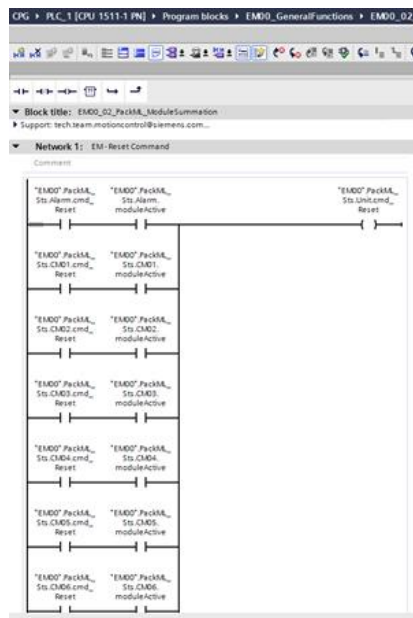
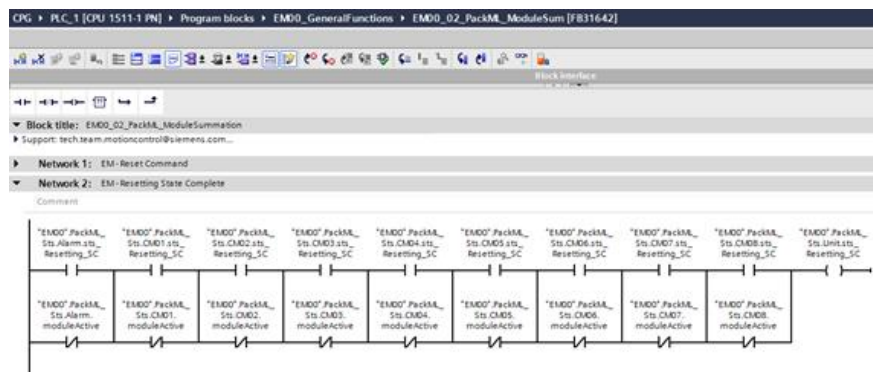


Figure 4-15: Module sum for networks with SC bits (Starting_SC, Stopping_SC, etc.)



If the *status* bits did not exist, it is also necessary to add message prefixes for the new CM. Open the config DB *MessagePrefix*. In the containing EM's structure, add a new tag of *CPG_typeMessagePrefix*, and give it the name of the CM. These message prefixes can now be used in the CM's user program.

Figure 4-16: Message prefixes for all modules

Name	Data type	Start value	Retain	Accessible f.	Write	Visible in	Segment	Supervision	Comment
1. Status	Struct								MessagePrefix for Unit
2. Unit	Struct								MessagePrefix for EquipmentModuleEM00
3. EventControl	Struct								MessagePrefix for EventControl
4. CM01	"CPG_typeMessagePrefix"								MessagePrefix for ControlModule#01
5. CM02	"CPG_typeMessagePrefix"								MessagePrefix for ControlModule#02
6. CM03	"CPG_typeMessagePrefix"								MessagePrefix for ControlModule#03
7. CM04	"CPG_typeMessagePrefix"								MessagePrefix for ControlModule#04
8. CM05	"CPG_typeMessagePrefix"								MessagePrefix for ControlModule#05
9. CM06	"CPG_typeMessagePrefix"								MessagePrefix for ControlModule#06
10. CM07	"CPG_typeMessagePrefix"								MessagePrefix for ControlModule#07
11. CM08	"CPG_typeMessagePrefix"								MessagePrefix for ControlModule#08
12. CM09	"CPG_typeMessagePrefix"								MessagePrefix for ControlModule#09
13. CM10	"CPG_typeMessagePrefix"								MessagePrefix for ControlModule#10
14. EM01	Struct								MessagePrefix for EquipmentModuleEM01
15. EM02	Struct								MessagePrefix for EquipmentModuleEM02
16. EM03	Struct								MessagePrefix for EquipmentModuleEM03
17. EM04	Struct								MessagePrefix for EquipmentModuleEM04
18. EM05	Struct								MessagePrefix for EquipmentModuleEM05
19. EM06	Struct								MessagePrefix for EquipmentModuleEM06
20. EM07	Struct								MessagePrefix for EquipmentModuleEM07
21. EM08	Struct								MessagePrefix for EquipmentModuleEM08
22. EM09	Struct								MessagePrefix for EquipmentModuleEM09

Compile the program. If prompted, renumber any blocks. The program should compile. If it does not, review the previous steps and ensure everything is created and properly named. It is now possible to write user logic in the new CM. Any state machine commands can be sent using the status bits for the new CM. Alarms can be added, and the CM can be given a unique message prefix.

NOTE

If user logic existed in the old CM before it was copied, any references to these status bits or message prefixes will require updating!

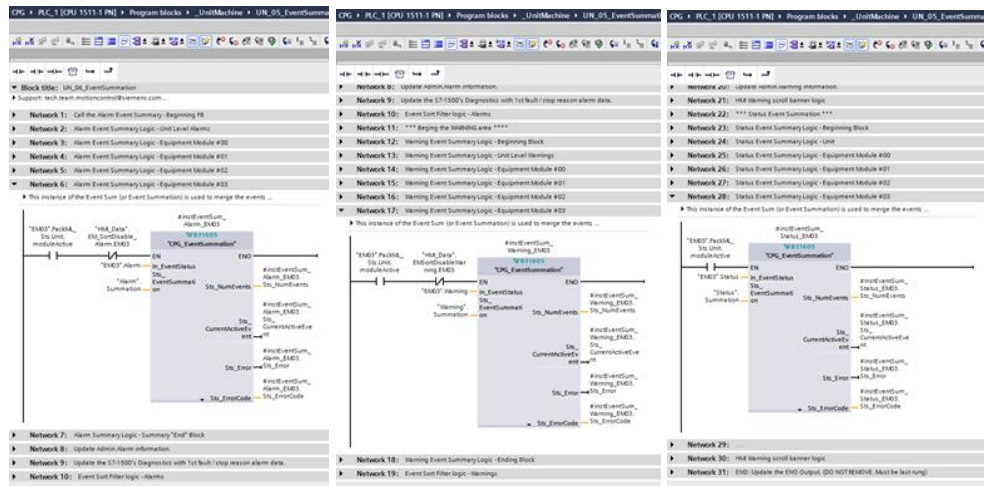
Figure 4-17 Call positions of the new event summation blocks inside *UN_06_EventSummation*

▶ Network 1:	Call the Alarm Event Summary - Beginning FB
▶ Network 2:	Alarm Event Summary Logic - Unit Level Alarms
▶ Network 3:	Alarm Event Summary Logic - Equipment Module #00
▶ Network 4:	Alarm Event Summary Logic - Equipment Module #01
▶ Network 5:	Alarm Event Summary Logic - Equipment Module #02
▶ Network 6:	Alarm Event Summary Logic - Equipment Module #03
▶ Network 7:	Alarm Summary Logic - Summary "End" Block
▶ Network 8:	Update Admin.Alarm information.
▶ Network 9:	Update the S7-1500's Diagnostics with 1st fault / stop reason alarm data.
▶ Network 10:	Event Sort Filter logic - Alarms
▶ Network 11:	*** Beging the WARNING area ***
▶ Network 12:	Warning Event Summary Logic - Beginning Block
▶ Network 13:	Warning Event Summary Logic - Unit Level Warnings
▶ Network 14:	Warning Event Summary Logic - Equipment Module #00
▶ Network 15:	Warning Event Summary Logic - Equipment Module #01
▶ Network 16:	Warning Event Summary Logic - Equipment Module #02
▶ Network 17:	Warning Event Summary Logic - Equipment Module #03
▶ Network 18:	Warning Event Summary Logic - Ending Block
▶ Network 19:	Event Sort Filter logic - Warnings
▶ Network 20:	Update Admin.Warning information.
▶ Network 21:	HMI Warning scroll banner logic
▶ Network 22:	*** Status Event Summation ***
▶ Network 23:	Status Event Summary Logic - Beginning Block
▶ Network 24:	Status Event Summary Logic - Unit
▶ Network 25:	Status Event Summary Logic - Equipment Module #02
▶ Network 26:	Status Event Summary Logic - Equipment Module #02
▶ Network 27:	Status Event Summary Logic - Equipment Module #02
▶ Network 28:	Status Event Summary Logic - Equipment Module #03

Copy and paste existing calls of *CPG_EventSummation* into the new positions. Be sure to update:

1. The *moduleActive* bit
2. The HMI_Data sortDisable bit (this bit may or may not exist in the program at this point)
3. The multi-instanced call data of *CPG_EventSummation*
4. The new EM's Alarm/Warning/Status data connected to *In_EventStatus*
5. The Alarm/Warning/Status information connected to *EventSummation* can remain the same
6. The outputs of *CPG_EventSummation* are optional

Figure 4-18: The three calls of CPG_EventSummation



If the HMI sortDisable bit does not exist in the user program:

Each EM has its own “sort” bit, which activates alarm filtering from the HMI. There are two separate bits, one for sorting alarms and one for sorting warnings. If the sort bits do not exist, they can optionally be created. Sort bits are located in the *HMI_Data* global DB. Open this DB and open the *EM_SortDisable_Alarm* structure for alarm sorting and the *EM_SortDisable_Warning* structure for warnings. Duplicate and rename an existing EM's data structure in both structures. The sort bits should be defined in the program after this addition.

Figure 4-19: The HMI_Data structures containing the sortDisable bits.

Name	Data type	Start value	Retain	Accessible from	Visible in	Signature	Supervision	Comment
resetPMA_Times	Bool	false						Reset the PMA timer values when set (1).
languageSelected	Int	0						This is the value of the language selected in th
modeCurrentTime	Time	0						Time spent in the current mode (seconds)
modeCumulativeTime	Time	0						Accumulative Time spent in the current mode
State_Current_Times	Array[0..7, PMA_VOS_...]							Current Time (s of seconds) spent for each st
State_Cumulative_Times	Array[0..7, PMA_VOS_...]							Accumulative Time (s of seconds) spent for e...
alarmSortDisable	Bool	false						Alarm Sort Disable for HMI - summary of Act...
EM_SortDisable_Alarm	Struct							Alarm Disable for Equipment Modules (EM)
Unit	Bool	false						If bool is reset, then Unit sort filter is enabled...
EM00	Bool	false						If bool is reset, then EM00 sort filter is enabled...
EM01	Bool	false						If bool is reset, then EM01 sort filter is enabled...
EM02	Bool	false						If bool is reset, then EM02 sort filter is enabled...
EM03	Bool	false						If bool is reset, then EM03 sort filter is enabled...
EM04	Bool	false						If bool is reset, then EM04 sort filter is enabled...
EM05	Bool	false						If bool is reset, then EM05 sort filter is enabled...
EM06	Bool	false						If bool is reset, then EM06 sort filter is enabled...
EM07	Bool	false						If bool is reset, then EM07 sort filter is enabled...
EM08	Bool	false						If bool is reset, then EM08 sort filter is enabled...
EM09	Bool	false						If bool is reset, then EM09 sort filter is enabled...
EM10	Bool	false						If bool is reset, then EM10 sort filter is enabled...
alarmFilter	Bool	false						If bool is reset, then alarm filter is enabled...
alarmFilter	Bool	false						HMI Commands to control the Alarm Sort Filter
warningSortDisable	Bool	false						Warning Sort Disable for HMI - summary of Act...
EM_SortDisable_Warning	Struct							Warning Disable for Equipment Modules (EM)
Unit	Bool	false						If bool is reset, then Unit sort filter is enabled...
EM00	Bool	false						If bool is reset, then EM00 sort filter is enabled...
EM01	Bool	false						If bool is reset, then EM01 sort filter is enabled...
EM02	Bool	false						If bool is reset, then EM02 sort filter is enabled...
EM03	Bool	false						If bool is reset, then EM03 sort filter is enabled...
EM04	Bool	false						If bool is reset, then EM04 sort filter is enabled...
EM05	Bool	false						If bool is reset, then EM05 sort filter is enabled...
EM06	Bool	false						If bool is reset, then EM06 sort filter is enabled...
EM07	Bool	false						If bool is reset, then EM07 sort filter is enabled...
EM08	Bool	false						If bool is reset, then EM08 sort filter is enabled...
EM09	Bool	false						If bool is reset, then EM09 sort filter is enabled...
EM10	Bool	false						If bool is reset, then EM10 sort filter is enabled...
warningFilter	Bool	false						If bool is reset, then warning filter is enabled...
warningFilter	Bool	false						HMI Commands to control the Warning Sort F...
modeNames	Array[0..7, PMA_VOS_...]							Mode Names for selected language
StateNames	Array[0..7, PMA_VOS_...]							State Names for selected modes
PLCStateTime	Time	0						Date Time for PLC State Time

NOTE

If user logic existed in the EM (or its CMs) before it was copied, any references to these status bits or message prefixes will require updating!

5 Additional information

5.1 Tips and Recommendations

5.1.1 Template Modifications

The CPG Template is provided as open-source and can be modified to fulfill unique customer requirements. Some modifications are typical, they include the following:

- Supporting of Unicode characters (Asian, Cyrillic, Arabic, etc.) in event (alarm, warning, and status) text or mode & state names, etc.
- Reducing the memory requirements - at the expense of some alarming filtering capability - to optimize the CPG Template on smaller CPU's
- Implementing one or more EMs or CMs on a remote PLC

If any of these changes are needed, then it may be necessary to modify the template yourself or contact the Siemens APC.

5.1.2 Global Memory

Global Memory is referenced within the CPG Template's Equipment Module (EM) and Control Module (CM) function calls (FCs) and function blocks (FBs) as this allows all data to be cross-referenced by TIA Portal. Data that is passed as inputs, outputs, or input/outputs to function calls (FC) and function blocks (FB) cannot be cross-referenced.

5.1.3 Number of Event Languages

To reduce the amount work memory required by the PLC, set the number of event languages to the smallest value required for the application. The CPG Constant `LPMLV30_LANGUAGES_UPPER_LIM` PLC tag is used to set the number of event languages and is the number of desired event languages – 1. The CPG Template PLC constants are found in *PLC > PLC tags > LPMLV30_Constants & CPG_Constants* area.

If only 1 event language is required then `LPMLV30_LANGUAGES_UPPER_LIM = 0`, if 3 event languages are required then `LPMLV30_LANGUAGES_UPPER_LIM = 2`.

6 Appendix

6.1 Service and support

Industry Online Support

Do you have any questions or need assistance?

Siemens Industry Online Support offers round the clock access to our entire service and support know-how and portfolio.

The Industry Online Support is the central address for information about our products, solutions and services.

Product information, manuals, downloads, FAQs, application examples and videos – all information is accessible with just a few mouse clicks:

support.industry.siemens.com

Technical Support

The Technical Support of Siemens Industry provides you fast and competent support regarding all technical queries with numerous tailor-made offers – ranging from basic support to individual support contracts. Please send queries to Technical Support via Web form:

www.siemens.com/industry/supportrequest

SITRAIN – Training for Industry

We support you with our globally available training courses for industry with practical experience, innovative learning methods and a concept that's tailored to the customer's specific needs.

For more information on our offered trainings and courses, as well as their locations and dates, refer to our web page:

www.siemens.com/sitrain

Service offer

Our range of services includes the following:

- Plant data services
- Spare parts services
- Repair services
- On-site and maintenance services
- Retrofitting and modernization services
- Service programs and contracts

You can find detailed information on our range of services in the service catalog web page:

support.industry.siemens.com/cs/sc

Industry Online Support app

You will receive optimum support wherever you are with the "Siemens Industry Online Support" app. The app is available for Apple iOS, Android and Windows Phone:

support.industry.siemens.com/cs/ww/en/sc/2067

6.2 Application Support

Siemens AG
 Digital Industries
 Factory Automation
 Production Machines
 DI FA PMA APC
 Frauenauracher Str. 80
 91056 Erlangen, Germany
 mailto: tech.team.motioncontrol@siemens.com

6.3 Links and literature

Table 6-1

No.	Topic
\1\	Siemens Industry Online Support https://support.industry.siemens.com
\2\	Link to this entry page of this application example https://support.industry.siemens.com/cs/ww/en/view/109475572
\3\	LPMLV30 https://support.industry.siemens.com/cs/ww/en/view/49970441
\4\	ANSI/ISA-TR88.00.02-2015 https://www.isa.org/
\5\	OMAC http://www.omac.org
\6\	PDI https://support.industry.siemens.com/cs/ww/en/view/86302104

6.4 Change documentation

Table 6-2

Version	Date	Modifications
V1.0	03/2019	First version